

Verification Continuum™

Verdi® UPF Architect User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

1. Preface	8
Customer Support	8
Synopsys Statement on Inclusivity and Diversity	9
2. Introduction	10
Overview	10
Supported Platforms	10
Setting Up the UPFA Tool	11
3. Using the UPFA Tool	12
Invoking the UPFA Tool	12
Use Model	12
Arguments	12
UPFA Tcl Commands	13
Commands for Library Files	13
ugo_search_path	13
ugo_link_library	14
Commands for HDL	14
ugo_vlog_analyze	14
ugo_vhdl_analyze	15
ugo_elaborate	15
ugo_save_vclp_session	16
ugo_restore_vclp_session	16
Command for Upgrading Message Severity	17
ugo_set_rule_severity	17
Commands for Power Intent	19
ugo_read_power_intent	19
ugo_remove_power_intent	19
ugo_check_power_intent	20
ugo_report	20
ugo_set_var	22
ugo_generate	23

ugo_include_vclp_script	26
ugo_optimize_power_intent	27
4. UPFA Power Intent	31
CSV Format	32
CSV Syntax	34
User-Defined Comment in the Header of Table	35
User-Defined UPF Comment in the Header of Table	35
User-Defined Variable	36
User-Defined Script	39
Power Domain	40
Supply Port Intent	42
Power Domain Intent	43
Supply	48
Supply Map	51
Switchable Supply	51
Power Switch Model	53
Power Switch Strategy	54
Logic Network	58
Power Spec	59
Power Spec Intent	63
State Transition	64
Isolation	65
Level Shifter	71
Retention	75
Repeater	78
Load UPF	80
Connection	82
Port Map	83
Parameters	86
Apply Power Model	86
Attribute	87
LSGEN Config	91

Contents

UPF Patch	92
Global PST Template	94
Global Supply Template	100
Design Hierarchy	105
Set Equivalent	106
ISOGEN Config	107
Scope Condition	109
Special Syntax	111
The if-else Structure Support	111
SET_SCOPE Support	113
Foreach support	114
For Loop Statement Support	115
Partition UPF Integration	116
Key points to Note:	121
Example	121
5. UPFA Power Intent Editor GUI	123
Viewing the UPFA Power Intent Editor GUI	123
File View	126
Viewing Power Intent of the Generated UPF File	132
Detail View	135
Console	151
Support for Special Rules in CSV Format	151
Menu Options	153
Tcl Options	154
ugoReloadFile	155
ugoFileViewDump	155
ugoDetailViewDump	156
upfaClearConsole	156
ugoDetailViewSectionMoved	157
upfaSetPreference	157
ugoDetailViewRow	159
ugoDetailViewColumn	159
ugoDetailViewInsertCopiedRows	160
ugoDetailViewInsertCopiedColumns	161

ugoDetailViewCut	162
ugoDetailViewInsertCutRows	162
ugoDetailViewInsertCutColumns	163
ugoShowUPFInVerdi	163
6. Implicit Naming Rule	165
Supply net	165
Supply port	165
Power Switch	166
Isolation Naming	166
Level Shifter Naming	166
7. Inherit Values for CSV Column	167
Power Domain Intent: NAME	167
Power Switch Model: MODEL_NAME	168
Power Switch Strategy: SW_NAME	169
Connection: SCOPE, IS_RTL	170
Port Map: NAME	170
8. Simple CPU Design With UPFA	172
9. Known Issues and Limitations	175
General Limitations	175
User-Defined Variable Limitations	175
10. Appendix A: Tcl commands to Specify Detail Categories in Constraint File . .	177
Introduction	177
Use Model	177
Tcl Commands	178
create_category	178
gen_upf	179
Example	180
Specifying Category Using UPF Commands	181

Contents

create_supply_set	181
connect_supply_net	183
Priority	184
Error and Warning Handling	184
UPF Commands Supported by UPFA	185
Constraint Variables	187
11. Appendix B: ISOGEN Native Support in UPFA	191
Introduction	191
Prerequisites	191
Syntax	191
Inputs	193
Usage Model	193
Output	193
Isolation Configuration File Format	194
Hierarchical Isolation Configuration File Format	197

1

Preface

The Verdi UPF Architect (UPFA) tool automatically generates the UPF based on the latest UPF syntax and suggested coding style supported by Synopsys low power tools.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

Product L1 as Verdi

Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

2

Introduction

Overview

Power intent for a design is captured using the UPF (IEEE P1801). Generating an individual UPF for each design instance exponentially increases the generated file size. For a large design, it becomes overhead for the tools to handle it.

The Verdi UPF Architect (UPFA) tool automatically generates the UPF based on the latest UPF syntax and suggested coding style supported by Synopsys low power tools. For example, the UPFA tool can generate UPF 2.0, 2.1, or 3.0 syntax with same input.

The UPFA tool allows you to maintain single power intent for different tools or the UPF versions and improves the UPF overhead in compilation, simulation, and emulation stages. It also checks the power intent correctness with or without RTL and reports missing isolation strategies (VC Static tool is required for this feature).

Supported Platforms

The following table shows the platforms and operating systems supported by the Verdi-UPFA platform for Synopsys S-2021.09 release:

Table 1 Verdi-UPFA Supported Platforms

Platforms	Bits	Baseline Operating System	Binary Compatible Operating System	Architecture
Linux64	64	CentOS 7.3.1611	Centos: 7.3.1611+ RHEL: 7.3+ SLES: 12-SP4	X86_64

Note:

- 32-bit platforms are not supported
- CentOS and RHEL 8+ operating systems are not supported
- ARM platform is not supported

Setting Up the UPFA Tool

To set up the UPFA tool, you need to set the `VERDI_HOME` variable to your UPFA install path. The following is an example to setup TD UPFA in Synopsys environment:

```
setenv VERDI_HOME /remote/vtgimages/SAFE/linux64_TD.VERDI/release-  
structure/verdi
```

To support HDL accessing, the `VC_STATIC_HOME` variable is required for the following commands:

- `ugo_vlog_analyze`
- `ugo_vhdl_analyze`
- `ugo_elaborate`
- `ugo_include_vclp_script`
- `ugo_optimize_power_intent`

3

Using the UPFA Tool

The following sections describe how to invoke and use the Verdi UPFA tool:

- [Invoking the UPFA Tool](#)
- [UPFA Tcl Commands](#)

Invoking the UPFA Tool

You can use the `upfa` compile option to invoke the UPFA tool. In the command mode, UPFA uses Tcl (Tool Command Language) as user interface, therefore you can use *Normal Tcl Commands*.

The following section describes the UPFA Tcl commands:

- [Use Model](#)
- [Arguments](#)

Use Model

```
upfa [-batch] [-db <dir_name>] [-file <file_name>] [-help] [-id] [-log  
<file_name>]
```

Arguments

`-batch`

Starts the tool in the batch mode (non-interactive).

`-db`

Specifies the name of database directory. Default directory is `ugo.db` in the current directory.

`-file`

Specifies the name of the script file to execute after setup.

`-help`

Prints help message.

`-id`

Prints the build/environment information of the tool.

`-log`

Specifies the name of log file. Default file name is `ugo.log` in the current directory.

UPFA Tcl Commands

The following sections describe the UPFA Tcl commands:

- [Commands for Library Files](#)
- [Commands for HDL](#)
- [Command for Upgrading Message Severity](#)
- [Commands for Power Intent](#)

Commands for Library Files

Following are the Tcl commands for library files:

- [ugo_search_path](#)
- [ugo_link_library](#)

ugo_search_path

Description

Specifies a list of directories where the tool searches for library (.db), design, or UPF files.

Syntax

```
ugo_search_path <dir_list>
```

Arguments

<dir_list>

Specifies all the directories where the library, design, or UPF files must be searched. The path may be absolute or relative to the working directory of the tool.

Value Returned

Returns 1 always.

Example

```
ugo_search_path {. ./rtl ./upf}
```

ugo_link_library

Description

Specifies a list of library (.db) files.

Syntax

```
ugo_link_library <db_list>
```

Arguments

<db_list>

Specifies all the library (.db) files.

Value Returned

Returns 1 always.

Example

```
ugo_link_library {lib1.db lib2.db}
```

Commands for HDL

Following are the Tcl commands for HDL:

- [ugo_vlog_analyze](#)
- [ugo_vhdl_analyze](#)
- [ugo_elaborate](#)
- [ugo_save_vclp_session](#)
- [ugo_restore_vclp_session](#)

ugo_vlog_analyze

Description

Analyzes Verilog or SystemVerilog files.

Syntax

```
ugo_vlog_analyze {<vlogan_options>}
```

Arguments

<vlogan_options>

Uses the options same as `vlogan`.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_vlog_analyze {-f run.f -sverilog}
```

ugo_vhdl_analyze

Description

Analyzes VHDL files.

Syntax

```
ugo_vhdl_analyze {<vhdlan_options>}
```

Arguments

<vhdlan_options>

Use the options as same as `vhdlan`.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_vhdl_analyze {-f run.f -vhdl08}
```

ugo_elaborate

Description

Elaborates HDL.

Syntax

```
ugo_elaborate -top {[<library name>.<top module>]*} [{<vcs_options>}]
```

Arguments

<library name>

Specifies the library of top module.

<top module>

Specifies the top module for elaboration.

<vcs_options>

Specifies the additional options for elaboration.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_elaborate -top {TB_LIB.tb1 TB_LIB.tb2}
```

ugo_save_vclp_session

Description

Saves a VCLP session. The background VCLP tries to save a session. The session is a directory and can be restored by VCLP or UPFA. This command can be used to save the RTL design session.

Syntax

```
ugo_save_vclp_session [-session <session_name>]
```

Arguments

-session <session_name>

Specifies the session name.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_save_vclp_session [-session <ses1>]
```

ugo_restore_vclp_session

Description

Restores a VCLP session. The background VCLP tries to load the session.

Syntax

```
ugo_restore_vclp_session [-session <session_name>]
```

Arguments

-session <session_name>

Specifies the session name.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_restore_vclp_session [-session <ses1>]
```

Command for Upgrading Message Severity

Following is the Tcl command for upgrading message severity:

- [ugo_set_rule_severity](#)

ugo_set_rule_severity

Description

Modifies the severity of the specified message. It does not change the name of the message.

Syntax

```
ugo_set_rule_severity  
[-error <message names>] \  
[-warning <message names >] \  
[-phase <phase>] \  
[-clear]
```

Arguments

-error

Specifies the new severity value.

<message names> lists the warning/information message names or patterns. Patterns can include the wildcard characters “*” and “?”. When multiple names or patterns are specified, they should be enclosed with “ ” or {}.

-warning

Specifies the new severity value. Valid values are parser, database, generator, optimizer, and upf2csv.

<message name> lists the information message names or patterns. Patterns can include the wildcard characters “*” and “?”. When multiple names or patterns are specified, they should be enclosed with “ ” or {}.

-phase

Specifies the phase of the given message to upgrade severity.

`-clear`

Clears all new severity rules of messages.

Note:

If the non-existing message name matches with the specified message name or pattern, UPFA reports the Info message.

Input

`upgrade_severity.tcl` – Collection of the `ugo_set_rule_severity` commands to define the new severity of the message.

Use Model

Usage Model-1:

```
upfa -f upgrade_severity.tcl -gui
```

Usage Model-2:

```
ugo_set_rule_severity ... ### you can set rule at any time
ugo_read_power_intent ...
ugo_set_rule_severity ...
ugo_check_power_intent
ugo_generate ...
```

Example

The following example sets the severity for `TCL_VAR_NOT_DEFINED_WARNING` and `FILE_NOT_FOUND_WARNING` to error:

```
ugo_set_rule_severity -error { TCL_VAR_NOT_DEFINED_WARNING
FILE_NOT_FOUND_WARNING } -phase database
```

Output:

```
Error-[database][FILE_NOT_FOUND_WARNING] File not found test.csv, 17
File 'partition.upf' cannot access: No such file.
Please specify a valid file.
Error -[database][TCL_VAR_NOT_DEFINED_WARNING] Tcl variable is not
defined test.csv, 30
Tcl variable '${AAA}' is used but not found in user-defined variable
section.
Please consider to define a new item in user-defined variable section.
database: 2 error(s), 0 warning(s), 0 info
```

Commands for Power Intent

Following are the Tcl commands for power intent:

- [ugo_read_power_intent](#)
- [ugo_remove_power_intent](#)
- [ugo_check_power_intent](#)
- [ugo_report](#)
- [ugo_set_var](#)
- [ugo_generate](#)
- [ugo_include_vclp_script](#)
- [ugo_optimize_power_intent](#)

ugo_read_power_intent

Description

Reads the power intent from several forms.

Syntax

```
ugo_read_power_intent -csv <file name>
```

Arguments

-csv

Reads the power intent based on CSV format.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_read_power_intent -csv pd.csv
```

ugo_remove_power_intent

Description

Removes the current power intent.

Syntax

```
ugo_remove_power_intent
```

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_read_power_intent -csv cpu1.csv
ugo_check_power_intent
ugo_report -error -file cpu1.err
ugo_remove power_intent
ugo_read_power_intent -csv cpu2.csv
ugo_check_power_intent
ugo_report -error -file cpu2.err
```

ugo_check_power_intent

Description

Checks if there is any issue in the current power intent.

Syntax

```
ugo_check_power_intent
[-check_condition]
```

Where, `-check_condition` enables you to check the `UGO_CONDITION_IF` structure intention.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_read_power_intent -csv pd.csv
ugo_read_power_intent -csv ss.csv
# ... some ugo_read_power_intent commands
ugo_check_power_intent
```

ugo_report

Description

Displays detailed UPFA report.

Syntax

```
ugo_report [-error] [-warning] [-info] [-phase <phase_name>] [-file
<file_name>]
```

Arguments

-error

Displays error message.

-info

Displays information message.

-warning

Displays warning message.

-phase

You can use one of the following phase types:

- `parser`
- `database`
- `generator`

-file

Writes the messages to `<file_name>`

If this option is not specified, messages are displayed on console.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

- `ugo_report`

Shows the summary of all messages. Following is the example report:

```
parser: 0 error(s), 0 warning(s), 0 info(s)
database: 3 error(s), 1 warning(s), 0 info(s)
generator: 0 error(s), 0 warning(s), 0 info(s)
```

- `ugo_report -error`

Displays the detailed error message report on the console. The following steps describe the format of the message:

1. Format:

```
Severity-[phase][tag] Short description
formal description
[what next]
```

2. The line of short description has no indentation. The first letter of severity should be uppercase. Phase should be in lowercase. Tag should be in uppercase. First letter of the first word should be in uppercase, and others in lowercase.

3. All lines of formal description should have 2 indentation. If the formal description contains file name and line number, it should be specified in the first separated line without indentation.
4. What next section is optional. It should have 2 indentation.

Example:

```
Error-[database][TCL_VARIABLE_NOT_DEFINED] Tcl variable is
not defined
test.csv, 9
Tcl variable '$t1' is not checked without design.
Consider defining a new item in user-defined variable
section.
```

ugo_set_var

Description

Allows you to set up Tcl variable in the UPFA session. The Tcl variable setup is available in the UPF generation flow.

Syntax

```
ugo_set_var <variable name> <value>
```

Arguments

Following are some special Tcl variables you can set:

```
escape_hdl_index 1
```

UPFA replaces [to \[and] to \] for HDL index escaping for the generated UPF.

```
collection_result_display_limit <number>
```

Limit the result displayed for VCLP.

```
vclp_tmp_upf_dir <path>
```

Setup tmp_sub.upf path for VCLP check.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_set_var escape_hdl_index 1
ugo_set_var SIM 1
```

ugo_generate

Description

Generates the UPF based on the current power intent.

Syntax

```
ugo_generate
-upf <file name>
[-version <upf version>]
[-apr <file name>]
[-stub <file name>]
[-constraint <file name>]
[-gen_spec <apr_spec>]
[-no_implicit_supply_function]
[-disable_opt_pst]
[-flatten_connection]
[-lrm]
[-bias_ref]
[-iso_config <file name>]
[-iso_info <file name>]
[-iso_constraint <file name>]
[-hier_iso_config <file name>]
[-include_pst]
[-exclude_pst]
[-put_begin_path]
[-disjoin_connection]
[-disable_sort_supply_function]
[-prefix <prefix>]
```

Arguments

-upf

Allows you to specify the file name for the generated UPF.

-version <2.0|2.1|3.0>

Allows you to specify the UPF version for the generated UPF. For example, -version 2.0.

-apr

Allows you to specify the name for APR refine file.

-stub

Specify the name for the STUB UPF file. This file only contains the following UPF commands:

- upf_version
- set

- `find_objects`
- `source`
- `create_supply_port`
- `create_supply_net`
- `connect_supply_net:top_port`
- `create_supply_set`
- `create_power_domain`
- `set_port_attributes`
- `set_related_supply_net`

-constraint

Allows you to specify the constraint file. see *Appendix A: Tcl commands to Specify Detail Categories in Constraint File*.

-gen_spec <0 | 1 | 2 | 3 | 4>

value = 0, nwell/pwell/ground is in RTL, power is in APR

value = 1, nwell/pwell is in RTL, power/ground is in APR

value = 2, all (nwell/pwell/ground/power) is in APR

value = 3, all (nwell/pwell/ground/power) is in RTL

value = 4, power/ground is in RTL, nwell/pwell is in APR

-no_implicit_supply_function

Tool does not generate implicit supply function unspecified by the user.

-disable_opt_pst

Disables the power state table optimization feature.

-flatten_connection

Flattens foreach syntax to individual command for the [Connection] section.

-lrm

Generates UPF by strictly following the UPF LRM.

-bias_ref

If nwell and pwell are not specified, then nwell follows power and pwell follows ground.

`-iso_config <file name>`

Generates the isolation configuration file for the ISOGEN flow. You must specify the required intention in the [ISOGEN Config] section.

`-iso_info <file name>`

Generates the `isolation.info` file for the ISOGEN flow. You must specify the required intention in the [ISOGEN Config] section.

`-iso_constraint <file name>`

Generates isolation constraint file for the ISOGEN flow. You must specify the required intention in the [Scope Condition] section.

`-hier_iso_config <file name>`

Generates isolation configuration file for the ISOGEN flow. You must specify the required intention in the [ISOGEN Config] section.

`-include_pst`

Used together with the `-stub` option. It includes power states and power state table in the generated stub file.

`-exclude_pst`

Does not generate power states and power state table in the output file.

`-disjoin_connection`

Enables `disjoin_connection` constraint setting. Tool does not merge [Connection] text format based on the SCOPE attribute.

`-disable_sort_supply_function`

Enables `disable_sort_supply_function` constraint setting. Tool follows the name order in the specified supply functions in [Supply].

`-put_begin_path`

Enables `put_begin_path` constraint setting. Tool generates “puts” commands for [UPF Patch].

`-prefix <prefix>`

Adds `<prefix>_` for every LRM UPF command.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

```
ugo_generate_upf -upf system.upf -version 3.0 -apr system_refine.upf
```

Key Points to Note

- The `-upf` option generates the RTL UPF.
- The `-apr` option generates the APR UPF part. This UPF is the additional UPF for the RTL UPF.
- The UPFA tool separates the power intent into the following two parts:
 - Power intent for major RTL
 - Power intent for APR connection
- In the front-end verification, the UPFA tool separates the physical supply net connection from the RTL UPF. You can use the `-apr` option to direct such physical information into another APR UPF file.
- The supply net and supply port along with their connection information (for supply set's power supply function) should be generated in the APR UPF file. For example:

```
[Supply]
NAME,power,ground,nwell,pwell
SS1,VDD,VSS,VDDC,VSS
```

From the above example, the UPF generation of supply net VDD and supply set's power function would be the physical connection. If you want to generate them, you can use the `-apr` option to direct them into the APR UPF file.

- By default, the ground, nwell and pwell information is generated in the major RTL UPF, whereas power information is generated in the APR UPF file.
- The `-constraint` option provides another mechanism to generate UPF. For more information, see [Appendix A: Tcl commands to Specify Detail Categories in Constraint File](#).

ugo_include_vclp_script

Description

Allows you to pass a script to background VCLP.

Syntax

```
ugo_include_vclp_script
[-f <to_vclp_tcl>]
```

Arguments

`-f`

Allows you to specify the script file.

The `ugo_include_vclp_script` Tcl command enables you to pass the vclp script for background VCLP. This script should contain both analyze and elaboration information.

ugo_optimize_power_intent

Description

Optimizes power intent based on the current power intent, and generates a supplemental CSV file.

To use this command, you must import the design and read UPF (optional). In some cases, search path and link library are also required for elaborating the design information for VCLP.

Note:

Before using the `ugo_optimize_power_intent` command, you must generate the design information using the `ugo_include_vclp_script` command.

Syntax

```
ugo_optimize_power_intent
  [-isolation_strategy -csv <output_file_name>]
  [-isogen [-upf <upf_file_name>] [-iso_config <iso_config_file>]]
  [-dump_cross_domain <iso_info_file>]]
  [-dump_iso_domain <iso_info_file>]
  [-v <upf version>]
  [-map_iso]
  [-allow_unconnected]
  [-src_sink_iso]
  [-iso_place <source|sink>]
  [-hetero_iso]
  [-strict_lrm]]
  [-spagen
  -partition <partition list>
  [-upf <upf file name>]
  [-ref_spa_file <file name>]
  [-force_spa_file <file name>]
  [-rewrite_spa]]
  [-pstgen
  -upf <upf file name>]
  [-hier_iso_gen
  -hier_iso_config <hier_iso_config JSON file>
  [-upf <upf file name>]
  [-iso_place <source|sink>]
  [-hetero_iso]]
  [-out_dir <output directory>]
  [-wrapper_upf <wrapper upf file name>]
```

Arguments

`-isolation_strategy -csv <output file name>`

Optimizes isolation strategies/intent and writes to `<output file name>` with CSV format. This option checks for any missing/uncovered isolation strategy and generates a supplemental CSV file to complete the power intent based on the current power specification.

`-isogen`

Enables the ISOGEN flow. You cannot specify `-spagen` or `-pstgen` together.

`-upf`

Specifies the UPF file input to the `-isogen/-spagen/-pstgen` flow. This option is mandatory for the `-pstgen` flow

`-iso_config`

Allows you to specify the isolation configuration file in JSON format.

`-dump_cross_domain`

Generates all cross-domain settings required for the UPF. The output is reported to the specified file.

`-dump_iso_domain`

Generates all isolation domain settings required for the UPF. The output is reported to the specified file.

`-v <2.0 | 3.0>`

Specifies the version of the output UPF.

`-map_iso`

Enables the generation of the `map_isolation` commands.

`-allow_unconnected`

Includes Logic Sink Unconnected elements for `iso_gen`.

`-src_sink_iso`

Generates the source/sink type isolation command.

`-iso_place <source|sink>`

Specify source or sink. In ISOGEN flow, this option allows you to set up the preference of generating isolation at the source domain side or sink domain side.

-hetero_iso

Generates isolation at the sink domain side.

-strict_lrm

Generates the UPF strictly following LRM.

-spagen

Enables SPAGEN flow. You cannot specify *-isogen* and *-pstgen* together. The output file name is *<partition>_spa.upf* for sub-scope partition and *<top scope>_spa.upf* for top-most scope.

-partition

Specifies the block list for generating SPA strategy. You must specify this option for the SPAGEN flow. You can specify "." for the top-most scope.

-ref_spa_file

Specifies the input file to compare with the final result SPA. The input file format should be as follows:

```
<signal name> <supply set>
<signal name> <supply set>
...
```

-force_spa_file <file name>

```
<signal name> <supply set>
<signal name> <supply set>
...
```

-rewrite_spa

Specifies the SPAGEN flow to rewrite only the boundary SPA value.

-pstgen

Enables the PSTGEN flow. You cannot specify both *-isogen* and *-spagen* together.

-hier_iso_gen

Enables hierarchical ISOGEN flow. You cannot specify *-isogen*, *-spagen*, or *-pstgen* together.

-hier_iso_config

Allows you to specify the isolation configuration file in JSON format.

-wrapper_upf

Generates a wrapper UPF which includes generated UPFs.

`-out_dir`

Specifies the output directory for the isogen/spagen generated UPF files.

`-wrapper_upf`

Generates a wrapper UPF which includes generated UPFs.

Value Returned

Returns 1 if there is no error. Otherwise, return 0.

Example

Example 1

```
ugo_read_power_intent -csv system.csv
ugo_check_power_intent
ugo_optimize_power_intent -isolation_strategy -csv system.iso.csv
// ... modified the proper value for system.iso.csv
ugo_read_power_intent system.iso.csv
ugo_check_power_intent
```

Example 2

```
ugo_include_vclp_script -f to_vclp.tcl
// the Tcl script should contain analyze, elaborate, read_upf
ugo_optimize_power_intent -isogen -iso_config iso_gen.config
```

Example 3

```
ugo_include_vclp_script -f to_vclp.tcl
// the Tcl script should contain analyze, elaborate, read_upf
ugo_optimize_power_intent -spagen -partition "par1 par2"
ugo_optimize_power_intent -spagen -partition "."
```

For more information on ISOGEN, see [Appendix B: ISOGEN Native Support in UPFA](#).

4

UPFA Power Intent

The following sections describe the power intent of the Verdi UPFA tool:

- [CSV Format](#)
- [CSV Syntax](#)
- [User-Defined Comment in the Header of Table](#)
- [User-Defined Variable](#)
- [User-Defined Script](#)
- [Power Domain](#)
- [Supply Port Intent](#)
- [Power Domain Intent](#)
- [Supply](#)
- [Switchable Supply](#)
- [Power Switch Model](#)
- [Power Switch Strategy](#)
- [Logic Network](#)
- [Power Spec](#)
- [Power Spec Intent](#)
- [State Transition](#)
- [Isolation](#)
- [Level Shifter](#)
- [Retention](#)
- [Repeater](#)
- [Load UPF](#)

- [Connection](#)
- [Port Map](#)
- [Attribute](#)
- [Design Hierarchy](#)
- [Set Equivalent](#)
- [ISOGEN Config](#)
- [Scope Condition](#)
- [Special Syntax](#)
- [Partition UPF Integration](#)

CSV Format

The following sections describe the CSV format of the Verdi UPFA tool:

- [Keywords](#)
- [Tcl Variable](#)
- [Comment](#)
- [Boolean Expression](#)

Keywords

All special keywords are wrapped with #. For example, #INTERNAL#.

Tcl Variable

All Tcl variables are wrapped with "\${var_name}". For example, "\${pd_name}".

Comment

The comment should begin with // in a separate line.

Boolean Expression

Boolean expression can be specified in specific cells. The following Boolean operators are supported:

Operator	Description
!	Logical negation

~	Bitwise negation
<	Less than
<=	Less than or equal
>	Greater than
>=	Greater than or equal
==	Equal
!=	Not equal
&	Bitwise conjunction
^	Bitwise exclusive disjunction
	Bitwise disjunction
&&	Logical conjunction

Fast Signal-Based Expression Statement

Following is the fast signal-based expression statement:

```
<#AND# | #OR# | #LAND# | #LOR# | #XOR#>{<word>[RANGE]*}
```

This statement results in a parenthesis expression that contains the expansion according to the keyword.

- #AND#

The expression is expanded by '&'.

- #OR#

The expression is expanded by '| '.

- #LAND#

The expression is expanded by '&&'.

- #LOR#

The expression is expanded by '| | '.

- #XOR#

The expression is expanded by '^'.

- `<word>`

Specifies the prefix of each expansion. Verdi issues an error message if `word` contains any Boolean operator.

- `[RANGE] = [start:end:step=1]`

The `start` option specifies the starting value for an expansion.

If only `start` is specified, `RANGE` is treated as a bit signal and is not expanded.

The `end` option specifies the end value for an expansion. Verdi issues an error message if `end` is smaller or equal than `start`.

The `step` option specifies the step for an increment. Verdi issues an error message if `step` is smaller or equal than 0.

Example 4

```
"ena && #AND#{sig[0][-2:5:2][0]}" results in  
"ena && (sig[0][-2][0] & sig[0][0][0] & sig[0][2][0] &  
sig[0][4][0])"
```

CSV Syntax

You can use the following format to describe the CSV syntax:

```
[Section Name]  
MUST_HAVE[, OPTIONAL, <user-defined field>*]
```

Where,

- `Section Name` is the section header.
 - It tells the tool how to handle the table between any two section headers.
 - If there are multiple tables in one section, use an empty line to separate it.
- `MUST_HAVE` is must have field for the table.
- An error is issued if any cell under the must have field is empty.
- `OPTIONAL` is the optional field for the table.
 - The optional field is wrapped by `[ ]`
 - Cells under optional field can be empty
- `<user-defined field>` allows you to define the field.
- `<user-defined field>*` means the field can be defined multiple times.

- **Single Value** means the field supports only single value. If you specify it multiple times for the same object, the value is updated and only keeps the last value. Verdi issues error message if you specify multiple values in the field.
- **List Value** means the field supports multiple values. If you specify it multiple times for the same object, the list of values is appended to the field.

Note:

- If you specify multiple objects in the same cell, you must specify " " in the csv format.
- If you specify multiple tables for the same category, you must specify at least one \n character between each table.

User-Defined Comment in the Header of Table

You can add user-defined header for comment purpose only. Comment should be specified within parenthesis.

For example:

```
[Variable]  
NAME,VALUE, (MY_COMMENT)
```

Example: test, test_value, (this is comment)

Note:

- Multiple comments are supported in header
- Comments are supported in all sections
- If comma exists in the comment, it should be specified within quotations
- Comma do not reflect in the output to generated UPF

User-Defined UPF Comment in the Header of Table

You can use the new optional and single-value header `UPF_COMMENT` for adding comments. The value can contain commas, newlines, and semicolons. Commas are treated as normal characters, whereas newlines and semicolons are used to split comments into multiple lines. The comments are generated before the command you have specified. The following sessions support this feature:

```
[Power Domain], [Supply], [Switchable Supply], [Power Spec], [Power Spec  
Intent],  
[State Transition], [Isolation], [Retention], [Level Shifter],  
[Repeater], [Variable],
```

```
[Script], [Logic Network], [Connection], [Load UPF], [Supply Port  
Intent],  
[Power Domain Intent], [Power Switch Model], [Power Switch Strategy],  
[Attribute],  
[Apply Power Model], [Set Equivalent], [Global PST Template]
```

Example

```
[Apply Power Model]  
MODEL_NAME, UPF_COMMENT  
modell, "This is the comment for applying modell.;Hello World!"  
  
# This is the comment for applying modell.  
# Hello World!  
apply_power_model modell  
  
[Power Spec]  
#SS#,STATE,power,UPF_COMMENT  
SS1,ON,1.08,"SPEC1"  
SS1,PON,0.8,"SPEC2"  
SS1,OFF,0,"SPEC3"  
  
# SPEC1  
# SPEC2  
# SPEC3  
add_power_state -supply SS1 \  
-state { ON -supply_expr { power == {FULL_ON 1.08} } } \  
-state { PON -supply_expr { power == {FULL_ON 0.8} } } \  
-state { OFF -supply_expr { power == {FULL_ON 0} } }
```

User-Defined Variable

This section describes the syntax of the user-defined variable.

Syntax:

```
[Variable]  
NAME, PATTERN, OBJECT_TYPE, [,SCOPE, METHOD, TRANSITIVE, OTHER,  
DIRECTION, TRAVERSE_MACROS, CHECK_LIST]  
[Variable]  
NAME, VALUE [,SUBSTITUTE]
```

Description:

- **NAME**
 - Specifies the variable name for keeping the search result.
 - The variable name can be referenced by `${<var_name>}` in another table.
 - If the value contains any escaped identifier, it should be enclosed with `{ }`. For example, `{!@PO}iop`

- **PATTERN**

Specifies the string used for searching.

- **METHOD (optional)**

Specifies the method used for string searching. The value can be one of the following:

- **#GLOB#**: Uses Tcl glob expression
- **#REGEX#**: Uses regular expression
- **#EXACT#**: Disallows wildcard expansion

The default value is **#GLOB#**.

- **OBJECT_TYPE**

Limits the result objects. The value can be one of the following:

- **#MODEL#**: Returns the instances of the models whose name matches the pattern.
- **#INST#**: Returns the instances whose name matches the pattern.
- **#PORT#**: Returns the ports whose name matches the pattern.
- **#NET#**: Returns the nets whose name matches the pattern.
- **#PROC#**: Returns the process whose name matches the pattern.

The default value is **#INST#**.

- **TRANSITIVE (optional)**

- **#YES#**: Performs search in entire sub-tree
- **#NO#**: Default is sub-scopes only

- **OTHER (optional)**

Performs search with other constrains. The value can be one of the following:

- `#CASE_INSENSITIVE#`: Performs case-insensitive search. Default is case-sensitive search.
- `#TRANSITIVE#`: Performs search in entire sub-tree. Default is sub-scopes.
- `#LEAF#`: Returns leaf objects only.
- `#NON_LEAF#`: Returns non-leaf objects only.
- `DIRECTION` (Optional)
 - `#IN#`
 - `#OUT#`
 - `#INOUT#`
- `TRAVERSE_MACROS` (optional)

Generate `-traverse_macros` for the `find_objects` command. It could be:

 - `#YES#`
 - `#NO#`: Default
- `CHECK_LIST` (optional)

Generate UPF context to check whether the `find_objects` result is empty. It could be:

 - `#YES#`
 - `#NO#`: Default
- `VALUE`

Specifies literal string or Tcl expression. String is treated as the `LIST` value if there is `,"` as the separator.
- `SUBSTITUTE`

Specifies if expand Tcl variable is in `VALUE`. It can be:

 - `#YES#` -> Uses `" "` to quote the value
 - `#NO#` -> Uses `{ }` to quote the value

The default value is `#YES#`.
- `NAME`

Allows you to identify if different attributes are specified for the same object.

Example

NAME	VALUE
m_inst	m_cpu1
	m_cpu2
	m_cpu3
UPF_DIR	::env(UPF_DIR)
m_ports	original_value
m_ports	new_value

NAME	PATTERN	OBJECT_TYPE
m_bit_insts	bit_inst*	#INST#

Generated UPF:

```
set m_inst "m_cpu1 m_cpu2 m_cpu3"
set UPF_DIR "::env(UPF_DIR)"
set m_ports "new_value"
set m_bit_insts [find_objects . -pattern bit_inst* -object_type inst]
```

User-Defined Script

This section describes the syntax of the user-defined script.

Syntax:

```
[Script]
TYPE, PATHS
```

Description:

- **TYPE**

Specifies the type for script sourced. The value can be one of the following:

- **#BEGIN#**: Pre-script(s) which sources the PATHS (files) at the beginning of the generated UPF.
- **#END#**: Post-script(s) which sources the PATHS (files) at the end of the generated UPF.

The default value is #BEGIN#.

- PATHS

Limits the result objects. The value can be one of the following:

- Specifies the file list of scripts.
- Considers the order of values in the same TYPE.

Example

TYPE	PATHS
#BEGIN#	/remote/test/script1
#BEGIN#	/remote/CPU/script2
#END#	/remote/SYSTEM/test/script3

Generated UPF

```
set m_inst "m_cpu1 m_cpu2 m_cpu3"
## begin of UPF
source /remote/test/script1
source /remote/CPU/script2
...
## end of UPF
source /remote/SYSTEM/test/script3
```

Power Domain

This section describes the syntax of Power Domain.

Syntax:

```
[Power Domain]
NAME, ELEMENTS, [EX_ELEMENTS, AVAILABLE_SUPPLIES, UPDATE <supply
handle>*]
```

Description:

- NAME

Specifies the name of the power domain.

- ELEMENTS

Specifies the applied elements of the power domain. You can specify the following keywords in this column:

- `#DEFAULT#`: Specifies top power domain. An error is issued if no or more than one of these keywords is specified.
- `${<variable name>}`: See [User-Defined Variable](#) section.
- Specifying "." in the `ELEMENTS` column is the same as specifying `#DEFAULT#`.

In addition to keywords, you can also specify list of instances.

Values are appended to power domain object if the object is previously defined.

- `EX_ELEMENTS` (optional)

Specifies a list of instances or ports to which the strategy does not apply.

- `AVAILABLE_SUPPLIES` (optional)

Specifies the supply sets that are available for use by the implementation tool to insert power cell for the power domain. You can specify the following keyword in this column:

- `#EMPTY#`: Forces the tool to generate the `-available_supplies` option and value `{ }`.

Values are appended to the power domain object if the object is previously defined.

- `UPDATE`

Generates `-update` for the `create_power_domain` UPF command. It could be:

- `#YES#`
- `#NO#`

Default is `#NO#`

It is an error if `UPDATE=#YES#` but power domain object does not exist.

- `<supply handle>*`

Specifies the supply handle. The name and value of supply handle and value cannot be a Tcl variable.

If the name of `supply handle` exists in previously defined power domain object, then the value of supply handle is updated.

- `NAME` is the key to identify if you are specifying different attributes for the same object.

Example: Variable

NAME	PATTERN	OBJECT_TYPE
cpu_list	CPU	#MODEL#

Example: Power Domain

NAME	ELEMENTS	AVAILABLE_SUPPLIES	primary	supply_2
PD_TOP	#DEFAULT#	SS1,SS2	SS1	SS2
PD_RX	U1,U2,U3	SS1,SS1_SW	SS1_SW	
PD_CPU	\${cpu_list}	SS1	SS1	

Generated UPF

```
set cpu_list [find_object . -pattern CPU -object_type model]
create_power_domain PD_TOP -elements { . } -available_supplies { SS1
  SS2 } -supply { primary SS1 } -supply { supply_2 SS2 }
create_power_domain PD_RX -elements { U1 U2 U3 } -available_supplies
  { SS1 SS1_SW } -supply { primary SS1_SW }
create_power_domain PD_CPU -elements ${cpu_list} -available_supplies
  { SS1 } -supply { primary SS1 }
```

Supply Port Intent

This section describes the syntax of Supply Port Intent

Syntax:

```
[Supply Port Intent]
NAME, [DIRECTION, NET]
```

Description:

- NAME

Specifies the name of the supply port.

- DIRECTION

Creates related supply port and then connects to supply net. It could be:

- #IN#
- #OUT#
- #INOUT#

The default value is #IN#.

- NET

Specifies the supply net to connect. If the data is empty, VCS finds the same `NAME` of supply net and performs the connection. It should be a warning if the supply net is not found.

Example: Supply Port Intent

NAME	DIRECTION	NET
VDD	#IN#	
VSS	#IN#	
VDDX	#IN#	VDD

Generated UPF

```
create_supply_port VDD -direction in
connect_supply_net VDD -ports VDD
create_supply_port VSS -direction in
connect_supply_net VSS -ports VSS
create_supply_port VDDX -direction in
connect_supply_net VDD -ports VDDX
```

Power Domain Intent

This section describes the syntax of Power Domain Intent.

Syntax:

```
[Power Domain Intent]
NAME, ELEMENTS[, POWER, GROUND, RESOLVE, STATENUM, VOLTAGE,
  LOGIC_CONDITION, NWELL, PWELL]
```

Description:

- NAME

Specifies the name of the power domain.

If not specified, Verdi creates supply net instead of power domain.

- ELEMENTS

Specifies the applied elements of the power domain:

- #DEFAULT#: Specifies instances
- #EMPTY#: Creates supply set with NAME and supply net instead of power domain.

Specifying " . " in the ELEMENTS column is the same as specifying #DEFAULT#.

- POWER

The supply net that would be created for the primary power supply of the domain.

- GROUND

The supply net that would be created for the primary ground supply of the domain.

- RESOLVE

Used for tool to create APR refinement. It could be:

- #UNRESOLVED#
- #ONE_HOT#
- #PARALLEL#
- #PARALLEL_ON_HOT#

The default value is #UNRESOLVED#

- STATENUM

Specifies the number of power state for the supply net.

Reports an error if mismatch occurs with data input.

- VOLTAGE

Digital value that specifies the domain's state.

The value cannot be a Tcl variable.

- LOGIC_CONDITION

Logic expression specified for the supply set state.

Specifying Tcl variable is allowed. However, tool generates expanded value in output UPF.

- `NWELL`

The supply net that would be created for the supply set function.

- `PWELL`

The supply net that would be created for the supply set function.

Note:

- The `NAME` is the id key to identify the domain.
- If the `NAME` value is empty, previous line value would be referred.
- UPFA creates supply set for the primary of power domain, and specifies `POWER`, `GROUND`, `NWELL`, and `PWELL` to the supply functions. The name of supply set would be following format:

`SS_<PD NAME>`

If `NAME` is `#EMPTY#`, the format would be `SS_<IDX>`.

- UPFA creates the supply set' state and power domain's state by referring the `VOLTAGE` value with the following format:

`SS_<PD NAME>_ON_<VOLTAGE>`: For supply set, and if `VOLTAGE` is not OFF.

`SS_<PD NAME>_OFF`: For supply set, and if `VOLTAGE` is OFF.

`SS_<IDX>_ON_<VOLTAGE>`: For supply set, empty power domain, if `VOLTAGE` is not OFF

`SS_<IDX>_OFF`: For supply set, empty power domain, if `VOLTAGE` is OFF.

`<PD NAME>_ON_<VOLTAGE>`: For power domain, and if `VOLTAGE` is not OFF.

`<PD NAME>_OFF`: For power domain, and if `VOLTAGE` is OFF

- The `-simstate` option is added with:

`NORMAL` if `VOLTAGE` is not OFF.

`CORRUPT` if `VOLTAGE` is OFF.

- The ground is assumed to be 0.0v.
- Following is the limitation to specify the STATENUM,VOLTAGE,LOGIC_CONDITION:

It is NOT allowed to redefine the
STATENUM,VOLTAGE,LOGIC_CONDITION for the same POWER.

Example 1: Power Domain Intent

NAME	Elements	Power	Ground	Resolve	State num	Voltage	LOGIC_CONDITION
PD_TOP	#DEFAULT#	VDD	VSS		3	1	PSW[0] & PSW[50]
						0.6	PSW[1] & PSW[2]
						OFF	!(PSW[0] & PSW[50])
PD_CPU	m_cpu	VDD_SW	VSS		2	1	PSW[1] & PSW[49]
						OFF	!(PSW[1] & PSW[49])
#EMPTY#		VDDC	VSS	#PARALLEL#	1	1	
PD_SS	#EMPTY#	VDDC	VSS				

Generated UPF

```
## Supply Net
create_supply_net VDD
create_supply_net VSS
create_supply_net VDDC -resolve parallel
create_supply_net VDD_SW

## Supply Set
create_supply_set SS_PD_TOP
create_supply_set SS_PD_TOP -function { power VDD } -function { ground
VSS } -update
create_supply_set SS_PD_CPU
create_supply_set SS_PD_CPU -function { power VDD_SW } -function { ground
VSS } -update
create_supply_set SS_1
create_supply_set SS_1 -function { power VDDC } -function { ground VSS }
-update
create_supply_set SS_PD_SS
create_supply_set SS_PD_SS -function { power VDDC } -function { ground
VSS } -update

## Power Domain
```

```
create_power_domain PD_TOP -elements { . } -supply { primary SS_PD_TOP }
create_power_domain PD_CPU -elements { m_cpu } -supply { primary
  SS_PD_CPU }

## Power Spec
add_power_state SS_PD_TOP \
  -state { SS_PD_TOP_ON_1_0 -supply_expr { power == { FULL_ON 1.0 }
    && ground == { FULL_ON 0 } } -logic_expr { PSW[0] & PSW[50] } -simstate
  NORMAL } \
  -state { SS_PD_TOP_ON_0_6 -supply_expr { power == { FULL_ON 0.6 }
    && ground == { FULL_ON 0 } } -logic_expr { PSW[1] & PSW[2] } -simstate
  NORMAL } \
  -state { SS_PD_TOP_OFF -supply_expr { power == { OFF } &&
    ground == { FULL_ON 0 } } -logic_expr { !(PSW[0] & PSW[50]) } -simstate
  CORRUPT }

add_power_state SS_PD_CPU \
  -state { SS_PD_CPU_ON_1_0 -supply_expr { power == { FULL_ON 1.0 }
    && ground == { FULL_ON 0 } } -logic_expr { PSW[1] & PSW[49] } -simstate
  NORMAL } \
  -state { SS_PD_CPU_OFF -supply_expr { power == { OFF } && ground
    == { FULL_ON 0 } } -logic_expr { !(PSW[1] & PSW[49]) } -simstate
  CORRUPT}

add_power_state SS_1 \
  -state { SS_1_ON_1_0 -supply_expr { power == { FULL_ON 1.0 } &&
    ground == { FULL_ON 0 } } -simstate NORMAL }
```

Example 2

NAME	VALUE	SUBSTITUTE
CON_1	XXX/YYY/PSW1	#YES#
	& XXX/YYY/PSW2	
	& XXX/YYY/PSW3	
CON_2	a[0]	#NO#
	& a[1]	
	& a[2]	

Power Domain Intent

NAME	Elements	Power	Ground	Resolve	State num	Voltage	LOGIC_CONDITION
PD_TOP	#DEFAULT#	VDD	VSS		3	1	#{CON_1}

						0.6	\${CON_2}
						OFF	!((\${CON_1}) (\${CON_2}))

Generated UPF

```
# [Variable]
set CON_1 "XXX/YYY/PSW1 & XXX/YYY/PSW2 & XXX/YYY/PSW3"
set CON_2 { a[0] & a[1] & a[2] }

# [Supply Set]
create_supply_set SS_PD_TOP

# [Power Domain]
create_power_domain PD_TOP -elements { . } -supply { primary SS_PD_TOP }

# [Supply Set]
create_supply_net VSS
create_supply_net VDD -resolve parallel
create_supply_set SS_PD_TOP -function { ground VSS } -function { power
  VDD } -update

# [Power Spec]
add_power_state \
  -state { SS_PD_TOP_OFF -supply_expr { power == OFF && ground
    == {FULL_ON 0} } -logic_expr { !((XXX/YYY/PSW1 & XXX/YYY/PSW2 &
    XXX/YYY/PSW3) | (a[0] & a[1] & a[2])) } -simstate CORRUPT } \
  -state { SS_PD_TOP_ON_0_6 -supply_expr { power == {FULL_ON 0.6} &&
    ground == {FULL_ON 0} } -logic_expr { a[0] & a[1] & a[2] } -simstate
    NORMAL } \
  -state { SS_PD_TOP_ON_1 -supply_expr { power == {FULL_ON 1} &&
    ground == {FULL_ON 0} } -logic_expr { XXX/YYY/PSW1 & XXX/YYY/PSW2 &
    XXX/YYY/PSW3 } -simstate NORMAL } \
  -supply SS_PD_TOP
```

Supply

This section describes the syntax of a supply.

Syntax:

```
[Supply]
NAME, [DIRECTION, RESOLVE, SS_ONLY, <supply function>*]
```


Description:

- NAME

Specifies the name of the supply set. If you specify #EMPTY#, supply set is not created, only supply net for supply function is created.

- DIRECTION (optional)

Allows Verdi to create APR refinement. You can use the following keywords:

- #IN#
- #OUT#
- #INTERNAL#

The default value is #INTERNAL#.

- RESOLVE (optional)

Allows Verdi to create APR refinement. You can use the following keywords:

- #UNRESOLVED#
- #ONE_HOT#
- #PARALLEL#
- #PARALLEL_ON_HOT#
- <resolution _function_name>

The default value is #UNRESOLVED#.

- <supply function>* (optional)

Allows Verdi to create APR refinement.

If the value is non-empty, Verdi uses this value to name the supply net.

If the value is empty, Implicit Naming Rule is considered to name the supply net.

The name of supply function and the value of supply function should be a single value. If the name of supply function exists in the previously defined supply, the value of supply function is updated. **The name and value of supply function cannot be a Tcl variable.**

- SS_ONLY

It can be:

- #YES#
- #NO#

If value = #YES#, only the supply set object and supply set function are created.
Supply net and supply port are not created.

You can use this option while creating a supply set and to refer existing supply net as the supply set function.

Example: Supply

NAME	DIRECTION	RESOLVE	power	ground	nwell	pwell
SS1	#IN#					
SS1_SW	#INTERNAL#	#PARALLEL#	VDD	VSS	VDDC	VSS
SS2	#OUT#	#PARALLEL#	VDD	VSS	VDDC	

Generated UPF

```
## default, ground, nwell, pwell will be in RTL, power will be in APR
## RTL UPF
create_supply_net VSS
create_supply_port VSS
connect_supply_net VSS -ports { VSS }
create_supply_net VDD_nwell_of_SS1
create_supply_port VDD_nwell_of_SS1
connect_supply_net VDD_nwell_of_SS1 -ports { VDD_nwell_of_SS1 }
create_supply_net VDD_pwell_of_SS1
create_supply_port VDD_pwell_of_SS1
connect_supply_net VDD_pwell_of_SS1 -ports { VDD_pwell_of_SS1 }
create_supply_set SS1 -function { ground VSS } -function { nwell
  VDD_nwell_of_SS1 } -function { pwell VDD_pwell_of_SS1 } -update
create_supply_net VDDC -resolve parallel
create_supply_set SS1_SW -function { ground VSS } -function { nwell
  VDDC } -function { pwell VSS } -update
create_supply_port VDDC -direction out
connect_supply_net VDDC -ports { VDDC }
create_supply_net VDD_pwell_of_SS2 -resolve parallel
create_supply_port VDD_pwell_of_SS2 -direction out
connect_supply_net VDD_pwell_of_SS2 -ports { VDD_pwell_of_SS2 }
create_supply_set SS2 -function { ground VSS } -function { nwell VDDC }
  -function { pwell VDD_pwell_of_SS2 } -update

## APR UPF
create_supply_net VDD_power_of_SS1
```

```
create_supply_port VDD_power_of_SS1
connect_supply_net VDD_power_of_SS1 -ports { VDD_power_of_SS1 }
create_supply_set SS1 -function { power VDD_power_of_SS1 } -update
create_supply_net VDD -resolve parallel
create_supply_set SS1_SW -function { power VDD } -update
create_supply_port VDD -direction out
connect_supply_net VDD -ports { VDD }
create_supply_set SS2 -function { power VDD } -update
```

Supply Map

This section describes the syntax of a supply map.

Syntax:

```
[Supply Map]
NAME, MAPPING
```

Description:

- NAME

Specifies the name of the supply map that is used for [Apply Power Model]:SUPPLY_MAP. If the value is empty, the value in the previous line is referred to.

- MAPPING

Specifies a list value. Following is the syntax:

- {<instance scope supply set ref> <current scope supply set ref>}
- "{<instance supply set1 ref> <current supply set1 ref>}, {<instance supply set2 ref> <current supply set2 ref>},..."
- Specify a space between <instance supply set ref> with <current supply set ref>
- You must surround {} with <instance supply set ref> and <current supply set ref>.

Switchable Supply

This section describes the syntax of the switchable supply.

Syntax:

```
[Switchable Supply]
OUT, STATE [, SW_NAME, PD, IN, ACK, ACK_DELAY, ACK_CONTROL, SUPPLY, CELL]
```

Description:

- `OUT`

Specifies the supply set name or supply net which can be switched. The supply set/supply net should be defined in [Supply] or [Power Domain Intent]

The value cannot be a Tcl variable

- `STATE`

Specifies the switch state for *OUT*. You can use the following keywords:

- `#OFF#{<condition>}`
- `#ON#{<condition>}`
- `#PARTIAL_ON#{<condition>}`
- Verdi issues an error message if `<condition>` is not specified
- If `STATE` is `#ON#` or `#PARTIAL_ON#`, `IN` must be specified and its cell should not be empty
- `<condition>` should be a Boolean expression and cannot be a Tcl variable

- `SW_NAME` (optional)

Specifies the name of generated power switch. If this option is not specified, the default value is *PSW_<OUT>*

- `PD` (optional)

Specifies the domain of power switch. You can use the following keyword:

- `#EMPTY#`: If specified, `-domain` is not specified. If this keyword is not specified, the default is top of power domain.

- `ACK` (optional)

Specifies the ack port for `STATE`. The value cannot be a Tcl variable.

- `ACK_CONTROL` (optional)

Specifies the ack control. Verdi issues an error message when `ACK` is not specified or its cell is empty. The value should be a simple Boolean expression and cannot be a Tcl variable.

- `SUPPLY` (optional)

Specifies the supply of the power switch.

- `CELL` (optional)

Specifies the power switch model to be used for implementation tool.

- `SW_NAME`

Allows you to identify if different attributes are specified for the same object.

Example 1: Switchable Supply

OUT	STATE	SW_NAME	PD	IN	CELL
VDD_SW	#ON#{VA1_PSW[0] &&VA1_PSW[50]}	PSW_VA1_G0	PD_VA1	VDD	PSW1
	#OFF#{...}	PSW_VA1_G0			

Generated UPF

```
create_power_switch PSW_VA1_G0 \
    -domain PD_VA1
    -output_supply_port { out VDD_SW } \
    -input_supply_port { in_1 VDD } \
    -control_port { ctrl_0 .../VA1_PSW[0]} \
    -control_port { ctrl_1 .../VA1_PSW[1]} \
    ...
    -control_port { ctrl_50 .../VA1_PSW[50]} \
    -on_state { on_1_in_1 in_1 ctrl_0 & ctrl_1 & ... & ctrl_50} \
    -off_state { off ... }
map_power_switch PSW_VA1_G0 -domain PD_VA1 -lib_cells PSW1
```

Power Switch Model

This section describes the syntax of Power Switch Model.

Syntax:

```
[Power Switch Model]
[MODEL_NAME, STATE, CONDITION[, IN, OUT, ACK, ACK_DELAY, ACK_CONTROL]
```

Description:

Defines a switch model. You should use Power Switch Model together with [Power Switch Strategy](#).

- `MODEL_NAME`

Specifies the name of power switch model.

It is error if this is an unresolved Tcl

- **STATE**

Specifies the switch state for *OUT*. You can use the following keywords:

- #OFF#
- #ON#
- #PARTIAL_ON#

- **CONDITION**

Specifies the condition for the STATE. It should be a Boolean expression and cannot be a Tcl variable.

- **IN**

Specifies the port name for input supply port name. The value cannot be a Tcl variable.

- **OUT**

Specifies the port name for output supply port name.

- **ACK**

Specifies the port name for ack port name. The value cannot be a Tcl variable.

- **ACK_DELAY**

Specifies the ack delay for ack port. The value cannot be a Tcl variable.

- **ACK_CONTROL**

Specifies the control for the ack port. It should be a Boolean expression and cannot be a Tcl variable.

Example: Power Switch Model

MODEL_NAME	STATE	CONDITION	IN	OUT	ACK	ACK_DELAY	ACK_CONTROL
SW_M1	#ON#	!EN1 &!EN2	VDD	VDDG	ACK1	10	EN1
					ACK2		EN2

Power Switch Strategy

This section describes the syntax of Power Switch Strategy.

Syntax:

```
[Power Switch Strategy]]  
[SW_NAME,MODEL_NAME[,PORT,NET,PD,PINNUM,SUPPLY,CELL]
```

Description:

Defines switch strategy.

- SW_NAME
Specifies the name of power switch.
- MODEL_NAME
Specifies the name of power switch model. It should be defined with [Power Switch Model](#). It is error if MODEL_NAME is unresolved Tcl.
- PORT
Specifies the port name specified in power switch model. The value cannot be a Tcl variable.
- NET
Specifies the connected net name for the PORT. The value cannot be a Tcl variable. If no value is specified, tool will not generate the ack port and related ack delay UPF code.
- PD
It could be #EMPTY#.
If #EMPTY# is specified, -domain will not be specified.
If PD is not specified, the default will be top of power domain.
- PINNUM
Specifies the total number of ports connected to the specified net.
- SUPPLY
Specifies the supply of the power switch.
- CELL
Specifies the power switch cells to be used for implementation tool.

Example 1: Power Switch Model

MODEL_NAME	STATE	CONDITION	IN	OUT	ACK	ACK_DELAY	ACK_CONTROL
------------	-------	-----------	----	-----	-----	-----------	-------------

SW_M1	#ON#	!EN1 &!EN2	VDD	VDDG	ACK1	10	EN1
					ACK2		EN2

Power Switch Strategy

SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL
CMD_SW	SW_M1	PD_VA1	6	VDD	VDD	PSW1CELL
				VDDG	VDDG	
				EN1	PSW[0]	
				EN2	PSW[1]	
				ACK1	A	
				ACK2	B	

Generated UPF

```
create_power_switch CMD_SW \
    -domain PD_VA1
    -output_supply_port { VDDG VDDG } \
    -input_supply_port { VDD VDD } \
    -control_port { EN1 PSW[0]} \
    -control_port { EN2 PSW[1]} \
    -on_state { on_1_VDD VDD { !EN1&!EN2} } \
    -ack_port { ACK1 A EN1 } \
    -ack_port { ACK2 B EN2 } \
    -ack_delay { ACK1 10 }
map_power_switch CMD_SW -domain PD_VA1 -lib_cells PSW1CELL
```

Example 2: Power Switch Model

MODEL_NAME	STATE	CONDITION	IN	OUT	ACK	ACK_DELAY	ACK_CONTROL
SW_M1	#ON#	!EN1 &!EN2	VDD	VDDG	ACK1	10	EN1
	#ON#	~EN1 & ~EN2 & EN3	VDD IN2		AK2	10	~EN2

	#OFF#	~((EN1 & EN2 & ~EN3) (EN1 & ~EN2 & EN3))			AK3		EN3
--	-------	--	--	--	-----	--	-----

Power Switch Strategy

SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL
PSW1	SW_M1	PD_SW	9	VDDIN1	VDD1	POWER_SW1
				VDDIN2	VDD2	
				VDDG	VDDOUT	
				EN1	PSW[0]	
				EN2	PSW[1]	
				EN3	PSW[2]	
				AK1	ACK[0]	
				AK2	ACK[1]	
				AK3	ACK[2]	

Generated UPF

```
create_power_switch PSW1 -domain PD_SW -output_supply_port {VDDG VDDOUT}
\
  -input_supply_port {VDDIN1 VDD1} -input_supply_port {VDDIN2 VDD2} \
  -on_state {on_1_VDDIN1 VDDIN1 EN1 & EN2 & ~EN3} \
    -on_state {on_2_VDDIN2 VDDIN2 ~EN1 & ~EN2 & EN3} \
    -off_state {OFF ~((EN1 & EN2 & ~EN3) | (EN1 & ~EN2 & EN3)) }
\
  -control_port {EN1 PSW[0]} -control_port {EN2 PSW[1]} -control_port
{EN3 PSW[2]} \
    -ack_port {AK1 ACK[0] ~EN1} -ack_port {AK2 ACK[1] ~EN2}
-ack_port {AK3 ACK[2] EN3} \
  -ack_delay {AK2 10ns}

map_power_switch PSW1 -domain PD_SW -lib_cells POWER_SW1
```

Logic Network

This section describes the syntax of the logic network.

Syntax:

```
[Logic Network]  
NAME,DIRECTION [, SUPPLY]
```

Description:

- NAME

Specifies the name of logic object

- DIRECTION

Specifies the direction of logic object. It can be:

- #IN#
- #OUT#
- #INTERNAL#

The default value is #INTERNAL#.

- SUPPLY

Specifies the driver supply or receiver supply.

If DIRECTION = #IN#, SUPPLY is applied at driver supply set.

If DIRECTION = #OUT#, SUPPLY is applied at receiver supply set.

If DIRECTION = #INOUT#, SUPPLY is applied at driver supply set and receiver supply set.

- NAME

Allows you to identify if different attributes are specified for the same object.

Example: Logic Network

NAME	DIRECTION	SUPPLY
lp_enable	#OUT#	VDD
lp_cpu_wake	#INTERNAL#	

Generated UPF

```
create_logic_net lp_enable
create_logic_port lp_enable -direction out
connect_logic_net lp_enable -ports lp_enable
set_port_attribute -ports lp_enable -receiver_supply SS_PD_VDD
create_logic_net lp_cpu_wake
```

Power Spec

This section describes the syntax of the power spec.

Syntax:

```
[Power Spec]
#SS#,STATE,<supply function>*,SIMSTATE,LOGIC_CONDITION
#PD#,STATE,<supply set | power domain>*
#PSG#,STATE,<supply set function | supply set | power domain | power
state group| supply port | supply net>*,LEGAL, ADD_ONLY
#PST#,STATE, SUPPLIES, SUPPLY_STATES*
```

Description:

- #SS#
The value should be the name of supply set.
- #PD#
The value should the name of power domain.
- #PSG#
The value should the name of power state group.
- STATE
The value should be the name of power state.
The value cannot be a Tcl variable.
- <supply function>
The value should be:
 - #OFF#
 - #PARTIAL_ON{<voltage>}#
 - <voltage>The value cannot be a Tcl variable.
- <supply set function>

The value should be the supply set's function.

The value cannot be a Tcl variable.

If supply function is specified in #PSG#, UPF2.0 is generated instead of UPF3.0.

- `<supply set>`

The value should be the power state of `<supply set>`.

The value cannot be a Tcl variable.

- `<power domain>`

The value should be the power state of `<power domain>`.

The value cannot be a Tcl variable.

- `SIMSTATE(optional)`

Specifies a simstate for the power states associated with a supply set.

The value should be:

- `NORMAL`
- `CORRUPT`
- `CORRUPT_ON_CHANGE`
- `CORRUPT_ON_CHANGE`
- `CORRUPT_STATE_ON_ACTIVITY`
- `CORRUPT_ON_ACTIVITY`
- `NOT_NORMAL`

The value cannot be a Tcl variable.

- `<power state group>`

The value should be the power state of `<power state group>`.

The value cannot be a Tcl variable.

- `LOGIC_CONDITION (optional)`

Logic expression specified for the supply set state.

The value cannot be a Tcl variable.

- `LEGAL(optional)`

Specify legal or illegal for the transition.

The value can be #YES# or #NO#.

- **ADD_ONLY(optional)**

Generates `add_power_state` and does not create power state group.

The value can be #YES# or #NO#.

- **SUPPLIES**

Specifies supply nets or supply ports.

Specifies the header supplies of the `create_pst` command. The value is #EMPTY# if you want to generate only `add_pst_state` as the PST header can be defined only once. Duplicate definitions generates an error.

If you leave this column empty, the value is inherited from the previous line.

- **SUPPLY_STATES**

The states correspond to the header, #EMPTY#, if you want to generate only the `create_pst` command.

For the `create_pst` command, the column `STATE` of this row is redundant and does not take any effect.

An error message is issued if the state number is different from the header supplies number.

The generation of state values follows your order in the csv file if you specify the #EMPTY# value to the `SUPPLIES` column.

Example: Power Spec

#SS#	STATE	power	ground
SS1	ON	0.78	0
SS1_SW	ON_78	0.78	0
SS1_SW	ON_85	0.85	0
SS1_SW	OFF	#OFF#	0
SS1	ON	0.78	0
SS1	OFF	#OFF#	0
SS1	ON	0.85	0

#PD#	STATE	SS1	SS3
PD_TOP	ON	ON	ON

#PSG#	STATE	PD_TOP	PD_RX
PSG_TOP	ON_HV	ON	ON
PSG_TOP	ON_LV	ON	ON_LV
PSG_TOP	OFF	ON	OFF

#PST#	STATE	SUPPLIES	SUPPLY_STATES
PST_TOP	ON_HV	"VDD,VSS"	"ON,OFF"
PST_TOP	ON_LV		"OFF,ON"
PST_1		"VDD,VSS"	#EMPTY#
PST_2	ON	#EMPTY#	"ON,OFF"

Generated UPF

```

add_power_state SS1 \
  -state {ON -supply_expr {power == {FULL_ON 0.78} && ground == {FULL_ON
0}}}

add_power_state SS1_SW \
  -state {ON_78 -supply_expr {power == {FULL_ON 0.78} && ground ==
{FULL_ON 0}}}
  -state {ON_85 -supply_expr {power == {FULL_ON 0.85} && ground ==
{FULL_ON 0}}}
  -state {OFF -supply_expr {power == {OFF} && ground == {FULL_ON 0}}}

add_power_state SS2 \
  -state {ON -supply_expr {power == {FULL_ON 0.78} && ground == {FULL_ON
0}}} \
  -state {OFF -supply_expr {power == {OFF} && ground == {FULL_ON 0}}} \

add_power_state SS3 \
  -state {ON -supply_expr {power == {FULL_ON 0.85} && ground == {FULL_ON
0}}}

add_power_state PD_TOP \
  -state {ON -logic_expr {SS1 == ON && SS3 == ON}}
```

```
add_power_state PD_RX \
  -state {ON_HV -logic_expr {SS1_SW == ON_85}} \
  -state {ON_LV -logic_expr {SS1_SW == ON_78}} \
  -state {OFF -logic_expr {SS1_SW == OFF}}

create_powr_state PSG_TOP
add_power_state PSG_TOP \
  -state {ON_HV -logic_expr {PD_TOP == ON && PD_RX == ON_HV}} \
  -state {ON_LV -logic_expr {PD_TOP == ON && PD_RX == ON_LV}} \
  -state {OFF -logic_expr {PD_TOP == ON && PD_RX == OFF}}

create_pst PST_TOP -supplies { VSS VDD }
add_pst_state ON_HV -pst PST_TOP -state { ON OFF }
add_pst_state ON_LV -pst PST_TOP -state { OFF ON }
create_pst PST_1 -supplies { VSS VDD }
add_pst_state ON -pst PST_2 -state { ON OFF }
```

Power Spec Intent

This section describes the syntax of Power Spec Intent.

Syntax:

STATE, [**<PD NAME>**, **<NET NAME>** ...]

Description:

- **STATE**

Specifies the name of state for power state table or power state group.

- **PD NAME**

Specifies the power domain name, which should be defined in [Power Domain Intent](#).

The value for **PD NAME** is the **VOLTAGE** that defined in Power Domain Intent's **VOLTAGE** attribute.

- **NET NAME**

Specifies the supply net name, which should be defined in Power Domain Intent's **POWER** attribute.

The value for **NET NAME** is the voltage that defined in Power Domain Intent's **VOLTAGE** attribute.

Example: Power Spec Intent

STATE	VDD	VDD_SW	VDDC
PM_Mode_IDEL_0	1	1	1

PM_Mode_FUSE_0	0.6	1	1
PM_Mode_OFF	OFF	OFF	1

Generated UPF

```
create_power_state_group PSG_TOP
add_power_state -group PSG_TOP \
    -state { PM_Mode_IDLE_0 -logic_expr { SS_PD_TOP == SS_PD_TOP_ON_1_0
    && SS_PD_CPU == SS_PD_CPU_ON_1_0 && SS_1 == SS_1_ON_1_0 } } \
    -state { PM_Mode_FUSE_0 -logic_expr { SS_PD_TOP == SS_PD_TOP_ON_0_6
    && SS_PD_CPU == SS_PD_CPU_ON_1_0 && SS_1 == SS_1_ON_1_0 } } \
    -state { PM_Mode_OFF -logic_expr { SS_PD_TOP == SS_PD_TOP_OFF &&
    SS_PD_CPU == SS_PD_CPU_OFF && SS_1 == SS_1_ON_1_0 } }
```

State Transition

This section describes the syntax of State Transition.

Syntax:

```
[State Transition]
NAME, OBJECT, FROM, TO[, LEGAL
```

Description:

- NAME

Specifies the name of transition.

- OBJECT

Specifies the target object of transition.

The value should be:

- Supply Set
- Power Domain
- Power State Group

- FROM

Specifies the state list of the begin of transition.

The value cannot be a TCL variable.

- TO

Specifies the state list of the end of transition.

The value cannot be a Tcl variable.

- **LEGAL** (optional)

Specifies legal or illegal for the transition.

The value should be one of the following:

- #YES#
- #NO#

The default value is #YES#.

- **OBJECT** is the key to identify if you are specifying different attributes for the same object.

Example: State Transition

NAME	OBJECT	FROM	TO	LEGAL
legal_0	PG1	PD2_ON	PD1_ON,ALL_OFF	#YES#
Illegal_0	PG1	PD1_ON	ALL_OF	#NO#

Generated UPF

```
add_state_transition -group PG1 \
  -transition {legal_0 -from {PD2_ON} -to {PD1_ON ALL_OFF}} \
  -transition {illegal_0 -from {PD1_ON} -to {ALL_OFF} -illegal}
```

Isolation

This section describes the syntax of isolation.

Syntax:

[Isolation]

NAME [, **PD**, **INSERT_SIDE**, **ISO_SIGNAL**, **ELEMENTS**, **EX_ELEMENTS**, **CLAMP**, **ISO_SENSE**, **SRC**, **SINK**, **LOCATION**, **SUPPLY**, **CELL**, **NO_ISOLATION**, **OPTIMIZABLE**, **NAME_PREFIX**, **NAME_SUFFIX**, **X**, **DIFF_SUPPLY_ONLY**, **FORCE**, **APPLIES_TO_BOUNDARY**, **UPDATE**]

Description:

- NAME

Specifies the name of the isolation rule.

- PD

Specifies the applied power domain of the isolation rule.

- INSERT_SIDE

- If `INSERT_SIDE` is present, (1) `SRC` and `SINK` are mandatory, and power domain should be specified (2) It is an error if `PD`, `LOCATION`, or `SUPPLY` is specified.

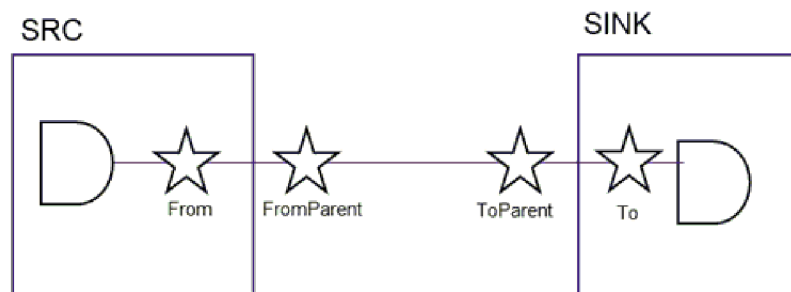
- The following four keyword values can be specified:

#FROM# - PD is SRC location is self, supply is the SINK domain's primary.

#FROMPARENT# - PD is SRC, location is parent, supply is the SINK domain's primary.

#TOPARENT# - PD is SINK, location is parent, supply is the SINK domain's primary.

#TO# - PD is SINK, location is self, supply is the SINK domain's primary.



- ISO_SIGNAL

Specifies the control signal for the isolation cell.

- ELEMENTS (optional)

Specifies the applied elements of the isolation rule. You can specify the following keywords in this column:

- #INPUTS#: The elements are restricted to any port that the direction is input relative to the specified domain.
- #OUTPUTS#: The elements are restricted to any port that the direction is output relative to the specified domain.

- **#BOTH#**: The elements are restricted to any port that the direction is both relative to the specified domain.
- **#ALL#**: Every port on the interface of the domain is applied
- `${<variable name>}`: See [User-Defined Variable](#) section.

Other than keywords, you can also specify list of instances or ports.

The values are appended to isolation object if it is previously defined. However, if multiple **#INPUTS#** and **#OUTPUTS#** are specified, only the last keyword value is applied. If keyword **#ALL#** is specified, all values are ignored except **#INPUTS#** and **#OUTPUTS#**.

- **EX_ELEMENTS** (optional)

Specifies a list of instances or ports to which the strategy does not apply.

- **CLAMP** (optional)

Specifies the value of the inferred isolation cell's output when isolation is enabled. You can specify one of the following:

- 0 (the logic value 0)
- 1 (the logic value 1)
- Z (the logic value Z)
- latch

- **ISO_SENSE** (optional)

Specifies the active level of the specified control signal.

- **SRC** (optional)

When specified, the elements are restricted to any port that is driven by logic powered by a supply set that matches the specified one.

- **SINK** (optional)

When specified, the elements are restricted to any port that is present on a path for which the receiver supply set matches the specified one.

- **LOCATION** (optional)

Specifies the location for the inferred isolation cell to be placed. It can be:

- **#SELF#**
- **#PARENT#**
- **#FANOUT#**

The default value is `#SELF#`.

- `SUPPLY` (optional)

Specifies the power domain, supply set, or supply net that powers the isolation cell.

- `CELL` (optional)

Specifies the isolation model to be used for implementation tool.

- `NO_ISOLATION` (optional)

Specifies no-isolation. It can be:

- `#YES#`: Verdi generates `-no_isolation` for the power intent.
- `#NO#`

The default value is `#NO#`

If value is `#YES#`, (1) PD is mandatory, (2) `ISO_SIGNAL`, `CLAMP`, `ISO_SENSE`, `SUPPLY`, `LPCATION`, `INSERT_SIDE` should not present.

- `OPTIMIZABLE` (optional)

Indicates if UPFA can be used to optimize the isolation rule. It can be:

- `#YES#`: Verdi optimizes the isolation rule based on the following:
 - Source and sink are always same state in PST.
 - Source and sink are connected to same supply source.
- `#NO#`

The default value is `#NO#`.

- `NAME_PREFIX` (optional)

Specifies the name format for generated isolation instances or nets related to the implementation of isolation strategy.

- `NAME_SUFFIX` (optional)

Specifies the name format for generated isolation instances or nets related to the implementation of isolation strategy.

- `DIFF_SUPPLY_ONLY` (optional)

This option can be the following:

- #YES#(Default value)
- #NO#
- **FORCE (optional)**

This option can be the following:

- #YES#
- #NO#

The default value is #NO#

- **APPLIES_TO_BOUNDARY (optional)**

Generates `-applies_to_boundary` for the `set_isolation` UPF command. It can be:

- #YES#
- #NO#

The default value is #NO#

- **UPDATE (optional)**

Generates `-update` for the `set_isolation` UPF command. It can be:

- #YES#
- #NO#

The default value is #NO#

It is an error when `UPDATE=#YES#` but isolation object does not exist.

- **NAME** and **PD** are the keys to identify if you are specifying different attributes for the same object.
- The output CSV format file from command `ugo_optimize_power_intent` is same as the earlier description. However, you must modify some values of attribute, for example, **LOCATION**, **CLAMP_VALUE**, **ISO_SIGNAL**, **ISO_SENSE**, and so on. They are surrounded with '<' and '>'. You must specify proper values into the attribute.

Example: Isolation

NAME	PD	SRC	SINK	ISO_SIGNAL	EX_ELEM ENTS	CLAMP

ISO_EX	PD_SD	PD_SD	PD_AON	ISOEN	\${iso_pin_1}	0
ISO_SENSE	CELL	NAME_PREFIX	LOCATION	SUPPLY		
#LOW#	ISO_AN2	ISO_	#SELF#	PD_AON		

NAME	PD	SRC	SINK	ISO_SIGNAL	ELEMENTS	CLAMP	ISO_SENSE	LOCATION	CELL
iso_in	PD_CPU	SS1	SS2	U1/iso[0]	#INPUTS#	0	#HIGH#	#SELF#	ISO*NOR
iso_rx	PD_RX		SS1	U1/iso[1]	p1,p2	1	#HIGH#	#PARENT#	
iso_in	PC_CPU			U1/iso[0]	#BOTH#,p3				

Generated UPF

```
set_isolation ISO_EX -domain PD_SD -isolation_signal ISOEN -source
PD_SD.primary -sink PD_AON.primary -exclude_elements ${iso_pin_1}
-clamp_value 0 -isolation_sense low -isolation_supply PD_AON.primary
-location self -name_prefix ISO_
use_interface_cell ugo_interface_ISO_EX_PD_SD -strategy ISO_EX
-domain PD_SD -lib_cells ISO_AN2

set_isolation iso_in -domain PD_CPU -applies_to both -elements { p3 }
-src SS1 -sink SS2 -clamp_value 0 -iso_signal U1/iso[0] -isolation_sense
high -location self
use_interface_cell ugo_interface_ISO_IN_PD_CPU -strategy iso_in -domain
PD_CPU -lib_cells { ISO*NOR* }

set_isolation iso_rx -domain PD_RX -elements { p1 p2 } -sink SS1
-clamp_value 1 -iso_signal U1/iso[1] -isolation_sense high -location
parent
```

NAME	SRC	SINK	ISO_SIGNAL	INSERT_SIDE	EX_ELEMENTS	CLAMP	ISO_SENSE	CELL	NAME_PREFIX
ISO_EX	PD_SD	PD_AON	ISOEN	#FROM#	\${iso_pin_1}	0	#LOW#	ISO_AN2	ISO_

Generated UPF

```
set_isolation ISO_EX -domain PD_SD -isolation_signal ISOEN -source  
SS_PD_SD -sink SS_PD_AON -exclude_elements ${iso_pin_1} -clamp_value  
0 -isolation_sense low -isolation_supply SS_PD_AON -location self  
-name_prefix ISO_  
use_interface_cell ugo_interface_ISO_EX_PD_SD -strategy ISO_EX -domain  
PD_SD -lib_cells ISO_AN2
```

Level Shifter

This section describes the syntax of level shifter.

Syntax:

[Level Shifter]

NAME, [**PD**,
INSERT_SIDE, **ELEMENTS**, **EX_ELEMENTS**, **RULE**, **SRC**, **SINK**, **INPUT_SUPPLY**, **OUTPUT_SUPPLY**, **LOCATION**, **THRESHOLD**, **CELL**, **NO_SHIFT**, **NAME_PREFIX**, **NAME_SUFFIX**, **FORCE**, **APPLIES_TO_BOUNDARY**, **UPDATE**]

Description:

- **NAME**

Specifies the name of the level shifter rule.

- **PD**

Specifies the applied power domain of the level shifter rule.

- **INSERT_SIDE**

If **INSERT_SIDE** is present, (1) **SRC** and **SINK** are mandatory, and power domain should be specified (2) It is an error if **PD** or **LOCATION** is specified.

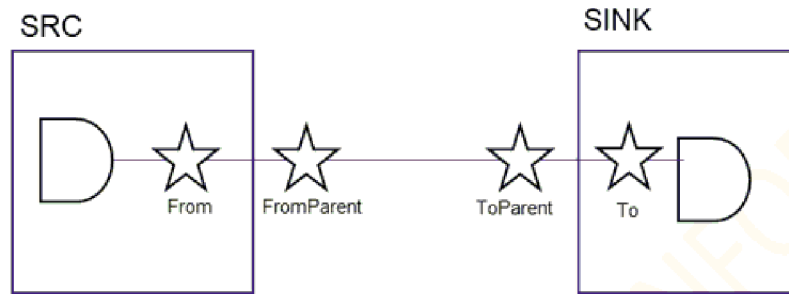
The following four keyword values can be specified:

#FROM# - **PD** is **SRC** location is self.

#FROMPARENT# - **PD** is **SRC**, location is parent.

#TOPARENT# - **PD** is **SINK**, location is parent.

#TO# - **PD** is **SINK**, location is self.



- **ELEMENTS (optional)**

Specifies the applied elements of the level shifter rule. You can specify the following keywords in this column:

- **#INPUTS#**: The elements are restricted to any port that the direction is input relative to the specified domain.
- **#OUTPUTS#**: The elements are restricted to any port that the direction is output relative to the specified domain.
- **#BOTH#**: The elements are restricted to any port that the direction is both relative to the specified domain.
- **#ALL#**: Every port on the interface of the domain is applied
- **\${<variable name>}**: See [User-Defined Variable](#) section.

Other than keywords, you can also specify list of instances or ports.

The values are appended to the level shifter object if it is previously defined. However, if multiple **#INPUTS#** and **#OUTPUTS#** are specified, only the last keyword value is applied. If keyword **#ALL#** is specified, all values are ignored except **#INPUTS#** and **#OUTPUTS#**.

- **EX_ELEMENTS (optional)**

Specifies a list of instances or ports to which the strategy does not apply.

- **RULE (optional)**

The elements are restricted to any port for which the driving and receiving logic have the specified voltage relationship. It can be:

- **#H2L#**
- **#L2H#**
- **#BOTH#**(Default value)

- **LOCATION (optional)**

Specifies the location for the inferred level shifter cell to be placed. It can be:

- `#SELF#`
- `#PARENT#`
- `#FANOUT#`

The default value is `#SELF#`.

- **SRC (optional)**

When specified, the elements are restricted to any port that is driven by the logic powered by a supply set that matches the specified one.

- **SINK (optional)**

When specified, the elements are restricted to any port that is present on a path for which the receiver supply set matches the specified one.

- **INPUT_SUPPLY (optional)**

Allows you to power the input portion of the level-shifter.

- **OUTPUT_SUPPLY (optional)**

Allows you to power the output portion of the level-shifter.

- **THRESHOLD (optional)**

When specified, the elements are restricted by any port for which the magnitude of the difference between the driver and receiver supply voltages can exceed a specified threshold value.

The default value is 0.000000

- **CELL (optional)**

Specifies the level shifter model to be used for implementation tool.

- **NO_SHIFT (optional)**

Specifies the no-shift. It can be:

- `#YES#`: Tool generates *-no_shift* for the power intent
- `#NO#`

The default value is `#NO#`.

If value is `#YES#`, (1) PD is mandatory, (2) `INSERT_SIDE` should not present.

- `CELL` (optional)
Specifies the isolation model to be used for implementation tool.
- `NAME_PREFIX` (optional)
Specifies the name format for the generated isolation instances or nets related to implementation of isolation strategy.
- `NAME_SUFFIX` (optional)
Specifies the name format for the generated isolation instances or nets related to implementation of isolation strategy.
- `NAME_PREFIX` (optional)
Specifies the name format for generated isolation instances or nets related to the implementation of isolation strategy.
- `NAME_SUFFIX` (optional)
Specifies the name format for generated isolation instances or nets related to the implementation of isolation strategy.
- `NAME` and `PD` are the keys to identify if you specify different attributes for the same object.
- `FORCE` (optional)
Generates `-force_shift` for the `set_level_shifter` UPF command. It could be:
 - `#YES#`
 - `#NO#`**Default is `#NO#`**
- `APPLIES_TO_BOUNDARY`
Generates `-applies_to_boundary` for the `set_level_shifter` UPF command. It could be:
 - `#LOWER#`
 - `#UPPER#`
 - `#BOTH#`

- UPDATE (optional)

Generates `-update` for the `set_isolation` UPF command. It could be:

- #YES#
- #NO#

Default is #NO#

It is an error if `UPDATE=#YES#` but isolation object does not exist.

Example: Level Shifter

NAME	ELEMENTS	PD	SRC	SINK	CELL	NAME_PREFIX	LOCATION
LS1	u_AON12_TOP/I1	PD_AON	PD_AON	PD_AON12	LS_LH1	pre_LS1	#PARENT#

Generated UPF

```
set_level_shifter LS1 -domain PD_AON -source PD_AON.primary
-sink PD_AON2.primary -elements u_AON12_TOP/I1 -location parent
-name_prefix pre_LS1
use_interface_cell ugo_interface_LS1_PD_AON -strategy LS1 -domain
PD_AON -lib_cells LS_LH1
```

NAME	ELEMENTS	INSERT_INSIDE	SRC	SINK	CELL	NAME_PREFIX
LS1	u_AON12_TOP/I1	#FROM#	PD_AON	PD_AON12	LS_LH1	pre_LS1

Generated UPF

```
set_level_shifter LS1 -domain PD_AON -source SS_PD_AON -sink SS_PD_AON12
-elements u_AON12_TOP/I1 -location self -name_prefix pre_LS1
use_interface_cell ugo_interface_LS1_PD_AON -strategy LS1 -domain
PD_AON -lib_cells LS_LH1
```

Retention

This section describes the syntax of retention.

Syntax:

```
[Retention]
```

```
NAME, PD[, TYPE, ELEMENTS, RES_SIGNAL, RES_SENSE, SAVE_SIGNAL, SAVE_SENSE, RES_CONDITION, SAVE_CONDITION, SUPPLY, CELL, LIB_MODEL, PORT_MAP, NO_RETENTION]
```

Description:

- NAME

Specifies the name of the retention rule.

- PD

Specifies the applied power domain of the retention rule.

- TYPE

Specifies the implementation cell type of the retention rule. It can be:

- #1_PIN#: Single-pin retention flip-flop
- #2_PIN#: Two-pin retention flip-flop
- #ZPR#: Zero-pin retention flip-flop
- #NORET#: No retention

- ELEMENTS

Specifies the applied elements of the retention rule. You can specify the following keywords in this column:

- #ALL#: Every port on the interface of the domain is applied
- \${<variable name>}: See [User-Defined Variable](#) section

Other than keywords, you can also specify list of instances or ports.

The values are appended to retention object if the object is previously defined. However, if keyword #ALL# is specified, all values are ignored.

- RES_SIGNAL

Specifies the restore signal for the retention cell

Verdi issues an error message if #2_PIN# or #1_PIN is specified for TYPE, but this column is not present in the table.

Both RES_SIGNAL and RES_SENSE should be specified together.

- `RES_SENSE`

Specifies the active level of the specified restore signal. It can be:

- `#HIGH#`
- `#LOW#`

Both `RES_SIGNAL` and `RES_SENSE` should be specified together.

- `SAVE_SIGNAL`

Specifies the save signal for the retention cell.

Verdi issues an error message if `#2_PIN#` is specified for `TYPE`, but this column is not present in the table.

Verdi issues an error message if `#1_PIN` is specified for `TYPE`, but this column is present in the table.

- `SAVE_SENSE`

Specifies the active level of the specified save signal. It can be:

- `#HIGH#`
- `#LOW#`

Verdi issues an error message if `#2_PIN#` is specified for `TYPE`, but this column is not present in the table.

Verdi issues an error message if `#1_PIN` is specified for `TYPE`, but this column is present in the table.

Both `SAVE_SIGNAL` and `SAVE_SENSE` should be specified together.

- `RES_CONDITION`

Specifies a Boolean expression. It cannot be a Tcl variable.

- `SAVE_CONDITION`

Specifies a Boolean expression. It cannot be a Tcl variable.

- `SUPPLY` (optional)

The power domain, supply set, or supply net that powers the retention cell.

- `CELL` (optional)

Specifies the retention model to be used for implementation tool.

- `LIB_MODEL`

Specifies the name of the library cell or behavioral model.

- `PORT_MAP`

Specifies the name defined in the [Port Map] section.

- `NO_RETENTION` (optional)

Specifies no retention. It can be:

- `#YES#`: Tool generates `-no_retention` for the power intent.
- `#NO#`.

The default value is `#NO#`.

It is an error if `#NORET#` is specified for `TYPE`, but this column does not present in the table or the value is `#NO#`.

`NAME` and `PD` are the keys to identify if you are specifying different attributes for same object.

Example: Retention

NAME	PD	ELEMENTS	RESIGNAL	SAVESENSE	SAVE_SIGNAL	SAVESENSE	SUPPLY	CELL
SR_XXX	PD_VA0	.../inst_IP_OFF0	.../yy_RETFF_NRSTR	#LOW#	../yy_RETFF_SAVE	#HIGH#	VDD	"RSLVT..."

Generated UPF

```
set_retention SR_XXX -domain PD_VA0 -elements ".../inst_IP_OFF0"
  -restore_signal { ../yy_RETFF_NRSTR low } -save_signal
  { ../yy_RETFF_SAVE high } -supply SS_PD_TOP
map_retention_cell SR_XXX -domain PD_VA0 -lib_cells "RSLVT"
```

Repeater

This section describes the syntax of repeater.

Syntax:

```
[Repeater]
NAME, PD, SUPPLY [ , ELEMENTS, EX_ELEMENTS, SRC, SINK, CELL, NAME_PREFIX, NAME_SUFFIX]
```

Description:

- NAME

Specifies the name of the repeater rule.

- PD

Specifies the applied power domain of the repeater rule.

- SUPPLY

The power domain, supply set, or supply net that powers the repeater cell.

- ELEMENTS

Specifies the applied elements of the repeater rule. You can specify the following keywords in this column:

- #INPUTS#: The elements are restricted to any port whose direction is input relative to the specified domain.
- #OUTPUTS#: The elements are restricted to any port whose direction is output relative to the specified domain.
- #ALL#: Every port on the interface of the domain is applied.
- #BOTH#: The elements are restricted to any port that the direction is both relative to the specified domain.

Other than keywords you can also specify list of instances or ports.

Values are appended to repeater object if it is previously defined. However, if multiple #INPUTS# and #OUTPUTS# are specified, only the last keyword value is applied. If keyword #ALL# is specified, all values are ignored except #INPUTS# and #OUTPUTS#.

- RES_SIGNAL

Specifies the restore signal for the retention cell

Verdi issues an error message if #2_PIN# or #1_PIN is specified for TYPE, but this column is not present in the table.

Verdi issues an error message if #ZPR# is specified for TYPE, but this column is present in the table.

Both RES_SIGNAL and RES_SENSE should be specified together.

- EX_ELEMENTS (optional)

Specifies a list of instances or ports to which the strategy does not apply.

- **SRC (optional)**
The elements are restricted to any port that is driven by logic powered by a supply set that matches the specified one.
- **SINK (optional)**
The elements are restricted to any port that is present on a path for which the receiver supply set matches the specified one.
- **CELL (optional)**
Specifies the repeater model to be used for implementation tool.
- **NAME_PREFIX (optional)**
Specifies the name format for generated isolation instances or nets related to implementation of isolation strategy.
- **NAME_SUFFIX(optional)**
Specifies the name format for generated isolation instances or nets related to implementation of isolation strategy.

NAME and *PD* are the keys to identify if you are specifying different attributes for same object.

Example: Repeater

NAME	PD	ELEMENTS	SUPPLY
rep_rx	PD_RX	#OUTPUT#	SS1

Generated UPF

```
set_repeater rep_rx -domain PD_RX -applies_to outputs -repeater_supply  
SS1
```

Load UPF

This section describes the syntax of load UPF.

Syntax:

```
[Load UPF]  
UPF, SCOPE, [PRE_SCRIPT_PRE_SCOPE, PRE_SCRIPT_POST_SCOPE,  
POST_SCRIPT_PRE_SCOPE, POST_SCRIPT_POST_SCOPE, LOADER_TOP]
```


Description:

- `UPF`
Specifies the UPF file to be loaded.
- `SCOPE`
Specifies the scopes to load the UPF file.
- `PRE_SCRIPT_PRE_SCOPE` (optional)
Specifies the scripts that are sourced before setting to the corresponding scope.
- `PRE_SCRIPT_POST_SCOPE` (optional)
Specifies the scripts that are sourced before loading the UPF file.
- `POST_SCRIPT_PRE_SCOPE` (optional)
Specifies the scripts that are sourced after loading the UPF file.
- `POST_SCRIPT_POST_SCOPE` (optional)
Specifies the scripts that are sourced after setting scope back to top.
- `LOADER_TOP` (optional)
Specifies a (list of) top UPF name. There are some cases that would need to load the UPF. If this option is specified, then only the loader top file name is matched and the `load_upf` command is generated.

Example: Load UPF

UPF	SCOPE	PRE_SCRIPT PRE_SCOPE	PRE_SCRIPT POST_SCOPE	POST_SCRIPT PRE_SCOPE	POST_SCRIPT POST_SCOPE
test.upf	v0	pre0.tcl	pre1.tcl	post0.tcl	post1.tcl,post2.tcl
macro.upf	\${macros}				

Generated UPF

```
foreach __ugo_scope__ [concat v0] {
    set __ugo_bkcnt__ "[expr {[llength [split ${__ugo_scope__} /]]}]"
    source pre0.tcl
    set_scope ${__ugo_scope__}
    source pre1.tcl
    load_upf test.upf
    source post0.tcl
    set_scope [string repeat ../ $__ugo_bkcnt__]
```

```

        source post1.tcl
        source post2.tcl
    }

    foreach __ugo_scope__ [concat ${macros}] {
        set __ugo_bkcnt__ "[expr {[llength [split ${__ugo_scope__} /]]}]"
        set_scope ${__ugo_scope__}
        source macro.upf
        set_scope [string repeat ../ $__ugo_bkcnt__]
    }

```

Connection

This section describes the syntax of connection.

Syntax:

```

[Connection]
HIGH_CONN, SCOPE, LOW_CONN [, PINNUM, TYPE, IS_RTL]

```

Description:

- HIGH_CONN

Specifies the highconn to make connection. It should be supply set, supply net, or logic net. The value cannot be a Tcl variable.

- SCOPE

Specifies the scopes to connect the lowconn.

- PINNUM (optional)

Specifies the number of connections for the scope.

It is an error if there is mismatch with data input.

- LOW_CONN

Specifies the loconn to connect with highconn. It should be supply set, supply port, or logic port. The value cannot be a Tcl variable.

- TYPE (optional)

Specifies the type of the object. It can be:

- #SS#: associate_supply_set with HIGH_CONN and LOW_CONN
- #SN#: connect_supply_net with HIGH_CONN and LOW_CONN
- #LN#: connect_logic_net with HIGH_CONN and LOW_CONN

- IS_RTL(optional)

The type of scope is RTL It can be:

- #YES#
- #NO#
- #YES# is specified when the SCOPE is in RTL or lib cell instance
- #NO# is specified when the SCOPE is not in RTL.

The default value is #YES#.

- TYPE, IS_RTL, HIGH_CONN and LOW_CONN are the keys to identify if you are specifying different attributes for the same object.

Example: Connection

SCOPE	TYPE	HIGH_CONN	LOW_CONN	IS_RTL
m_cpu, m_cpu2	#SN#	VDD	VDD	#YES#
	#SN#	VSS	VSS	
	#SN#	VDDC	VDDC	
	#LN#	top_logic_net	low_logic_net	

Generated UPF

```
#[Connection]
foreach i [concat m_cpu1 m_cpu2] {
    connect_supply_net VDDC -ports ${i}/VDDC
    connect_supply_net VDD -ports ${i}/VDD
    connect_supply_net VSS -ports ${i}/VSS
    connect_supply_net top_logic_net -ports ${i}/      top_logic_net
}
```

Port Map

This section describes the syntax of attribute.

Syntax:

```
[Port Map]
NAME, MAPPING
```

Description:

- NAME
Specifies the name of Port Map, which is used for [Retention]:PORT_MAP.
If the value is empty, refers the value in the previous line.
- MAPPING
Specifies a list value.

Syntax

```
{<Port Ref> <Net Ref>}  
"{<Port1 Ref> <Net1 Ref>}, {<Port2 Ref> <Net2 Ref>},..."
```

Should have a space between <Port Ref> and <Net Ref>.

Should surround {} with <Port Ref> and <Net Ref>.

Example1: Port Map

[Port Map]	
NAME	MAPPING
model1	"{CL UPF_GENERIC_CLOCK}, {D UPF_GENERIC_DATA}"

[Retention]				
NAME	PD	CELL	LIB_MODEL	PORT_MAP
ret_top	PD_TOP	"xxx, yyy, zzz"	GPSDRFF_D1	model1

Generated UPF

```
# [MAP_RETENTION_CELL]  
map_retention_cell ret_top \  
-domain PD_TOP \  
-lib_cells "xxx zzz yyy" \  
-lib_model_name GPSDRFF_D1 \  
-port_map {  
  {CL_UPF_GENERIC_CLOCK}  
  {D_UPF_GENERIC_DATA}  
}
```

Example2: Port Map

[Port Map]	
NAME	MAPPING
C54L08L10_Model	{CL UPF_GENERIC_CLOCK}
	{D UPF_GENERIC_DATA}
	{RETN uacro_a0_logic/ss_top/uGCR/uEPU/pd_pd4_restore_no}
	{SE 0}
	{SI 0}
	{VDD ss_PD_N_UPP_PD4.power}
	{VDDG ss_PD0.power}
	{VSS ss_PD_N_UPP_PD4.ground}

[Retention]			
NAME	PD	LIB_MODEL	PORT_MAP
ret_top	PD_TOP	GPSDRFF_D1	C54L08L10_Model

Generated UPF

```
# [MAP_RETENTION_CELL]
map_retention_cell ret_top \
  -domain PD_TOP \
  -lib_model_name GPSDRFF_D1 \
  -port_map {
    {CL UPF_GENERIC_CLOCK}
    {D UPF_GENERIC_DATA}
    {Q UPF_GENERIC_OUTPUT}
    {RETN uacro_a0_logic/ss_top/uGCR/uEPU/pd_pd4_restore_no}
    {SE 0}
    {SI 0}
    {VDD ss_PD_N_UPP_PD4.power}
    {VDDG ss_PD0.power}
    {VSS ss_PD_N_UPP_PD4.ground}
  }
```

Parameters

This section describes the syntax of a Parameters.

Syntax:

```
[Parameters]  
NAME, MAPPING
```

Description:

- NAME

Specifies the name of the Parameter that is used for [Apply Power Model]:PARAMETERS. If the value is empty, the value in the previous line is referred to.

- MAPPING

Specifies a list value. Following is the syntax:

- {<power model parameter> <design object>}
- "{<power model parameter1> <design object1>}, {<power model parameter2> <design object2>},..."
- Specify a space between <power model parameter> with <design object>
- You must surround {} with <power model parameter> and <design object>.

Apply Power Model

This section describes the syntax of Apply Power Model.

Syntax:

```
[Apply Power Model]  
MODEL_NAME, [ELEMENTS, SUPPLY_MAP, PORT_MAP, PARAMETERS]
```

Description:

- MODEL_NAME

Specifies the applied model name. If the value is empty, the value in the previous line is referred to.

- ELEMENTS

Specifies the instances to apply the power model. If the values of MODEL_NAME and ELEMENTS are empty, this value refers to the value in the previous line.

- SUPPLY_MAP

Specifies the supply map names defined in `[SUPPLY_MAP]`.

- `PORT_MAP`

Specifies the port map names defined in `[PORT_MAP]`.

- `PARAMETERS`

Specifies the parameter names defined in `[PARAMETERS]`.

Attribute

This section describes the syntax of attribute.

Syntax:

```
[Attribute]
```

```
PORTS, [RELATED_SUPPLY, DRIVER_SUPPLY, RECEIVER_SUPPLY, REPEATER_SUPPLY,  
MODEL, IS_ANALOG, APPLIES_TO, CLAMP, RELATED_POWER, RELATED_GROUND, RELATED_POWER_PORT,  
RELATED_GROUND_PORT,  
EX_PORTS]
```

```
[Attribute]
```

```
ELEMENTS, [DRIVER_SUPPLY, RECEIVER_SUPPLY, APPLIES_TO, CLAMP, RELATED_POWER,  
RELATED_GROUND, EX_ELEMENTS, EX_PORTS]
```

```
[Attribute]
```

```
ATTRIBUTE, VALUE, [MODEL, OBJECTS, IS_HARD_MACRO,  
IS_SOFT_MACRO]
```

Description:

- `PORTS`

Specifies the ports to apply the attribute.

- `ELMENETS`

Specifies the instances to apply the attribute.

- `RELATED_SUPPLY`

Specifies the supply set used by related supply net for the specified ports. If this attribute is specified, the `set_related_supply_net` UPF command is generated.

- DRIVER_SUPPLY

Specifies the supply set used by drivers of the ports. If this attribute is specified, the `set_port_attribute` UPF command is generated.

- RECEIVER_SUPPLY

Specifies the supply set used by receivers of the ports. If this attribute is specified, the `set_port_attribute` UPF command is generated.

- REPEATER_SUPPLY

Specifies the supply set used by repeaters of the ports. If this attribute is specified, the `set_port_attribute` UPF command is generated.

- IS_ANALOG

The type of port is (not) analog. It can be:

- `#YES#`
- `#NO#`

- APPLIES_TO

- `#INPUTS#`: Generates `-applies_to` inputs.
- `#OUTPUTS#`: Generates `-applies_to` outputs.
- `#INOUTS#`: Generate `-applies_to` both.

- CLAMP

Specifies the clamp value for the port.

- RELATED_POWER

Specifies the supply net name for `set_related_supply_net -power`.

- RELATED_GROUND

Specifies the supply net name for `set_related_supply_net -ground`.

- RELATED_POWER_PORT

Specifies power port name for attributed port.

- RELATED_GROUND_PORT

Specifies ground port name for attributed port.

- EX_ELEMENTS

Specifies a list of excluded elements from the command.

- `EX_PORTS`

Specifies a list of ports to be excluded from the command.

- `ATTRIBUTE`

The string of the attribute name. It can be:

- `UPF_dont_touch`
- `enable_bias`
- `enable_state_propagation_in_add_power_state`
- `lower_domain_boundary`
- `domain_interface_def`

If this attribute is specified, the `set_design_attribute` UPF command is generated.

The value for `OBJECTS` attribute is applied to `-elements` in the `set_design_attribute` command.

- `VALUE`

Specifies the value string applied to the attribute. It only allows `TRUE`, `true`, `FALSE`, `false` for attributes `enable_bias`, `enable_state_propagation_in_add_power_state`, and `lower_domain_boundary`.

- `MODEL`

- Specifies the models to be attributed. This option is a mandatory for the `UPF_dont_touch` attribute.
- Verdi issues an error message if `MODELS` and `OBJECTS` are empty.
- Verdi issues an error message if this attribute is used together with `DRIVER_SUPPLY/RECEIVER_SUPPLY`.

- `OBJECTS`

Specifies the instances or signal names. Verdi issues an error message if `MODELS` and `OBJECTS` are empty.

- You can use only `."` or `#ALL#` for the `enable_bias` attribute.

- `IS_HARD_MACRO`

Generates `-is_hard_macro` for `set_design_attributes`. It can be:

- `#YES#`
- `#NO#`
- Default is `#NO#`
- `IS_SOFT_MACRO`

Generates `-is_soft_macro` for `set_design_attributes`. It can be:

- `#YES#`
- `#NO#`
- Default is `#NO#`

Example1: Attribute

PORTS	DRIVER_SUPPLY	RECEIVER_SUPPLY
"DOUT0_c DOUT0_t ..."		VDDH
"DIN0_c DIN0_t ..."	VDDH	
"DINOUT0 DINOUT1 ..."	VDDH	VDDH

Generated UPF

```
set_port_attribute -ports "DOUT0_c DOUT0_t ..." -receiver_supply
SS_PD_VDDH -repeater_supply SS_PD_VDDH
set_port_attribute -ports "DIN0_c DIN0_t" -driver_supply SS_PD_VDDH
-repeater_supply SS_PD_VDDH
set_port_attribute -ports "DINOUT0 DINOUT1" -driver_supply SS_PD_VDDH
-receiver_supply SS_PD_VDDH
```

Example2: Attribute

ATTRIBUTE	VALUE	MODEL	OBJECTS
UPF_dont_touch	TRUE	CPU	a
enable_bias	TRUE		.

Generated UPF

```
set_design_attribute -models { CPU } -elements { a } -attribute  
{ UPF_dont_touch TRUE }  
set_design_attribute -elements { . } -attribute { enable_bias TRUE }
```

LSGEN Config

This section describes the syntax of LSGEN Configuration.

Syntax:

```
[LSGEN Config]  
  
SRC_PD, SINK_PD[, LS_THRESHOLD, LS_LOCATION, LS_PLACE, LS_CELL,  
PORTS]  
  
PD, DIFF_LS_LOCATION  
  
EX_ELEMENTS
```

Description:

- SRC_PD
Specifies the ports to apply the attribute.
- SINK_PD
Specifies the sink domain constraint of the crossover. Specifying wildcards is supported.
- LS_THRESHOLD
Specifies the threshold of a level shifter.
- LS_LOCATION
Specifies the location of a level shifter. It can be:
 - #SELF#
 - #PARENT#
- LS_PLACE
Specifies the place where the level shifter is targeted. It can be:
 - source
 - sink

- `LS_CELL`
Specifies the level shifter cell.
- `PORTS`
Specifies the port constraint.
- `PD`
Specifies the target domain where different isolation location is required. It is used with `DIFF_LS_LOCATION`.
- `DIFF_LS_LOCATION`
Specifies the location of the target `PD`.
It is specified for the top power domain as `#SELF#`. It can be:
 - `#SELF#`
 - `#PARENT#`

Example: LSGEN Config

SRC_PD	SINK_PD	LS_THRESHOLD	LS_CELL	LS_LOCATION	PORTS
PD_TOP	*	0.0	Cell0	#SELF#	“sipo,piso”
PD_SIPO	PD_TOP	0.0	Cell1	#SELF#	
PD_PISO	*	0.0	Cell2	#SELF#	

Usage

```
ugo_generate -u test.upf -ls_config ls_config.json -ls_info  
level_shifter.info  
ugo_optimize_power_intent -lsген -ls_config ls_config.json ...
```

UPF Patch

This section describes the syntax of UPF Patch.

Syntax:

```
[UPF Patch]  
  
TEXT, FILE, TYPE
```

Description:

- `TEXT`

Specifies text. It can be any word.

- `FILE`

Specifies an existing file. The tool extracts the whole content of `FILE` and fills it into `TEXT`.

- `TYPE`

Specifies the type. It can be:

- `#BEGIN#`
- `#END#`
- Default is `#BEGIN#`

This section allows you to inject any text or code into the generated UPF at the beginning or end.

The generated order follows the same as CSV with the same `TYPE`.

Support for Keywords

Keywords in the content are replaced by the corresponding contents when UPF is generated.

- `#FILE#`

Specifies the UPF file name.

- `#VERSION#`

Specifies the version of UPFA.

- `#AUTHOR#`

Specifies the author who generates the UPF. It is the user login machine currently. It is equal to executing the command `whoami` in Linux.

- `#TIME#`

The time is decided when the `ugo_generate` command is executed. All the files would have the same time if generated at the same execution. The time format is `YYYY/MM/DD HH:MM:SS`. The time zone is dependent on the local machine.

Example: UPF Patch

TYPE	TEXT	FILE
------	------	------

#BEGIN#	# This is comment	
	# Second line: #AUTHOR#	
	# UPFA Version: #VERSION#	
#END#	# end of UPF file	
#BEGIN#		
#END#		upf_front_path.txt
		\${UPF_DIR}/upf_end_patch.txt

Generated UPF

```
# This is comment
# Second line: Jack
# UPFA Version: U-2023.03-SP2-1
... content of upf_front_patch.txt

...

# end of UPF file
... content of ${UPF_DIR}/upf_end_patch.txt
```

Global PST Template

This section describes the syntax of Global PST Template.

Syntax:

```
[Global PST Template]

#SS#,STATE, <supply function>*[,SIMSTATE,LEGAL]

#PSG#,STATE, <supply set function|supply set|power state group>*
[,LEGAL]
```

Description:

- #SS#

The value should be the name of the supply set.

- #PSG#

The value should be the name of the power state group.

- STATE
 - The value should be the name of a power state.
 - The value cannot be a Tcl variable.
- supply function

The value should be:

 - #OFF#
 - #PARTIAL_ON{<voltage>}#
 - <voltage>
 - The value cannot be a Tcl variable.
- supply set function
 - The value should be the supply set's function.
 - The value cannot be a Tcl variable.
 - If a supply function is specified in #PSG#, UPF2.0 is force generated instead of UPF3.0.
- supply set
 - The value should be the power state of <supply set>.
 - The value cannot be a Tcl variable.
- power state group
 - The value should be the power state of <power state group>.
 - The value cannot be a Tcl variable.
- SIMSTATE(optional)

Specifies a simstate for the power states associated with a supply set. It can be:

 - NORMAL
 - CORRUPT
 - CORRUPT_ON_CHANGE
 - CORRUPT_STATE_ON_CHANGE
 - CORRUPT_STATE_ON_ACTIVITY

- CORRUPT_ON_ACTIVITY
- NOT_NORMAL

The value cannot be a Tcl variable.

- **LEGAL(optional)**

Specifies legal or illegal for the transition. The value can be:

- #YES#.
- #NO#

Example:Global PST Template

Global PST Template: *global_pst.csv*

#SS#	STATE	power	ground
SS1	ON	1.08	
SS1	OFF	#OFF#	
SS1	GON		0.0
SS2	ON	1.08	
SS2	OFF	#OFF#	
SS3	ON	1.08	
SS3	OFF	#OFF#	
SS4	ON	1.08	
SS4	OFF	#OFF#	

#PSG#	STATE	SS1.power	SS2.power	SS3.power	SS4.power	SS1.ground
psg_1	p1	ON	ON	ON	ON	GON
psg_1	p2	ON	ON	ON	OFF	GON
psg_1	p3	ON	ON	OFF	ON	GON
psg_1	p4	ON	OFF	OFF	OFF	GON

Supply: partition1.csv

NAME	power	ground
SS1	VDD1	VSS
SS2	VDD2	VSS

Supply: partition2.csv

NAME	power	ground
SS1	VDD1	VSS
SS2	VDD2	VSS
SS4	VDD4	VSS

Supply: top.csv

NAME	power	ground
SS1	VDD1	VSS
SS2	VDD2	VSS
SS3	VDD3	VSS
SS4	VDD4	VSS

Generate UPF command sequence

```
### generate partition1 UPF ###
ugo_target_scope partition1
ugo_read_power_intent -c partition1.csv
ugo_read_power_intent -c global_pst.csv
ugo_check_power_intent
ugo_generate -u partition1.upf -g 3 -n -d
ugo_remove_power_intent

### generate partition2 UPF ###
ugo_target_scope partition2
ugo_read_power_intent -c partition2.csv
ugo_read_power_intent -c global_pst.csv
ugo_check_power_intent
ugo_generate -u partition2.upf -g 3 -n -d
ugo_remove_power_intent
```

```
### generate TOP UPF ###
ugo_read_power_intent -c top.csv
ugo_read_power_intent -c global_pst.csv
ugo_check_power_intent
ugo_generate -u top.upf -g 3 -n -d
```

Generated UPF

partition1.upf

```
# [Design Attribute]
set_design_attributes -elements { . } -attribute
{ enable_state_propagation_in_add_power_state TRUE }

# [Supply Set]
create_supply_set SS1
create_supply_set SS2

# [Supply Set]
create_supply_net VSS
create_supply_net VDD1
create_supply_set SS1 -function { ground VSS } -function { power VDD1 }
-update
create_supply_net VDD2
create_supply_set SS2 -function { ground VSS } -function { power VDD2 }
-update

# [Power Spec]
add_power_state -supply SS1 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS1 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS1 -state { GON -supply_expr { ground ==
{FULL_ON 0} } } -update
add_power_state -supply SS2 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS2 -state { OFF -supply_expr { power == OFF } }
-update
create_pst psg_1 -supplies { SS1.ground SS1.power SS2.power }
add_pst_state p1 -pst psg_1 -state { GON ON ON }
add_pst_state p4 -pst psg_1 -state { GON ON OFF }
```

partition2.upf

```
# [Design Attribute]
set_design_attributes -elements { . } -attribute
{ enable_state_propagation_in_add_power_state TRUE }

# [Supply Set]
create_supply_set SS1
create_supply_set SS2
create_supply_set SS4
```

```
# [Supply Set]
create_supply_net VSS
create_supply_net VDD1
create_supply_set SS1 -function { ground VSS } -function { power VDD1 }
-update
create_supply_net VDD2
create_supply_set SS2 -function { ground VSS } -function { power VDD2 }
-update
create_supply_net VDD4
create_supply_set SS4 -function { ground VSS } -function { power VDD4 }
-update

# [Power Spec]
add_power_state -supply SS1 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS1 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS1 -state { GON -supply_expr { ground ==
{FULL_ON 0} } } -update
add_power_state -supply SS2 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS2 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS4 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS4 -state { OFF -supply_expr { power == OFF } }
-update
create_pst psg_1 -supplies { SS1.ground SS1.power SS2.power SS4.power }
add_pst_state p1 -pst psg_1 -state { GON ON ON ON }
add_pst_state p2 -pst psg_1 -state { GON ON ON OFF }
add_pst_state p4 -pst psg_1 -state { GON ON OFF OFF }

### top.upf ###

# [Design Attribute]
set_design_attributes -elements { . } -attribute
{ enable_state_propagation_in_add_power_state TRUE }

# [Supply Set]
create_supply_set SS1
create_supply_set SS2
create_supply_set SS3
create_supply_set SS4

# [Supply Set]
create_supply_net VSS
create_supply_net VDD1
create_supply_set SS1 -function { ground VSS } -function { power VDD1 }
-update
create_supply_net VDD2
create_supply_set SS2 -function { ground VSS } -function { power VDD2 }
-update
```

```
create_supply_net VDD3
create_supply_set SS3 -function { ground VSS } -function { power VDD3 }
-update
create_supply_net VDD4
create_supply_set SS4 -function { ground VSS } -function { power VDD4 }
-update

# [Power Spec]
add_power_state -supply SS1 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS1 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS1 -state { GON -supply_expr { ground ==
{FULL_ON 0} } } -update
add_power_state -supply SS2 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS2 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS3 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS3 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS4 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS4 -state { OFF -supply_expr { power == OFF } }
-update
create_pst psg_1 -supplies { SS1.ground SS1.power SS2.power SS3.power
SS4.power }
add_pst_state p1 -pst psg_1 -state { GON ON ON ON ON }
add_pst_state p2 -pst psg_1 -state { GON ON ON ON OFF }
add_pst_state p3 -pst psg_1 -state { GON ON ON OFF ON }
add_pst_state p4 -pst psg_1 -state { GON ON OFF OFF OFF }
```

Global Supply Template

This section describes the syntax of the Global Supply Template.

Syntax:

```
[Global Supply Template]
PARTITION, HIGH_CONN, LOW_CONN,
[<supply function>*]
```

Description:

- PARTITION

Scopes of LOW_CONN supply set.

- `HIGH_CONN`
New supply set name in the top level.
- `LOW_CONN`
Origin supply set name in the sub-scope.
- `Supply_function(optional)`
Supply function of the supply set (`HIGH_CONN`).

Example: Global Supply Template

Supply: partition1.csv, the spec of SS2 here is 1.08V

NAME	power	ground
SS1	VDD1	VSS
SS2	VDD2	VSS

Supply: partition2.csv, the spec of SS2 here is 1.1V

NAME	power	ground
SS1	VDD1	VSS
SS2	VDD2	VSS
SS4	VDD4	VSS

Supply: top.csv

NAME	power	ground
SS1	VDD1	VSS
SS2_par1	VDD2_par1	VSS
SS2_par2	VDD2_par2	VSS
SS3	VDD3	VSS
SS4	VDD4	VSS

Global PST Template: global_pst.csv

#SS#	STATE	power	ground
SS1	ON	1.08	
SS1	OFF	#OFF#	
SS1	GON		0.0
SS2_par1	ON	1.08	
SS2_par1	OFF	#OFF#	
SS2_par2	ON	1.1	
SS2_par2	OFF	#OFF#	
SS3	ON	1.08	
SS3	OFF	#OFF#	
SS4	ON	1.08	
SS4	OFF	#OFF#	

#PSG#	STATE	SS1.power	SS2_par1.power	SS2_par2.power	SS3.power	SS4.power
psg_1	p1	ON	ON	ON	ON	GON
psg_1	p2	ON	ON	ON	OFF	GON
psg_1	p3	ON	OFF	ON	ON	GON
psg_1	p4	ON	OFF	ON	OFF	GON

Global Supply Template

HIGH_CONN	LOW_CONN
SS2_par1	SS2
SS2_par2	SS2

Generated UPF

```
### partition1.upf ###
```

```
# [Design Attribute]
set_design_attributes -elements { . } -attribute
{ enable_state_propagation_in_add_power_state TRUE }

# [Supply Set]
create_supply_set SS1
create_supply_set SS2

# [Supply Set]
create_supply_net VSS
create_supply_net VDD1
create_supply_set SS1 -function { ground VSS } -function { power VDD1 }
-update
create_supply_net VDD2
create_supply_set SS2 -function { ground VSS } -function { power VDD2 }
-update

# [Power Spec]
add_power_state -supply SS1 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS1 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS1 -state { GON -supply_expr { ground ==
{FULL_ON 0} } } -update
add_power_state -supply SS2 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS2 -state { OFF -supply_expr { power == OFF } }
-update
create_pst psg_1 -supplies { SS1.ground SS1.power SS2.power }
add_pst_state p1 -pst psg_1 -state { GON ON ON }
add_pst_state p4 -pst psg_1 -state { GON ON OFF }

### partition2.upf ###

# [Design Attribute]
set_design_attributes -elements { . } -attribute
{ enable_state_propagation_in_add_power_state TRUE }

# [Supply Set]
create_supply_set SS1
create_supply_set SS2
create_supply_set SS4

# [Supply Set]
create_supply_net VSS
create_supply_net VDD1
create_supply_set SS1 -function { ground VSS } -function { power VDD1 }
-update
create_supply_net VDD2
create_supply_set SS2 -function { ground VSS } -function { power VDD2 }
-update
create_supply_net VDD4
```

```

create_supply_set SS4 -function { ground VSS } -function { power VDD4 }
-update

# [Power Spec]
add_power_state -supply SS1 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS1 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS1 -state { GON -supply_expr { ground ==
{FULL_ON 0} } } -update
add_power_state -supply SS2 -state { ON -supply_expr { power == {FULL_ON
1.1} } }
add_power_state -supply SS2 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS4 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS4 -state { OFF -supply_expr { power == OFF } }
-update
create_pst psg_1 -supplies { SS1.ground SS1.power SS2.power SS4.power }
add_pst_state p1 -pst psg_1 -state { GON ON ON ON }
add_pst_state p2 -pst psg_1 -state { GON ON ON OFF }
add_pst_state p3 -pst psg_1 -state { GON ON OFF ON }
add_pst_state p4 -pst psg_1 -state { GON ON OFF OFF }

### top.upf ###

# [Design Attribute]
set_design_attributes -elements { . } -attribute
{ enable_state_propagation_in_add_power_state TRUE }

# [Supply Set]
create_supply_set SS1
create_supply_set SS2_par1
create_supply_set SS2_par2
create_supply_set SS3
create_supply_set SS4

# [Supply Set]
create_supply_net VSS
create_supply_net VDD1
create_supply_set SS1 -function { ground VSS } -function { power VDD1 }
-update
create_supply_net VDD2_par1
create_supply_set SS2_par1 -function { ground VSS } -function { power
VDD2_par1 } -update
create_supply_net VDD2_par2
create_supply_set SS2_par2 -function { ground VSS } -function { power
VDD2_par2 } -update
create_supply_net VDD3
create_supply_set SS3 -function { ground VSS } -function { power VDD3 }
-update
create_supply_net VDD4

```



```
create_supply_set SS4 -function { ground VSS } -function { power VDD4 }
-update

# [Power Spec]
add_power_state -supply SS1 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS1 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS1 -state { GON -supply_expr { ground ==
{FULL_ON 0} } } -update
add_power_state -supply SS2_par1 -state { ON -supply_expr { power ==
{FULL_ON 1.08} } }
add_power_state -supply SS2_par1 -state { OFF -supply_expr { power ==
OFF } } -update
add_power_state -supply SS2_par2 -state { ON -supply_expr { power ==
{FULL_ON 1.1} } }
add_power_state -supply SS2_par2 -state { OFF -supply_expr { power ==
OFF } } -update
add_power_state -supply SS3 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS3 -state { OFF -supply_expr { power == OFF } }
-update
add_power_state -supply SS4 -state { ON -supply_expr { power == {FULL_ON
1.08} } }
add_power_state -supply SS4 -state { OFF -supply_expr { power == OFF } }
-update
create_pst psg_1 -supplies { SS1.ground SS1.power SS2_par1.power
SS2_par2.power SS3.power SS4.power }
add_pst_state p1 -pst psg_1 -state { GON ON ON ON ON ON }
add_pst_state p2 -pst psg_1 -state { GON ON ON ON ON OFF }
add_pst_state p3 -pst psg_1 -state { GON ON ON OFF ON ON }
add_pst_state p4 -pst psg_1 -state { GON ON OFF OFF ON OFF }
```

Design Hierarchy

This section describes design information and generates `set_design_top`.

Syntax:

```
Design Hier]
SCOPE[, MODEL]
```

Description:

- SCOPE

Specifies the full hierarchical name of the scope. You can specify `%TOP%` for generating the `set_design_top` UPF command and specify the design top in `MODEL`.

- `MODEL(optional)`

Specifies the module name.

Example: Design Hierarchy

SCOPE	MODEL
<code>%TOP%</code>	<code>bit_slice</code>

Generated UPF

```
set_design_top bit_slice
```

Set Equivalent

This section describes the `set_equivalent` UPF command generation.

Syntax:

```
[Set Equivalent]  
NETS, SETS[, FUNC_ONLY]
```

Description:

- `NETS`

Specifies the equivalent supply net list.

- `SETS`

Specifies the equivalent supply set list.

- `FUNC_ONLY(optional)`

Specifies the function. It can be:

- `#YES#`
- `#NO#`

Note:

- You can only specify either `NETS` or `SETS`, not both.
- This syntax is supported from UPF 2.1 and above versions. A warning message is issued if a lower UPF version is used.

Example: Set Equivalent

NETS
<i>VDD, VDD1, VDD2</i>

Generated UPF

```
set_equivalent -nets "VDD VDD1 VDD2"
```

ISOGEN Config

This section describes the syntax of ISOGEN Config.

Syntax:

```
[ISOGEN Config]  
SRC_PD, ISO_SIGNAL [, SINK_PD, ISO_SUPPLY, ISO_LOCATION, ISO_SENSE,  
  ISO_CLAMP, ISO_PLACE, ISO_CELL, PORTS]  
[ISOGEN Config]  
SCOPE, UPF, ISO_CONFIG, ISO_CONSTRAINT
```

Description:

- SRC_PD
Specifies the source domain constraint of crossover.
- ISO_SIGNAL
Specifies control signal of isolation.
- SINK_PD
Specifies the sink domain constraint of crossover. Specifying wildcards is supported.
- ISO_SUPPLY
Specifies the supply set of isolation.
- ISO_LOCATION
Specifies the location of isolation. It can be:
 - #SELF#
 - #PARENT#

- `ISO_SENSE`

Specifies the sense of isolation. It can be:

- `#LOW#`
- `#HIGH#`

- `ISO_CLAMP`

Specifies the clamp value of isolation. It can be:

- `1`
- `0`
- `Z`
- `latch`

- `ISO_PLACE`

Specifies place where isolation is targeted. It can be:

- `source`
- `sink`

- `ISO_CELL`

Specifies the isolation cell.

- `PORTS`

Specifies the port constraint.

- `PD`

Specifies the target domain where different isolation location is required. This option is used with *DIFF_ISO_LOCATION*.

- `DIFF_ISO_LOCATION`

Specifies the location of target PD. Usually specified for the top power domain as `#SELF#`. It can be:

- `#SELF#`
- `#PARENT#`

- `SCOPE`

Specifies the target scope name for generating hierarchical `iso_config` file for the `-hier_isogen` flow.

- **UPF**
Specifies the UPF file name of target scope for generating hierarchical iso_config file for the `-hier_isogen` flow.
- **ISO_CONFIG**
Specifies the partition iso_config file of target scope for generating hierarchical iso_config file for the `-hier_isogen` flow.
- **ISO_CONSTRAINT**
Specifies the partition isolation constraint file of target scope for generating hierarchical iso_config file for the `-hier_isogen` flow.

Example1: ISOGEN Config

SRC_PD	SINK_PD	ISO_SIGNAL	ISO_SUPPLY	ISO_LOAD_CATEGORION	ISO_SENSE	ISO_CLAMP	ISO_PLACEMENT	ISO_CELLS	PORTS
PD_B1	*	iso_en1	SS_LOGIC	#SELF#	#LOW#	0	source	Cells_0	
PD_B1	*	iso_en2	SS_LOGIC	#SELF#	#LOW#	1	source	Cells_1	*_rdss
PD_B1	*	iso_en3	SS_LOGIC	#SELF#	#LOW#	1	source	Cells_1	*_pass

Usage

```
ugo_generate -u test.upf -iso_config iso_config.json -iso_info isolation.info
ugo_optimize_power_intent -isogen -iso_config iso_config.json
```

Scope Condition

This section describes the syntax of scope condition.

Syntax:

```
[Scope Condition]
SCOPE, CONDITION
```

Description

This section impacts Load UPF, Connection, and UGO_SET_SCOPE. The CONDITION is generated to guard the generated `load_upf`, `connect_supply_net/connect_logic_net`, and `set_scope` commands.

- SCOPE

Specifies the scope.

- CONDITION

Specifies the condition expression. Following are the four special keywords that can be specified in the expression string for referring SCOPE:

- @SCOPE@: It is replaced as capital and simple hierarchy of the SCOPE string.
- @FSCOPE@: It is replaced as capital and full hierarchy of the SCOPE string.
- @LSCOPE@: It is replaced as lowercase and simple hierarchy of the SCOPE string.
- @LFSCOPE@: It is replaced as lowercase and full hierarchy of the SCOPE string.

Example1: Scope Condition

[Scope Condition]	
SCOPE	CONDITION
cpu	[info exists LOAD_CPU] && \$LOAD_CPU=="1"
parsmc_wrap/ppu	[info exists LOAD_@SCOPE@] && [info exists LOAD_@FSCOPE@]
parpsf_wrap/ccu	[info exists LOAD_@LSCOPE@] && [info exists LOAD_@LFSCOPE@]

[Load UPF]	
UPF	SCOPE
cpu.upf	cpu
ppu.upf	parsmc_wrap/ppu
ccu.upf	parpsf_wrap/ccu

[Connection]	

SCOPE	HIGH_CONN	LOW_CONN
cpu	VDD	VDD
parsmc_wrap/ppu	VDD	VDD
parpsf_wrap/ccu	VDD	VDD

Generated UPF

```

if { [info exists LOAD_CPU] && $LOAD_CPU=="1" } {
    load_upf cpu.upf -scope cpu
}
if { [info exists LOAD_PPU] && [info exists LOAD_PARSMC_WRAP_PPU] } {
    load_upf ppu.upf -scope parsmc_wrap/ppu
}
if { [info exists LOAD_ccu] && [info exists LOAD_parpsf_wrap_ccu] } {
    load_upf ccu.upf -scope parpsf_wrap/ccu
}
if { [info exists LOAD_CPU] && $LOAD_CPU=="1" } {
    connect_supply_net VDD -ports cpu/VDD
}
if { [info exists LOAD_PPU] && [info exists LOAD_PARSMC_WRAP_PPU] } {
    connect_supply_net VDD -ports parsmc_wrap/ppu/VDD
}
if { [info exists LOAD_ccu] && [info exists LOAD_parpsf_wrap_ccu] } {
    connect_supply_net VDD -ports parpsf_wrap/ccu/VDD
}

```

Special Syntax

This section describes the following topics:

- [The if-else Structure Support](#)
- [SET_SCOPE Support](#)
- [Foreach support](#)
- [For Loop Statement Support](#)

The if-else Structure Support

UPFA supports if-else structure in the power intent csv file using the following keywords. This support allows you to generate if-else block UPF code.

- UGO_CONDITION_IF(<condition>)
- UGO_CONDITION_ELIF(<condition>)

- UGO_CONDITION_ELSE
- UGO_CONDITION_END

Example

```
UGO_CONDITION_IF( ${TOOL} == VCS )
[Power Domain]
NAME, ELEMENTS
PD_TOP_IF, #DEFAULT#
UGO_CONDITIONELIF( ${TOOL} == VCF )
[Power Domain]
NAME, ELEMENTS
PD_TOP_ELIF, #DEFAULT#
UGO_CONDITION_ELSE
[Power Domain]
NAME, ELEMENTS
PD_TOP_ELSE, #DEFAULT#
UGO_CONDITION_END
```

Generated UPF

```
if { ${TOOL} == VCS } {
    # [Power Domain]
    create_power_domain PD_TOP_IF -elements { . }
} elseif { ${TOOL} == VCF } {
    # [Power Domain]
    create_power_domain PD_TOP_ELIF -elements { . }
} else {
    # [Power Domain]
    create_power_domain PD_TOP_ELSE -elements { . }
}
```

Detailed Usage

- The `<condition>` inside `UGO_CONDITION_IF` and `UGO_CONDITION_ELIF` with surrounding with `()` would be the condition string and is generated into UPF condition with the string.
- All the keywords should be specified with an empty line to identify the block is begin.
- You must specify the `UGO_CONDITION_IF` keyword to identify if the condition is the begin block.
- You must specify the `UGO_CONDITION_END` keyword to identify the condition block is end.
- It is an error if there is `UGO_CONDITION_ELIF`, `UGO_CONDITION_ELSE`, `UGO_CONDITION_END`, but no previous `UGO_CONDITION_IF`.
- It is an error if there is previous `UGO_CONDITION_IF`, `UGO_CONDITION_ELIF`, `UGO_CONDITION_ELSE`, but no `UGO_CONDITION_END`.

- All sections in the CSV condition block are generated inside the UPF condition block.
 - The sections surrounding with `UGO_CONDITION_IF` and front of `UGO_CONDITION_ELIF/UGO_CONDITION_ELSE/UGO_CONDITION_END` are generated inside the “if” block in the UPF code.
 - The sections surrounding with `UGO_CONDITION_ELIF` and front of `UGO_CONDITION_ELIF/UGO_CONDITION_ELSE/UGO_CONDITION_END` are generated inside the “elseif” block in the UPF code.
 - The sections surrounding with `UGO_CONDITION_ELSE` and front of `UGO_CONDITION_END` are generated inside “else” block in the UPF code.
- Nested if-else block is supported.

Limitation

- All the sections inside the condition blocks are not checked for the power intent. That is, UPFA does not report power intent error if the section is inside the condition blocks.

SET_SCOPE Support

UPFA supports the following:

- The `set_scope` syntax in the power intent csv file using the following keywords:

```
UGO_SET_SCOPE(<scope name>)  
UGO_SET_SCOPE_END
```

- To generate a sub scope block UPF code.

This support allows you to generate the if-else block UPF code.

Example

```
[Power Domain]  
NAME,ELEMENTS,primary  
PD_TOP,#DEFAULT#,SS1  
[Supply]  
NAME  
SS1  
[Logic Network]  
NAME, DIRECTION  
TOP_LNET,#INTERNAL#  
[Load UPF]  
UPF,SCOPE  
a1.upf,a1  
// punch logic port  
UGO_SET_SCOPE (a1)  
[Logic Network]  
NAME,DIRECTION
```

```
A1_LNET,#IN#
UGO_SET_SCOPE_END
[Connection]
SCOPE,HIGH_CONN,LOW_CONN
a1,TOP_LNET, A1_LNET
```

Generated UPF

```
# [Load UPF]
load_upf a1.upf -scope a1
set_scope a1
# [CREATE_LOGIC_PORT]
create_logic_port A1_LNET
# [CREATE_LOGIC_NET]
create_logic_net A1_LNET
# [CONNECT_LOGIC_NET]
connect_logic_net A1_LNET -ports "A1_LNET"
set_scope ..
# [Supply Set]
create_supply_set SS1
# [Power Domain]
create_power_domain PD_TOP -elements { . } -supply { primary
SS1 }
# [Supply Set]
create_supply_net VSS
create_supply_net VDD_nwell_of_SS1
create_supply_net VDD_pwell_of_SS1
create_supply_set SS1 -function { ground VSS } -function {
nwell VDD_nwell_of_SS1 } -function { pwell VDD_pwell_of_SS1 }
-update
# [Logic Network]
create_logic_net TOP_LNET
# [Connection]
connect_logic_net TOP_LNET -ports a1/A1_LNET
```

Where, <scope name> could be a list (for example, UGO_SET_SCOPE(m_cpu1 m_cpu2 m_cpu3)) or a Tcl variable.

Foreach support

UPFA supports the `foreach` syntax in the power intent CSV file using the following keyword:

```
UGO_FOREACH(<value1>, <value2>)
UGO_FOREACH_END
```

This support allows you to generate the following UPF code.

Example

```
foreach <value1> <value2> {
}
```

Example Usage CSV File

```
UGO_FOREACH (port, [find_objects . -pattern xxport* -object_type port])
[Supply]
NAME, power, ground
SS_VDD_${port}, VDD_${port}, VSS
UGO_FOREACH_END

UGO_FOREACH (port, $anibVDDQPorts)
[Variable]
NAME, VALUE
port_doc, ${port}_doc

[Attribute]
PORTS, DRIVER_SUPPLY, RECEIVER_SUPPLY
${port_doc}, SS_VDD_${port}, VSS
UGO_FOREACH_END
```

Generated UPF

```
foreach port [find_objects . -pattern xxport* -object_type port] {
  # [CREATE_SUPPLY_NET]
  create_supply_net VSS
  create_supply_net VDD_${port}

  # [CREATE_SUPPLY_SET:NO_FUNC]
  create_supply_set SS_VDD_${port}

  # [CREATE_SUPPLY_SET:FUNC_SUPPLY_NET]
  create_supply_set SS_VDD_${port} -function { ground VSS } -function
  "power VDD_${port}" -update
}

foreach port $anibVDDQPorts {
  # [VARIABLE]
  set port_doc "${port}_doc"
  # [SET_PORT_ATTRIBUTES]
  set_port_attributes \
    -ports ${port_doc} \
    -driver_supply SS_VDD_${port} \
    -receiver_supply VSS
}
```

For Loop Statement Support

The following is the structure of the `for` loop in the power intent CSV file for generating the `for` loop in the UPF code:

```
UGO_FORLOOP(<start>,<test>,<next>)
UGO_FORLOOP_END
```

Example Usage CSV File

```
UGO_FORLOOP(${set x 0}, {$x<10}, {incr x})  
[Supply]  
NAME, DIRECTION, RESOLVE, ground, nwell  
SS1, #INTERNAL#, #ONE_HOT#, VSS, VDD  
  
UGO_FORLOOP(,,)  
UGO_FORLOOP_END  
  
UGO_FORLOOP_END
```

Generated UPF

```
for {set x 0} {$x<10} {incr x} {  
  for {} {} {} {}  
  # [CREATE_SUPPLY_NET]  
  create_supply_net VSS -resolve one_hot  
  create_supply_net VDD -resolve one_hot  
  # [CREATE_SUPPLY_SET:NO_FUNC]  
  create_supply_set SS1  
  # [CREATE_SUPPLY_SET:FUNC_SUPPLY_NET]  
  create_supply_set SS1 -function { ground VSS } -function {nwell VDD}  
  -update  
}
```

Detail Usage

1. You must specify the two keywords and an empty line to identify the block.
2. You must use `UGO_FORLOOP` to identify the opening of a `for` loop.
3. The `UGO_FORLOOP` keyword requires three arguments: `start`, `test`, and `next`.
4. If an argument is empty, the generated argument in the `for` loop is "{}".
5. If an argument is not empty, it should be enclosed with curly braces "{}".
6. You must use the `UGO_FORLOOP_END` keyword to identify the end of a `for` loop.
7. Nested for loop is supported.

Limitation

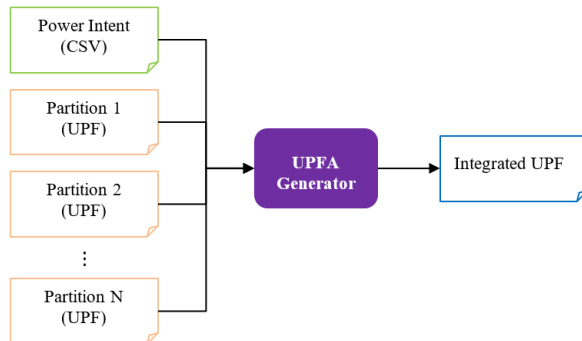
- All the sections inside the `for` loops are not checked for the power intent. That is, power intent error is not reported if the section is inside a `for` loop.

Partition UPF Integration

UPFA supports partition UPF integration [PIntegration] in CSV format to describe integration work. UPFA analyzes the partition UPF file to get information, generate network

and connection automatically. UPFA generator takes CSV file and partition UPF as input and output of the top level integrated UPF file.

Figure 1 *Partition UF Integration*



Syntax:

```
[PIntegration]
SCOPE, UPF [, LOW_CONN_INTEG, CONDITION, IS_MACRO]
[PROPAGATE_SS, PROPAGATE_PSTATE, INTERNAL_SUPPLY, COMMON_PD,
COMMON_POWER, COMMON_GROUND, COMMON_NWELL, COMMON_PWELL]
```

Description:

- SCOPE
Specifies the scope to load the UPF file.
- UPF
UPF file to be loaded.
- LOW_CONN_INTEG (Optional)
The value can be:
 - #YES#: Automatically connect top-level supply net logic net with block UPF supply port/logic port. The default value is #YES#.
 - #NO#: No connection.
- CONDITION (Optional)
Expression specified for integrating partition UPF. It can be:
 - #IF#{<condition>}
 - #ELIF#{<condition>}
 - #ELSE#

- **IS_MACRO** (Optional)

Specifies macro or not. It can be:

- #YES#
- #NO#

The default value is #NO#. By default, the connection of sub blocks is also generated inside the condition. If this value is #YES#, then the connection is generated outside the condition.

- **PROPAGATE_SS** (Optional)

Creates a supply set or supply net. This option takes effect only when **LOW_CONN_INTEG** is #YES#.

Creates a supply set with same supply functions in block UPF. In block UPF, if the supply function is not connected to the supply port, this supply net cannot be propagated and is discarded. If all supply functions are not connected to the supply port, this supply set is not propagated and discarded.

It can be:

- #YES#
- #NO#

The default value is #YES#

- **PROPAGATE_PSTATE**

Specifies power state and propagation in SoC UPF. This option takes effect only when **LOW_CONN_INTEG** and **PROPAGATE_SS** are #YES#.

In the following cases, the state is discarded:

- Supply set state with logic expression is not supported.
- Existing objects (supply set) cannot be propagated.
- The power state of power domain is discarded.

In the power state group and the power state table if the state's objects include power domain, supply net, supply port, logic port, and logic net, the state is discarded. The power domain cannot be propagated. The power state table does not support supply net and supply port. The power state group does not support logic port and logic net.

It can be:

- #YES#
- #NO#

The default value is #YES#

- INTERNAL_SUPPLY (Optional)

A supply that does not create a supply port at top level is considered an internal supply.

- COMMON_PD

Specify the name to create top-level power domain.

#EMPTY#: Specified when top-level power domain creation is not required.

Default is PD_TOP

Creates a supply set named `SS_<COMMON_PD>` as the primary supply set of `COMMON_PD`.

- COMMON_POWER (Optional)

Specify common power supply net in the following format:

```
<direction>{<resolve>{<Port/Net Name>}}
```

Following is the format for all possible cases:

```
#IN#|#OUT#|#INOUT#{#PARALLEL#|#ONE_HOT#|#PARALLEL_ONE_HOT#{<Port/Net  
Name>}}
```

If you specify a direction attribute, UPFA creates a supply port and connects it to the supply net with the same name. Otherwise, it creates a supply net only.

The default value is the most common power supply net for all partitions. UPFA automatically identifies the default value. If automatic identification fails, the default value is VDD.

This option is the power function of the primary supply set (`SS_<COMMON_PD>`).

- COMMON_GROUND (Optional)

Specify common ground supply net in the following format:

```
<direction>{<resolve>{<Port/Net Name>}}
```

Following is the format for all possible cases:

```
#IN#|#OUT#|#INOUT#{#PARALLEL#|#ONE_HOT#|#PARALLEL_ONE_HOT#{<Port/Net  
Name>}}
```

If you specify a direction attribute, UPFA creates a supply port and connects it to the supply net with the same name. Otherwise, it creates a supply net only.

The default value is the most common ground supply net for all partitions. UPFA automatically identifies the default value. If automatic identification fails, the default value is `VSS`.

This option is the ground function of the primary supply set (`SS_<COMMON_PD>`).

- `COMMON_NWELL` (Optional)

Specify common nwell supply net in the following format:

```
<direction>{<resolve>{<Port/Net Name>}}
```

Following is the format for all possible cases:

```
#IN#|#OUT#|#INOUT#{#PARALLEL#|#ONE_HOT#|#PARALLEL_ONE_HOT#{<Port/Net  
Name>}}
```

If you specify a direction attribute, UPFA creates a supply port and connects it to the supply net with the same name. Otherwise, it creates the supply net only.

The default value is the most common nwell net for all partitions. UPFA automatically identifies the default value.

This option is the nwell function of the primary supply set (`SS_<COMMON_PD>`).

- `COMMON_PWELL` (Optional)

Specify common pwell supply net in the following format:

```
<direction>{<resolve>{<Port/Net Name>}}
```

Following is the format for all possible cases:

```
#IN#|#OUT#|#INOUT#{#PARALLEL#|#ONE_HOT#|#PARALLEL_ONE_HOT#{<Port/Net  
Name>}}
```

If you specify a direction attribute, UPFA creates a supply port and connects it to the supply net with the same name. Otherwise, it creates the supply net only.

The default value is the most common pwell net for all partitions. UPFA automatically identifies the default value.

This option is the pwell function of the primary supply set (`SS_<COMMON_PD>`).

Key points to Note:

If common power, ground, nwell, or pwell is not defined, UPFA performs automatic identification as follows:

1. Initially, if a supply set specified in the block UPF can be propagated, the priority is to get the most frequent power, ground, nwell, or pwell supply functions as COMMON_POWER/COMMON_GROUND/COMMON_NWELL/COMMON_PWELL.

If the identification of COMMON_POWER is a success and COMMON_NWELL is a fail, then COMMON_POWER is assigned to COMMON_NWELL.

If the identification of COMMON_GROUND is a success and COMMON_PWELL is a fail, then COMMON_GROUND is assigned to COMMON_PWELL.

2. For inappropriate supply set or failed identification in the first step, UPFA identifies the power and ground from the supply net. It treats the supply net with a name starting with VSS (case insensitive, Vss, vss, and so on) as ground supply net. Others are treated as power supply net. The most frequent ground supply net is treated as COMMON_GROUND and the most frequent power supply net is treated as COMMON_POWER.
3. If there is no inappropriate supply net or failed identification in the second step, UPFA sets COMMON_POWER and COMMON_GROUND with the default value.

Example

Input:

Partition UPF: Partition1.upf, Partition2.upf, Partition3.upf, and *MARCO.upf*

Prerequisite: The integrated UPF files should pass the VCLP check. If dependency variables exist inside these UPF files, a complete definition of the variables at top level is required.

Input:

Power intent: PartitionIntegrate.csv

Example:

```
[PIntegration]
SCOPE , UPF , LOW_CONN_INTEG,          CONDITION ,
IS_MACRO
m_cpu1 , Partition1.upf                ,          , #IF#[[info
exists ::PAR1]]
m_cpu2 , Partition2.upf                ,          ,
#ELIF#[[info exists ::PAR2]]
```

```
m_cpu3 , Partition3.upf , #NO#
#ELSE#
m_macro*, MACRO.upf , #YES#
```

Output:

Top level UPF: Top.upf

Example:

```
# [Design Attribute]
set_design_attributes -elements { . } -attribute { enable_bias TRUE }
set_design_attributes -elements { . } -attribute
{ enable_state_propagation_in_add_power_state TRUE }
set_design_attributes -elements { . } -attribute { legacy_block FALSE }
set_design_attributes -elements { . } -attribute { lower_domain_boundary
TRUE }
set_design_attributes -elements { . } -attribute { user_define_attr aaa }

# [Load UPF]
load_upf MACRO.upf -scope m_macro*

if { [info exists ::PAR1] } {
    # [LOAD_UPF]
    load_upf Partition1.upf -scope m_cpu1
} elseif { [info exists ::PAR2] } {
    # [LOAD_UPF]
    load_upf Partition2.upf -scope m_cpu2
} else {
    # [LOAD_UPF]
    load_upf Partition3.upf -scope m_cpu3
}

# [Supply Set]
create_supply_set SS_VDD
create_supply_set SS_VDDY
create_supply_set SS_VCC1
create_supply_set SS_VCC3
```

5

UPFA Power Intent Editor GUI

The CSV format file allows you to describe power intent and generate UPF file automatically. The UPFA Power Intent Editor provides a GUI to view and edit the CSV format file.

The following sections describe this feature in detail:

- [Viewing the UPFA Power Intent Editor GUI](#)
- [File View](#)
- [Viewing Power Intent of the Generated UPF File](#)
- [Detail View](#)
- [Console](#)
- [Support for Special Rules in CSV Format](#)
- [Menu Options](#)
- [Tcl Options](#)

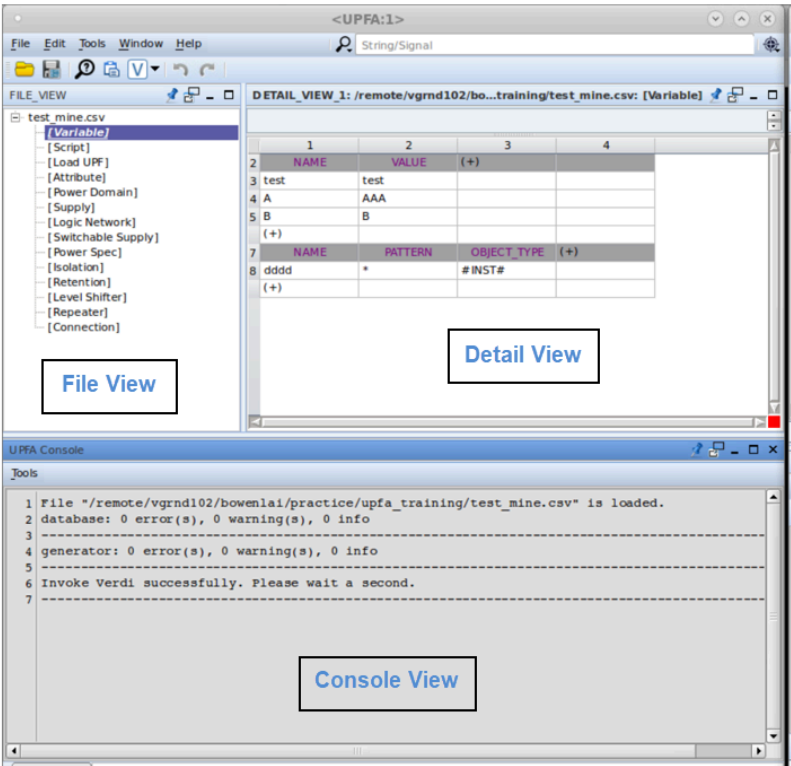
Viewing the UPFA Power Intent Editor GUI

To view the UPFA Power Intent Editor GUI, you can perform either of the following:

From command line: Use the `upfa -gui` command

From ugo command prompt: Use the `upfa` command to go to the `ugo` shell and then use the `ugo_gui` command.

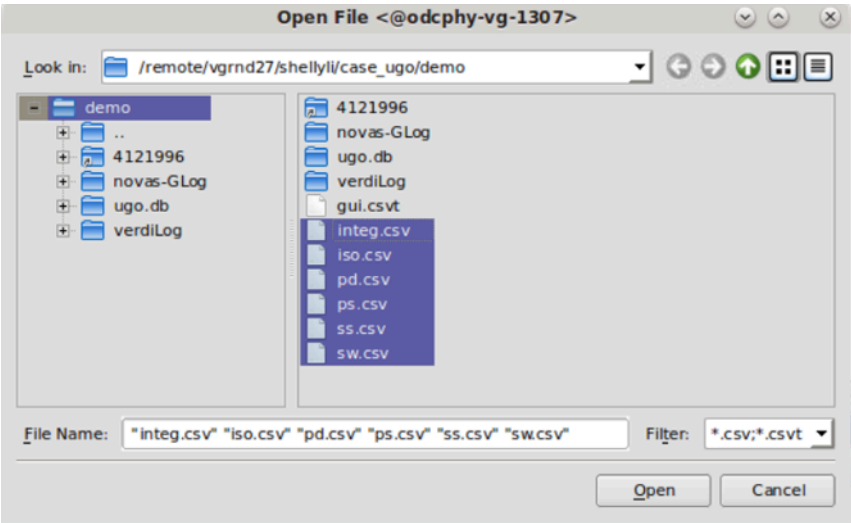
Figure 2 UPFA Power Intent Editor GUI



To open the CSV files in the UPFA Power Intent Editor GUI, perform the following:

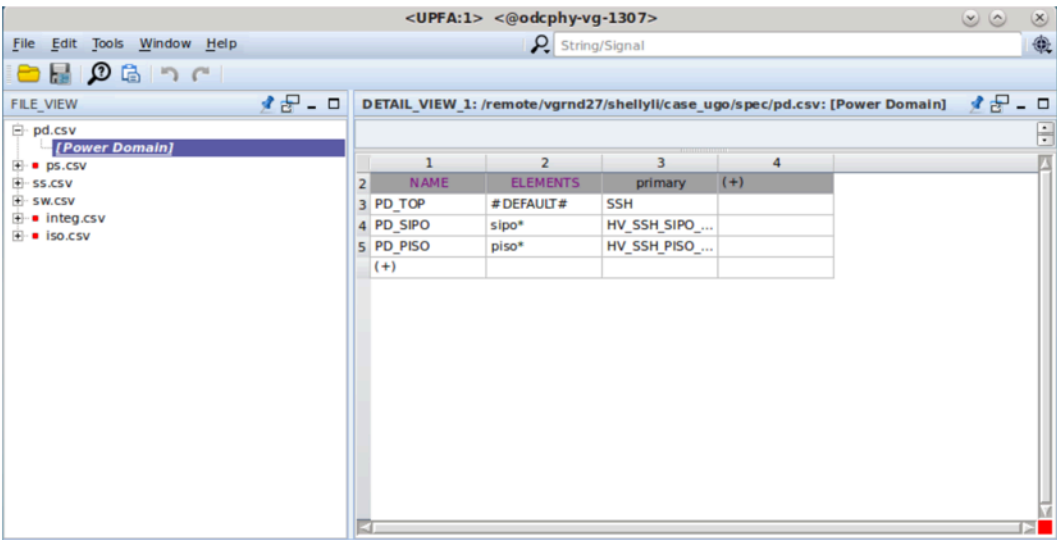
1. Select **File -> Open File**.
2. In the *Open File* dialog box, select a single or multiple CSV files and click **Open** as shown in the following figure:

Figure 3 *Open File Dialog Box*



The CSV file is loaded in the UPFA Power Intent Editor GUI as shown in the following figure:

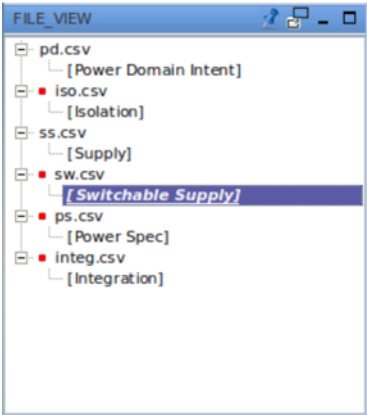
Figure 4 *Opening CSV Files*



File View

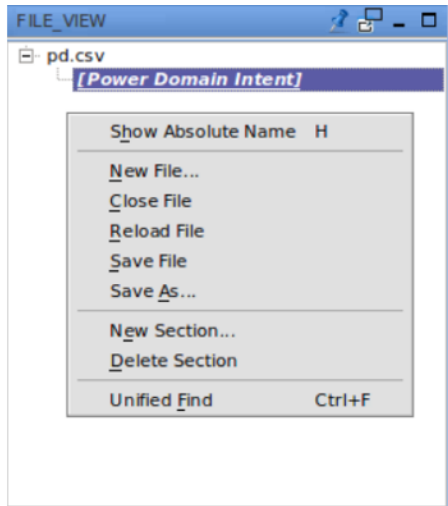
The *File View* displays the CSV files as shown in the following figure:

Figure 5 *File View*



The following right-click options are supported for the selected file:

Figure 6 *File View Right-Click Options*

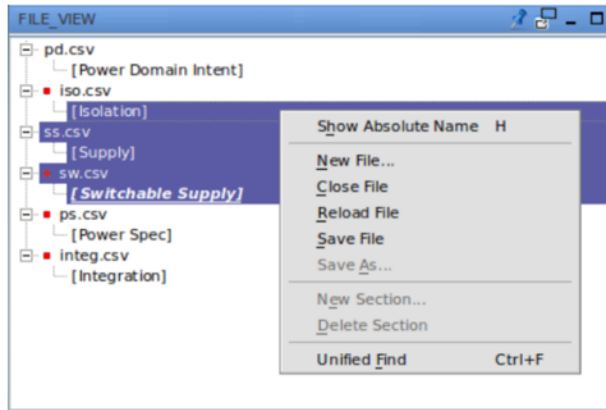


Right-Click Option	Description
Show Absolute Name	Displays the absolute path of all the CSV files in the view.
New File	Allows you to create a new CSV file.

Right-Click Option	Description
Close File	Closes the selected CSV file.
Reload File	Reloads the selected CSV file.
Save File	Saves the selected CSV file.
Save As	Allows you to save and open the selected CSV file with a different name.
New Section	Allows you to create a new section for a selected CSV file.
Delete Section	Allows you to delete the selected section.
Unified Find	Refreshes unified find interface. You can use <i>Ctrl+F</i> to copy selected item's text in File View to the Unified Find interface as search string.

You can select multiple sections and files. The *New File*, *Close File*, *Reload File*, and *Save File* right-click options are supported for the selected files or sections.

Figure 7 Selecting Multiple Files



Using the New File Right-Click Option

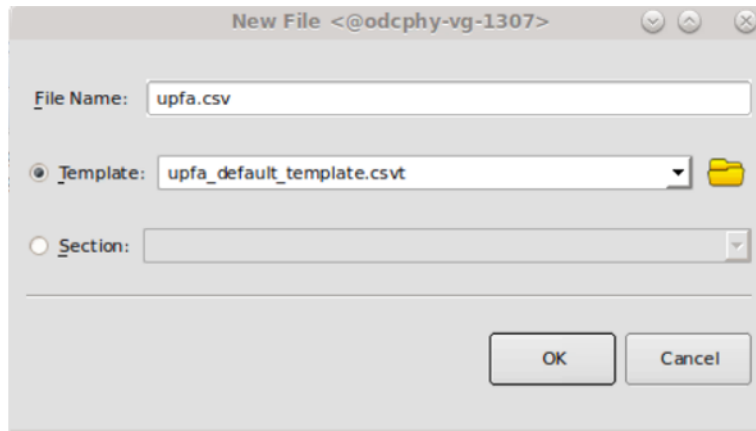
The *New File* option allows you to create a new CSV file using the following two ways:

- Creating a New File by Template

In the **New File** dialog box, specify file name in the *File Name* field. Select **Template** option and specify the template name in the Template field. The default template is

`upfa_default_template.csvt`. This file contains all sections, you can change it to others. Click **OK** button as shown in the following figure:

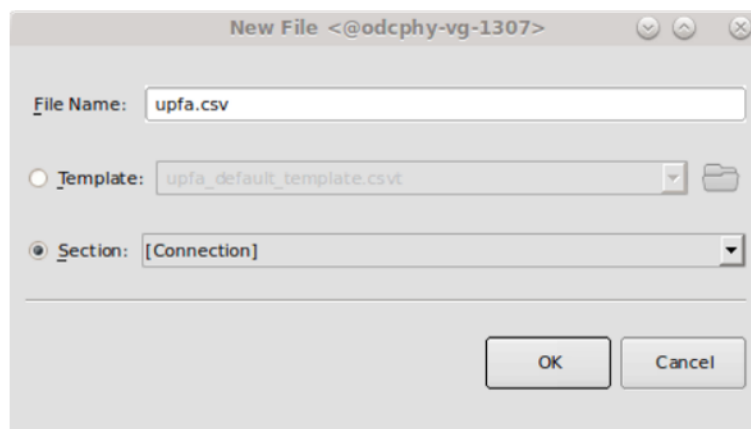
Figure 8 *Creating a New File by Template*



- Creating a New File With Specific Section

In the **New File** dialog box, specify file name in the *File Name* field. Select **Section** option and select a section name from the drop-down as shown in the following figure. Click **OK** button.

Figure 9 *Creating a New File With Specific Section*

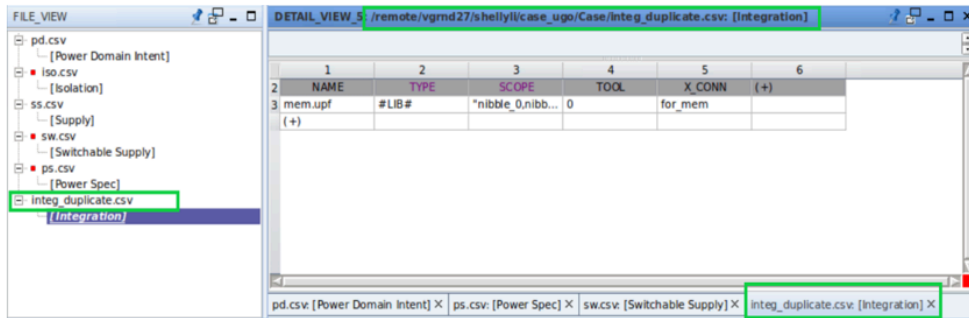


Using Save As Right-Click Option

The **Save As** dialog box allows you to save and open the selected CSV file with a different name. This option changes the currently edited file to a new file. For example, when

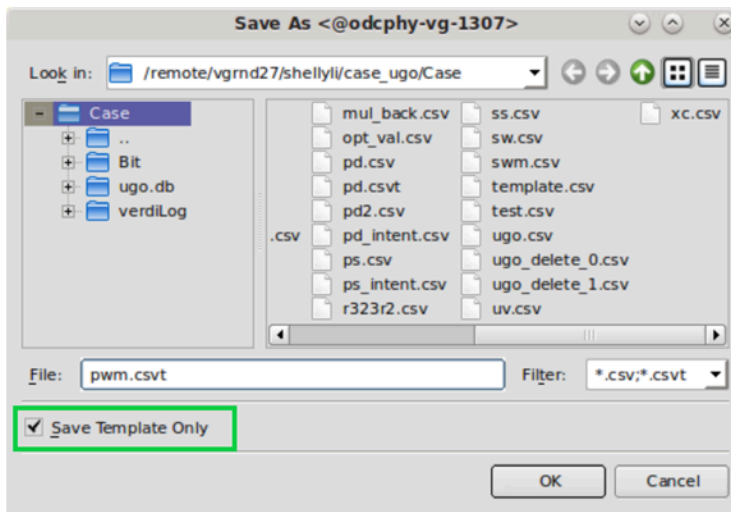
you save `integ.csv` as `integ_duplicate.csv`, then the *File View* and *Detail View* are updated to new file (`integ_duplicate.csv`) as shown in the following figure:

Figure 10 Using Save As Option



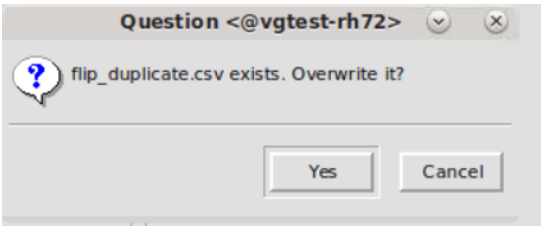
You can delete unused attributes and sections from the table and save the file as template file (`.csvt`). When the **Save Template Only** option is selected in the *Save As* dialog box as shown in the following figure, UPFA saves all sections to the new file except user-defined content. Also, the GUI does not change the current file to template file.

Figure 11 Save As Dialog Box



If the new file already exists, a confirmation dialog box is displayed as shown in the following figure. You can click **Yes** to overwrite the existing file and click **Cancel** to go back to the *Save As* dialog box.

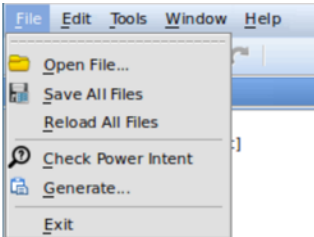
Figure 12 Confirmation Dialog Box



Using Reload File and Reload All Files Options

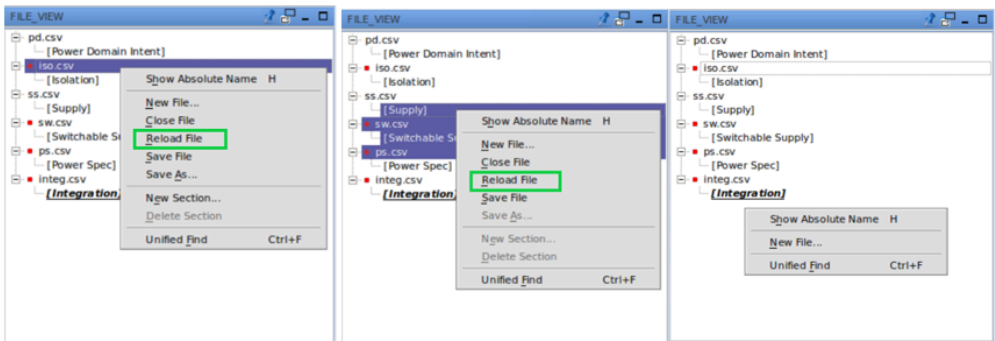
The **Reload File** right-click option allows you to reload the selected files. The **File -> Reload All Files** menu option can reload all opened files one time.

Figure 13 Reload All Files Option



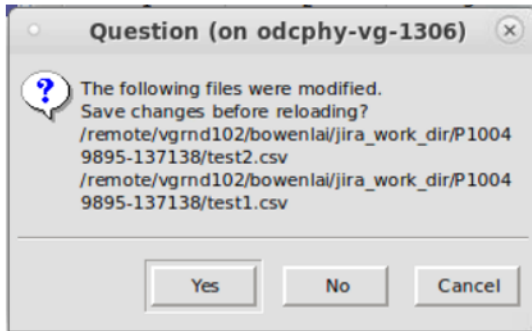
The **Reload File** right-click option is enabled only when you select one or more files (including selecting the sections of the files). Otherwise, it is hidden.

Figure 14 Reload File Option



When reloading a modified CSV file, a confirmation dialog box is displayed as shown in the following figure:

Figure 15 Confirmation Dialog for Reloading Modified Files



The confirmation dialog shows message with following format:

```
Please confirm reload following unsaved files:
[absolute file name1]
[absolute file name2]
...
```

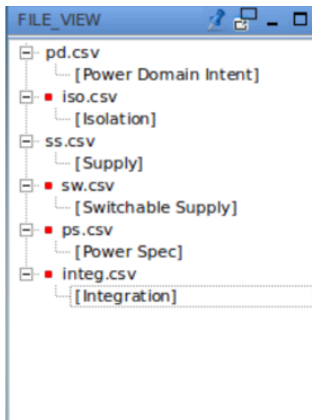
If the number of files is more than 5, the text uses “...” to replace.

You can click **Yes** to save the modified content before reloading, then reload the CSV file from the disk. You can click **Cancel** or **X** (close dialog) to stop reloading the selected files. Reloading CSV file does not support undo/redo.

Highlighting CSV Files With Errors in File View

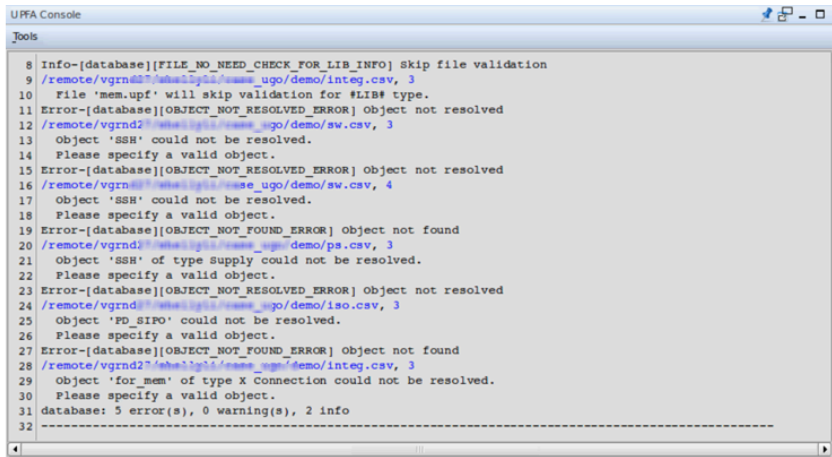
The file with errors is displayed with a red dot before the file name in the *File View* and the *Console View* displays the corresponding error message as shown in the following figures:

Figure 16 Highlighting CSV Files With Errors in File View



As shown in the above figure four files (*iso.csv/sw.csv/ps.csv/integ.csv*) have errors, so a red dot is displayed for these file names. The **Reload All Files** and **Reload File** options check power intent again.

Figure 17 Error Message Log in the Console View

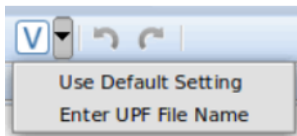


The Save As option clears the error information in the *Console View*.

Viewing Power Intent of the Generated UPF File

You can use the **Show UPF in Verdi** drop-down button  on the toolbar or the **File -> Show UPF in Verdi** menu option to view the power intent of the generated UPF file. This feature allows you to analyze and debug the power intent of the design using Verdi GUI. You can click the drop-down arrow to view the sub-menu list with the **Use Default Setting** and **Enter UPF File Name** options as shown in the following figure:

Figure 18 Power Intent Sub-Menu Options

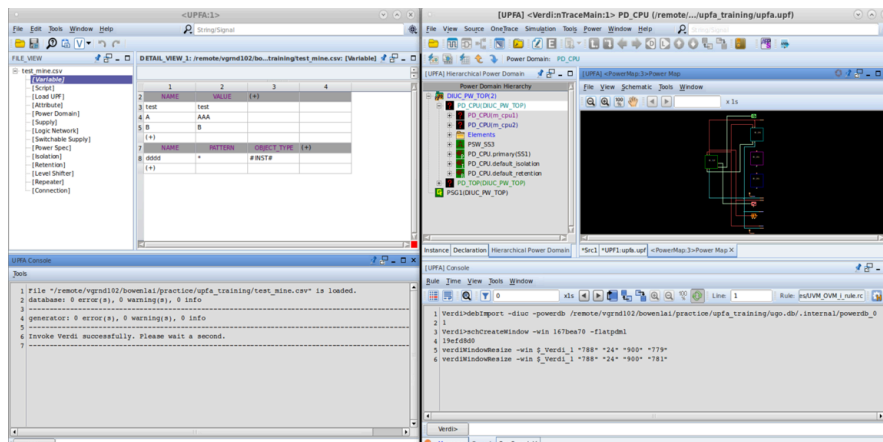


When you import the UPF files into the UPFA editor and click the **Power Intent** button or **Use Default Setting** option, UPFA automatically executes the Check Power Intent and Generate actions. The Generate process uses the *.tmp_for_show.upf* file as the default UPF file, the *.tmp_for_show.refine.upf* file as the default ARP refine file, and 3.0 as

Chapter 5: UPFA Power Intent Editor GUI Viewing Power Intent of the Generated UPF File

the default UPF version. If the Generate process is successful, the UPFA editor invokes Verdi and show the power intent of the generated UPF file as shown in the following figure:

Figure 19 Viewing the Power Intent of the Generated UPF File

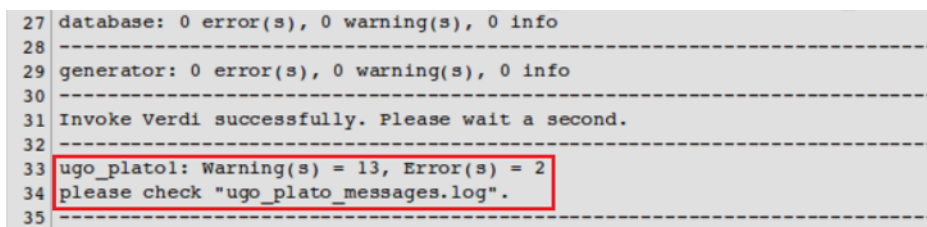


If you have already generated UPF file with your own settings, UPFA uses the latest settings to generate the UPF file.

UPFA does not invoke Verdi if no CSV file is imported.

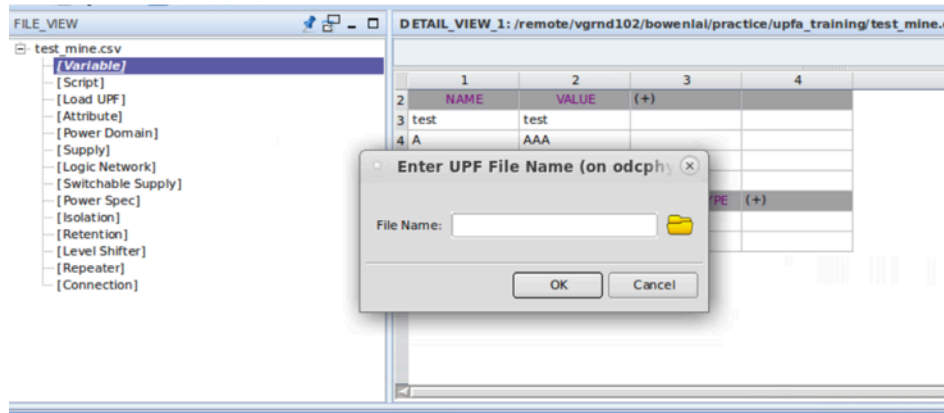
Even if the UPF file generation is successful, some errors might still be detected by the `ugo_plato1` process. If the error count is more than zero, the console shows the warning and error count and informs you to check the log file as shown in the following figure:

Figure 20 Viewing Warning and Error Count



When you click the **Enter UPF File Name** option, Verdi displays the **Enter UPF File Name** dialog as shown in the following figure:

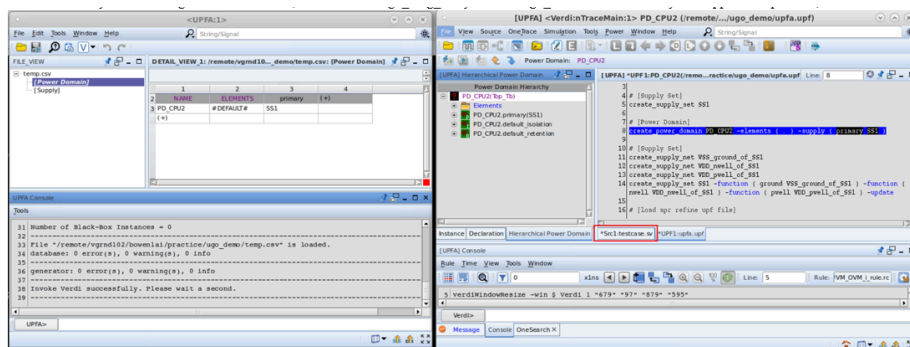
Figure 21 Viewing Warning and Error Count



This dialog allows you to enter the UPF file name you want to show in Verdi. You can enter the absolute or relative path of the file and click **OK** or press **Enter** key. If the file exists, UPFA invokes Verdi and shows the power intent of the specified UPF file. If the file does not exist, UPFA issues the File does not exist, check message in the Console.

If an RTL design is already loaded in UPFA (using commands `ugo_vlog_analyze` and `ugo_elaborate`), Verdi displays the power intent together with the RTL design as shown in the following figure:

Figure 22 Viewing Power Intent Along With RTL Design



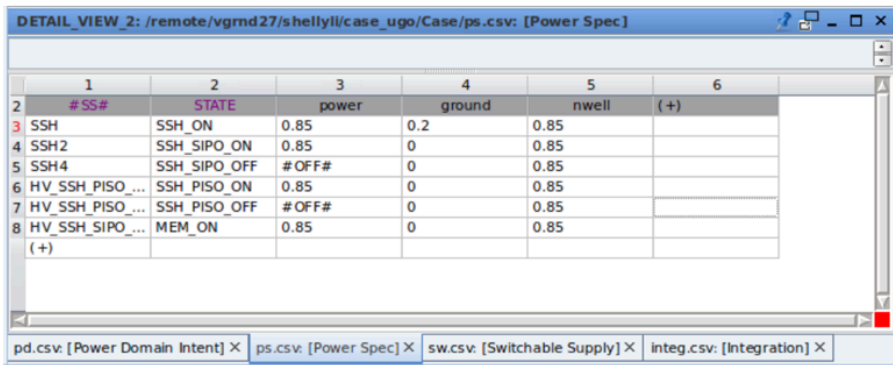
If you have already invoked Verdi and clicked the **Show UPF in Verdi** button again, Verdi reloads the latest power intent of the generated UPF file and displays the Reload power design successfully message in the Console.

When you close the UPFA editor, Verdi invoked by UPFA is also closed.

Detail View

The *Detail View* displays the information of the selected section in tabular format as shown in the following figure:

Figure 23 *Detail View*



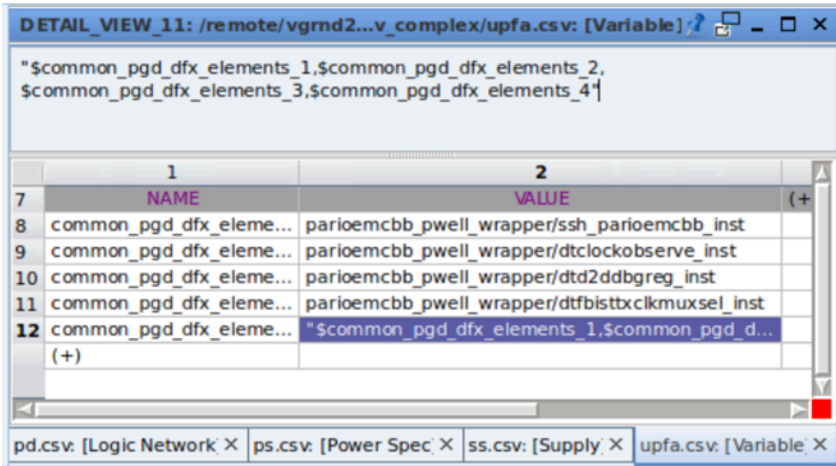
	1	2	3	4	5	6
	#SS#	STATE	power	ground	nwell	(+)
2						
3	SSH	SSH_ON	0.85	0.2	0.85	
4	SSH2	SSH_SIPO_ON	0.85	0	0.85	
5	SSH4	SSH_SIPO_OFF	#OFF#	0	0.85	
6	HV_SSH_PISO_...	SSH_PISO_ON	0.85	0	0.85	
7	HV_SSH_PISO_...	SSH_PISO_OFF	#OFF#	0	0.85	
8	HV_SSH_SIPO_...	MEM_ON	0.85	0	0.85	
	(+)					

You can double-click a section in the *File View* to view its information in the *Detail View*. The *Detail View* is file-based and supports editing multiple files with multiple tabs.

Editing String Input

You can use the *Text Editing Field* at the top of the Detail View to edit long string input. This field shows the text of the item selected in the table, as shown in the following figure, and supports read and write.

Figure 24 *Text Editing Field*



	1	2
	NAME	VALUE
7		
8	common_pgd_dfx_eleme...	parioemcbb_pwell_wrapper/ssh_parioemcbb_inst
9	common_pgd_dfx_eleme...	parioemcbb_pwell_wrapper/dtclockobserve_inst
10	common_pgd_dfx_eleme...	parioemcbb_pwell_wrapper/dtd2ddbgreg_inst
11	common_pgd_dfx_eleme...	parioemcbb_pwell_wrapper/dtbfisttxclkmuxsel_inst
12	common_pgd_dfx_eleme...	"\$common_pgd_dfx_elements_1,\$common_pgd_d...
	(+)	

Keywords are read-only and does not support customization. In the following figures, editing elements is supported for the NAME column in the Logic Network section. Editing

the keyword elements in the DIRECTION column is not supported. You can select a keyword element from the drop-down.

Figure 25 *Editing Name*

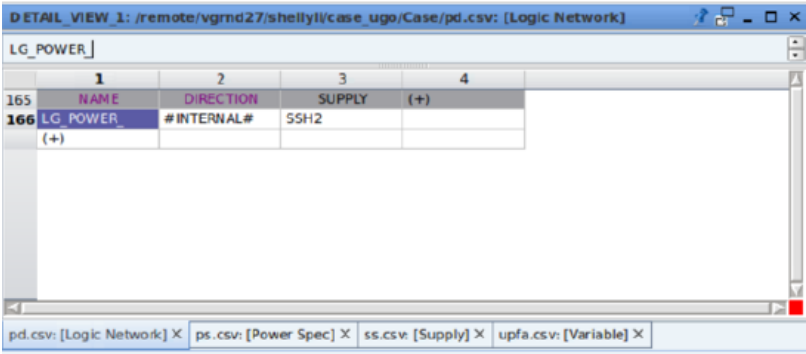
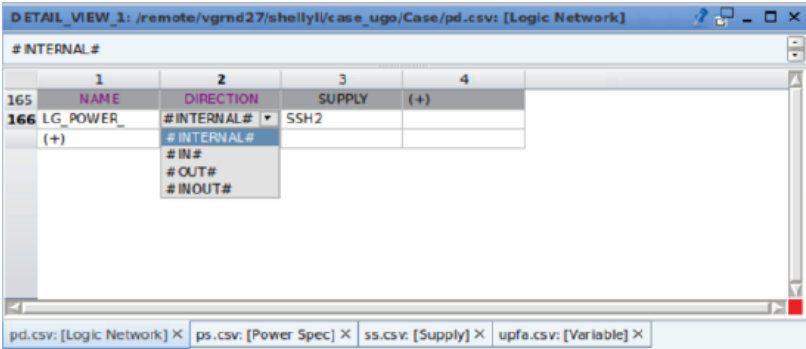


Figure 26 *Updating Direction*

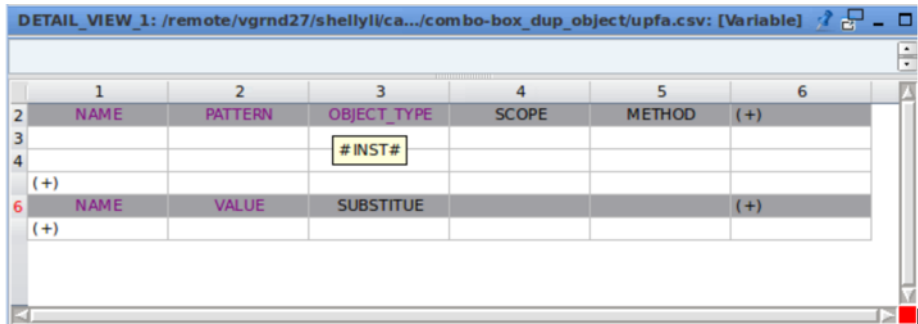


You can click “+” in the last row/column to add a new row/column. You can also drag a column or row header to adjust the order. These actions support undo or redo.

Viewing Default Value in the Column Header’s Tooltip

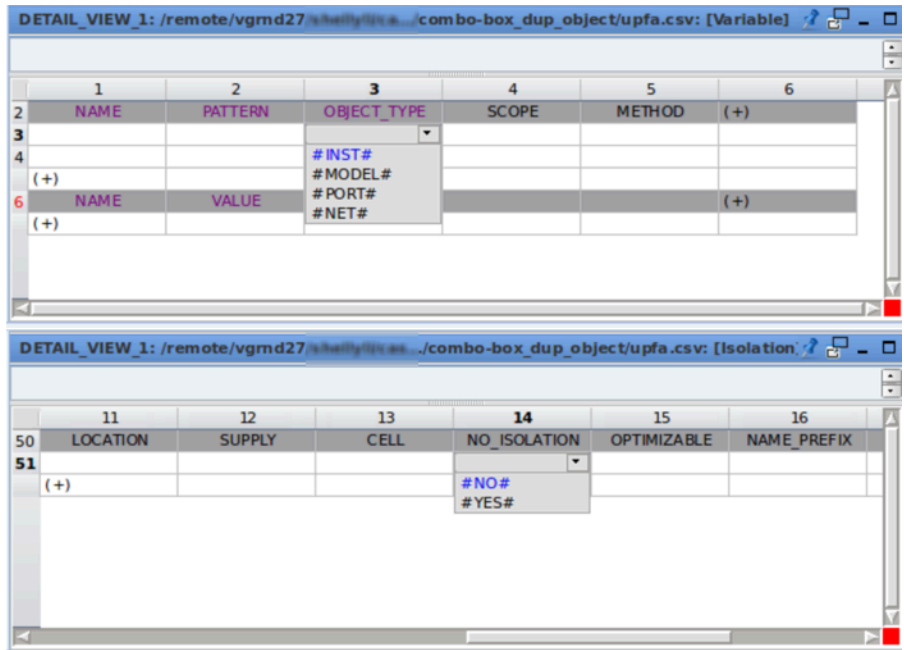
When you hover the mouse over column header, it displays default value in a tooltip only when the current column contains default value.

Figure 27 Viewing Default Value



The default value is displayed in blue color and as a first item in the drop-down list.

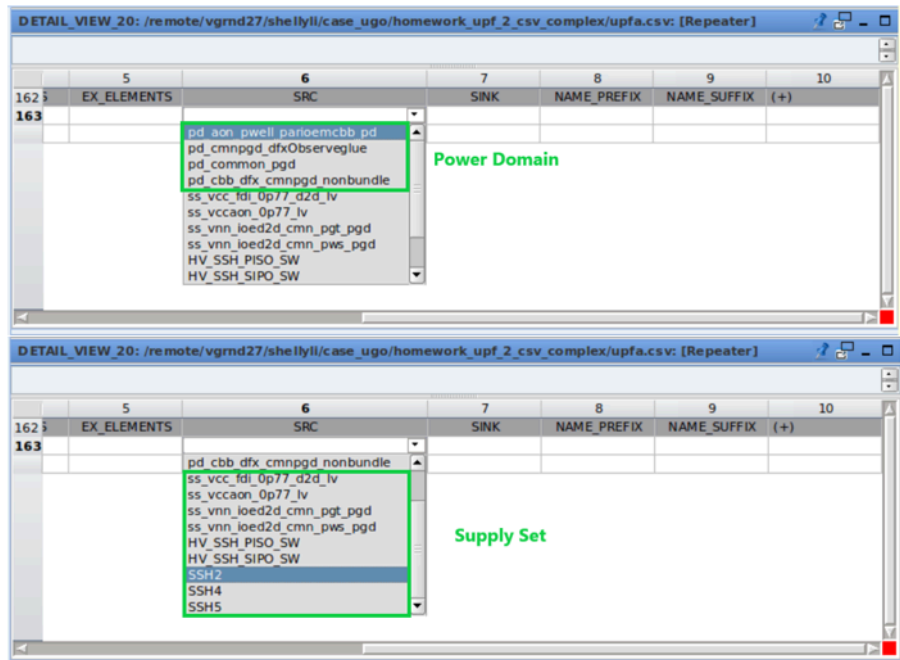
Figure 28 Viewing Default Value



You can double-click an item to start editing. The table displays optional values for an attribute in the drop-down list. You can double-click an attribute and click the drop-down button to view the list of optional values. Optional values include UPF object and keywords.

- These values are sorted by type. For example, attribute “SRC” could be power domain and supply set in the Repeater section. The optional values are listed by type (power domain/supply set) as shown in the following figures:

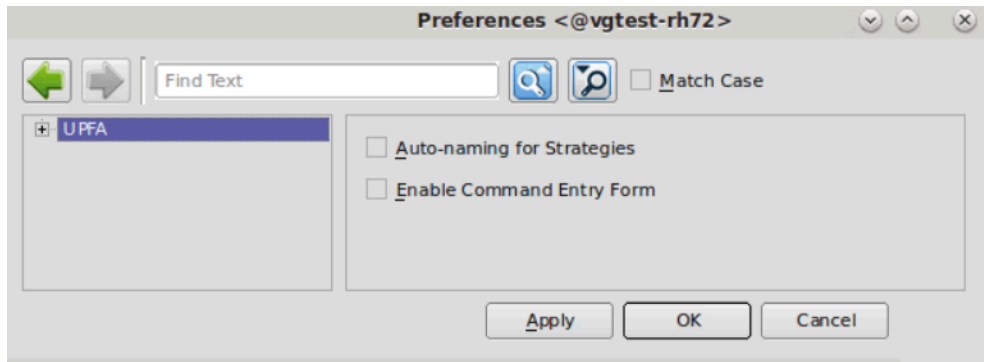
Figure 29 Viewing List of Optional Values in the Drop-Down



If you paste over the existing rows, Verdi expands the rows. The undo and redo operations are supported for this feature.

To enable naming for strategies, click **Tools > Preferences**, select the **Auto-naming for Strategies** check box in *Preferences* dialog, and then click **Apply**. Default status is *off*.

Figure 30 Selecting Auto-Naming For Strategies in Preferences Dialog



When you add a new line, tool sets default name with format `PREFIX_NUMSUFFIX`. Where, numerical suffix (`NUMSUFFIX`) is 0, 1, 2, and so on.

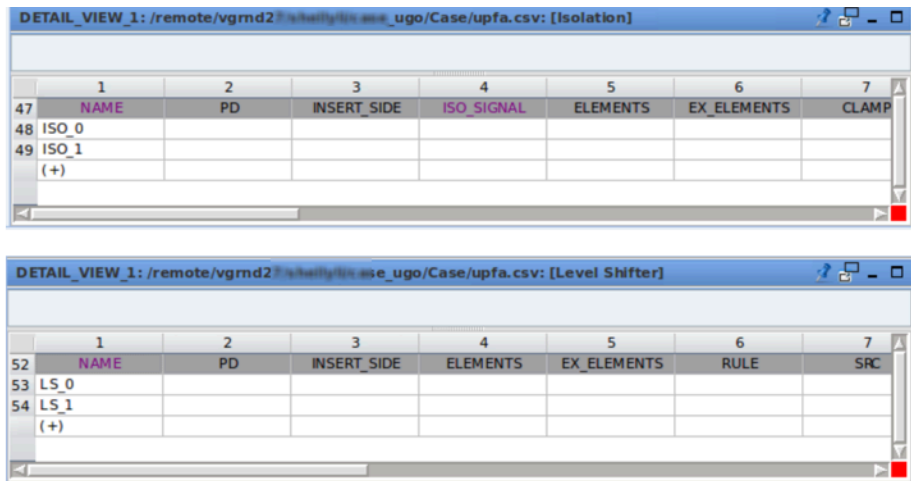
For Isolation, the name format is `ISO_<suffix>`

For Level Shifter, the name format is `LS_<suffix>`

For Retention, the name format is `RET_<suffix>`

For Repeater, the name format is `REP_<suffix>`

Figure 31 Name Format for Isolation and Level Shifter

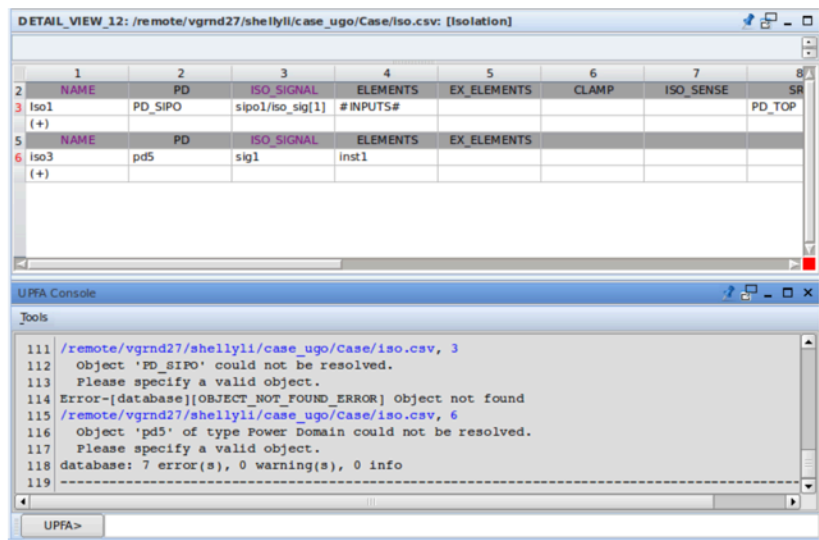


The suffix does not change even when you delete rows. When you do copy/paste, tool does not change the name automatically, general copy/paste rule is retained.

Highlighting Rows With Errors

For rows with errors, the *Detail View* highlights the row header with red color. As shown in the following figure, second row (line 3 in the CSV file) and fifth row (line 6 in the CSV file) have errors. The **Reload All Files** menu option and **Reload File** right-click option checks power intent again. The **Save As** right-click option clears error information.

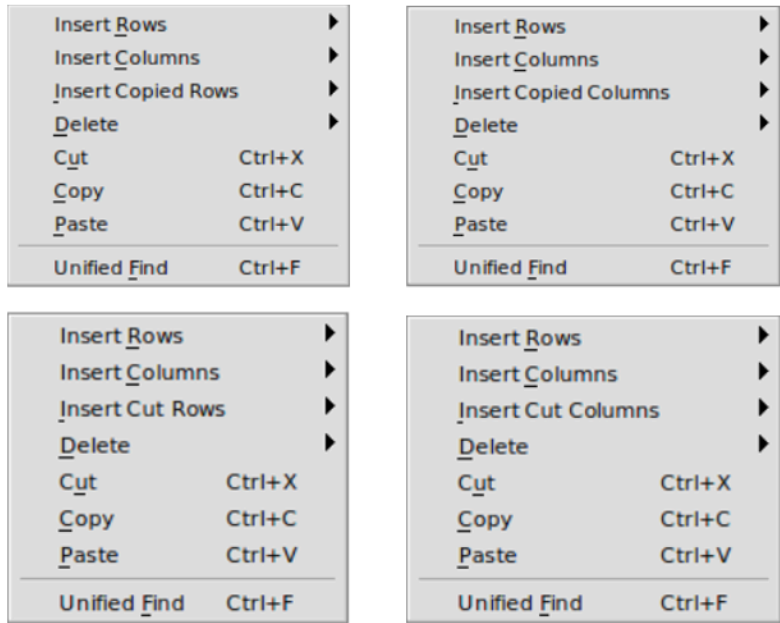
Figure 32 Highlighting Rows With Errors



Supported Right-Click Options

The following right-click options are supported for the selected item:

Figure 33 Right-Click Options



Insert Rows: Allows you to insert multiple rows.

Insert Columns: Allows you to insert multiple columns.

Insert Copied Rows: Allows you to insert copied rows in clipboard.

Insert Copied Columns: Allows you to insert copied columns in clipboard.

Insert Cut Rows: Allows you to insert copied rows.

Insert Cut Columns: Allows you to insert cut columns.

Delete: Allows you to delete row or column of the selected cells.

Cut: Allows you to copy the text of the selected cells to the clipboard. After pasting, the selected item's text is deleted.

Copy: Allows you to copy selected cells text to the clipboard.

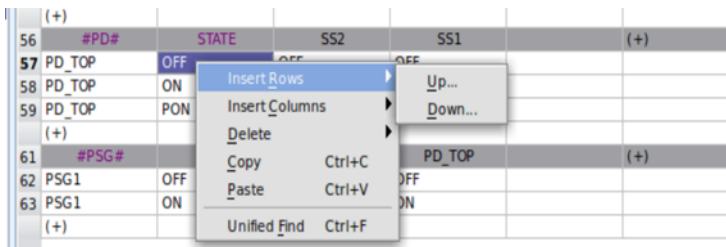
Paste: Allows you to paste text in clipboard to the selected cells.

Unified Find: Refreshes Unified Find interface. You can use Ctrl+F to copy the selected item's text in the *Detail View* to unified find interface as searching string.

Inserting Rows/Columns

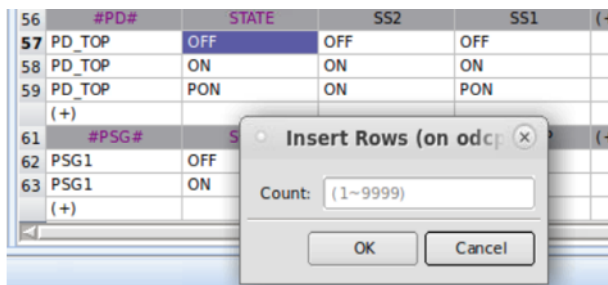
The **Insert Rows/Insert Columns** options allow you to insert multiple rows/columns.

Figure 34 Insert Rows



When you select “Up” or “Down”, a dialog box is displayed to input the number of rows you want to add.

Figure 35 Insert Rows Dialog Box



If you click the **Cancel** or **X** button, nothing happens. The input box only allows you to enter digital number between 1~9999.

If you click the **OK** button, the specified number of rows are added above or below the selected cell.

Figure 36 *Inserting Rows*

	#PD#	STATE	SS2	SS1	(+)
56					
57	PD_TOP	OFF	OFF	OFF	
58					
59					
60					
61	PD_TOP	ON	ON	ON	
62	PD_TOP	PON	ON	PON	
64	#PSG#	STATE	SS1	PD_TOP	(+)

Inserting Copied Rows

The **Insert Copied Rows** option is shown when the clipboard is not empty and the copied cells are complete rows in a table. Complete row means the column number in the clipboard is same as the column number in current selected cell.

Figure 37 *Inserting Copied Rows*

	1	2	3	4	5
49	#SS#	STATE	power	ground	LOGIC_CONDI...
50	SS2	OFF	0	0	a == 0
51	SS2	ON	1.08	0	a == 1
52	SS1	OFF	0		
53	SS1	ON	1.08		
54	SS1	ON	1.08		
55	SS1	ON	1.08		
56	PD_TOP			SS2	SS1
57	PD_TOP				
58	PD_TOP				
59	PD_TOP				
60					
61	PSG1			SS1	PD_TOP
62	PSG1	ON	ON	OFF	ON
63					

When you click “Up” or “Down”, UPFA inserts the copied rows above or below the selected cell’s row.

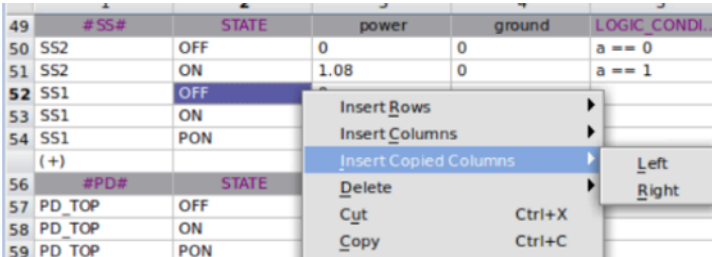
Figure 38 *Inserting Copied Rows*

	1	2	3	4	5	6
49	#SS#	STATE	power	ground	LOGIC_CONDI...	(+)
50	SS2	OFF	0	0	a == 0	
51	SS2	ON	1.08	0	a == 1	
52	SS1	OFF	0			
53	SS1	ON	1.08			
54	SS2	ON	1.08	0	a == 1	
55	SS1	OFF	0			
56	SS1	PON	0.8			
57						
58	#PD#	STATE	SS2	SS1		(+)

Inserting Copied Columns

The **Insert Copied Columns** option is shown when the clipboard is not empty, and the copied cells are complete columns in a table. Complete column means the row number in the clipboard is same as the row number in current selected cell.

Figure 39 Inserting Copied Columns

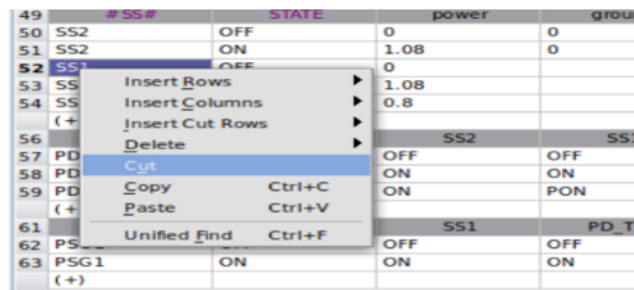


When you clicks “Left” or “Right”, UPFA inserts the copied columns on the left or right of the selected cell’s column.

Support Cut/Paste Actions

When you select one or more cells that can be combined and click the **Cut** option or press Ctrl+X, the text in the cells is saved to the clipboard. When you right-click another cell in the same section and click the **Paste** option, the cut cells’ text is pasted to the region corresponding to the cut rectangle. The text in the origin cells is removed and remain empty.

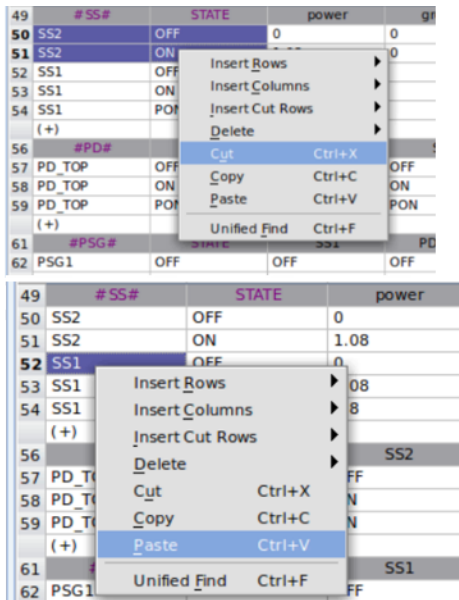
Figure 40 Using Cut and Paste Options



The cut data in the clipboard is cleared after paste is done or if there is any data change in the current section.

If the paste in a section is different with the section of the cut cells, the text in the origin cells is not removed. For example, see the following figures:

Figure 41 *Cut and Paste*



Following is the result:

Figure 42 *Result*

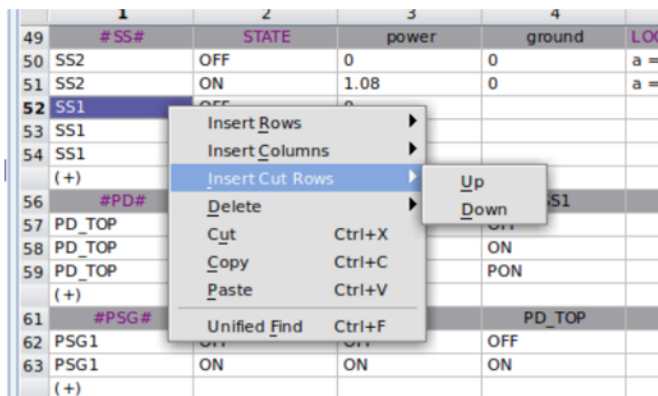
49	#SS#	STATE	power	
50			0	0
51			1.08	0
52	SS2	OFF	0	
53	SS2	ON	1.08	
54	SS1	PON	0.8	
	(+)			
56	#PD#	STATE	SS2	

Support Insert Cut Rows/Columns

You can use the **Insert Cut Rows/Insert Cut Columns** options to reorder multiple rows/ columns in the *Detail View*.

The **Insert Cut Rows** option is shown when the cut cells are complete rows in a table and the cell right clicked on is at the same table with the cut cells. You can click “Up” or “Down” when you hover the mouse pointer on the menu item.

Figure 43 Insert Cut Rows



Selecting a complete row means the selected cells include every cell of the row.

There are two ways to select complete rows:

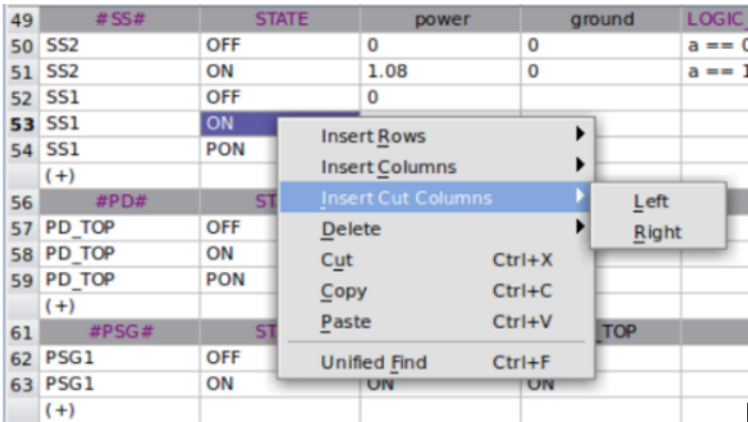
1. Press on and hold the **Ctrl** button on keyboard. Select the rows in the vertical header view.
2. Left click the left top item and drag it in the *Detail View* to select all the cells of the rows.

When you click “Up” or “Down”, UPFA move the cut rows above or below the selected cell’s row. The original rows are removed.

If the cut rows include header rows, the menu item is not shown because header rows should not be moved. If you select the entire table, the menu item is not shown.

The **Insert Cut Columns** option is shown when the selected cut cells are complete columns in a table and the cell right clicked on is at the same table with the cut cells. You can select “Left” or “Right” when you hover the mouse pointer on the menu item.

Figure 44 *Insert Cut Columns*



There are two ways to select complete columns:

1. Press on and hold the **Ctrl** button on keyboard. Select the columns in the horizontal header view.
2. Left click the left top item and drag it in the *Detail View* to select all the cells of the rows.

When you click “Left” or “Right”, UPFA moves the cut columns on the left or right of the selected cell’s column. The original columns is removed.

If you select the entire table, the menu item is not shown.

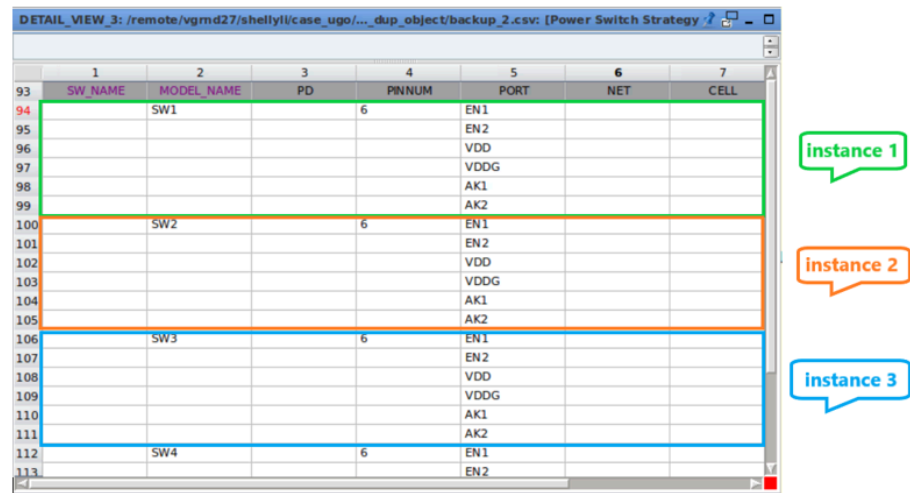
Supported Actions in the Power Switch Strategy Section

In the *Power Switch Strategy* section, the following operations on rows are applied on instance instead of individual row:

- Drag and drop a column or row header to adjust the order
- Insert rows
- Delete rows

Group of rows that correspond to an instance as shown in the following figure.

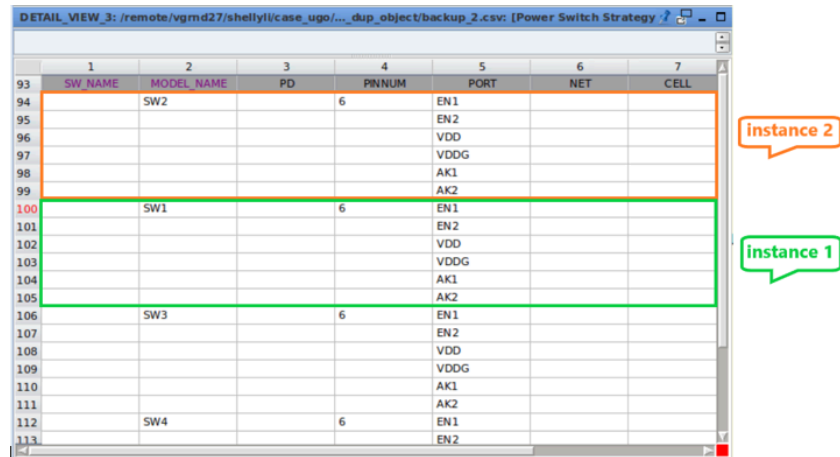
Figure 45 Power Switch Strategy Section



	1	2	3	4	5	6	7
	SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL
93							
94		SW1		6	EN1		
95					EN2		
96					VDD		
97					VDDG		
98					AK1		
99					AK2		
100		SW2		6	EN1		
101					EN2		
102					VDD		
103					VDDG		
104					AK1		
105					AK2		
106		SW3		6	EN1		
107					EN2		
108					VDD		
109					VDDG		
110					AK1		
111					AK2		
112		SW4		6	EN1		
113					EN2		

If you drag the fifth row (header number is 97) to the tenth row (header number is 102), the effect is to drag instance1 to the back of instance2, as shown in the following figure:

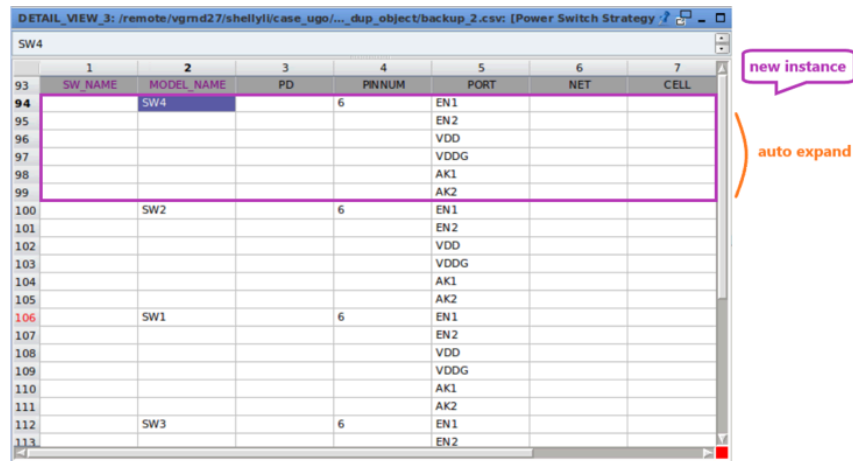
Figure 46 Drag and Drop of Instances



	1	2	3	4	5	6	7
	SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL
93							
94		SW2		6	EN1		
95					EN2		
96					VDD		
97					VDDG		
98					AK1		
99					AK2		
100		SW1		6	EN1		
101					EN2		
102					VDD		
103					VDDG		
104					AK1		
105					AK2		
106		SW3		6	EN1		
107					EN2		
108					VDD		
109					VDDG		
110					AK1		
111					AK2		
112		SW4		6	EN1		
113					EN2		

If you insert one row and select MODEL_NAME in the drop-down, UPFA auto expands other rows of instance as shown in the following figure:

Figure 47 Auto-Expanding Rows of Instances



Deleting rows in an instance deletes entire instance. Deleting selected rows as shown in the following figure, deletes entire instance (new instance and instance 2).

Figure 48 Deleting Rows in an Instance

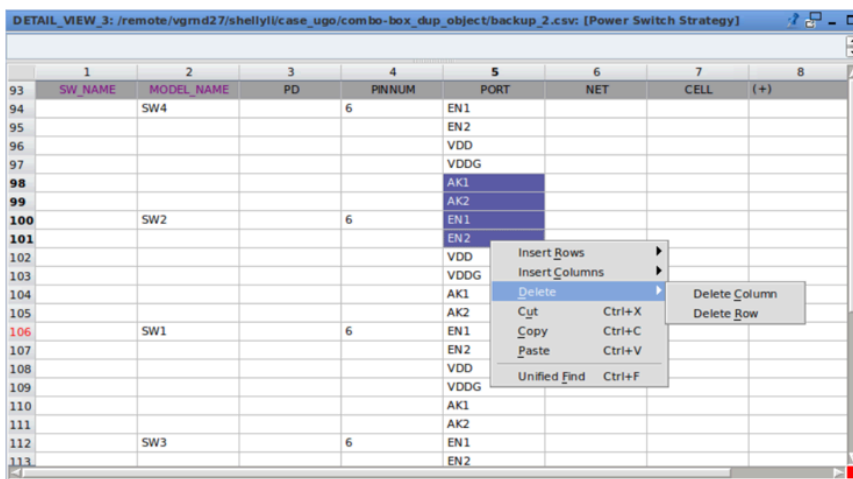


Figure 49 Deleting Rows in an Instance

	1	2	3	4	5	6	7	8
	SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL	(+)
93								
94		SW1		6	EN1			
95					EN2			
96					VDD			
97					VDDG			
98					AK1			
99					AK2			
100		SW3		6	EN1			
101					EN2			
102					VDD			
103					VDDG			
104					AK1			
105					AK2			
106		SW4		6	EN1			
107					EN2			
108					VDD			
109					VDDG			
110					AK1			
111					AK2			
	(+)							

Adding Copied Rows/Columns From Excel

You can use the **Paste**, **Insert Copied Rows**, and **Insert Copied Columns** options when rows/columns are copied from Excel. You can paste any rectangle cells copied from excel.

Figure 50 Cells Copied From Excel

	A	B	C	D
1	1	2	3	
2	4	5	6	
3	7	8	9	
4				
5				

Figure 51 Result

	NAME	PATTERN	OBJECT_TYPE	(+)
7				
8	1	2	3	
9	4	5	6	
10	7	8	9	
	(+)			

You can use the **Insert Copied Rows** option when the copied column number from excel is same as the column number of the selected table.

Figure 52 Cells Copied From Excel

	A	B	C
1	1	2	3
2	4	5	6
3	7	8	9
4	10	11	12
5			

Figure 53 Right-Click Menu Bar > Insert Copied Rows

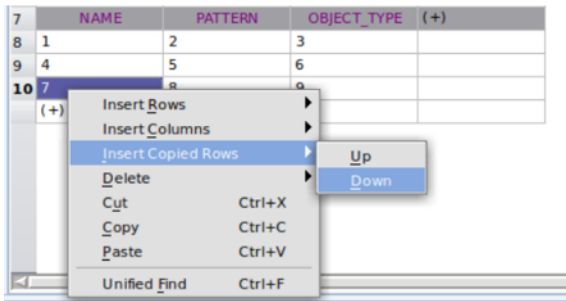


Figure 54 Result

7	NAME	PATTERN	OBJECT_TYPE	(+)
8	1	2	3	
9	4	5	6	
10	7	8	9	
11	10	11	12	
	(+)			

You can use the **Insert Copied Columns** option when the copied row number from excel is same as the row number of the selected table.

Figure 55 Cells Copied From Excel

	A	B	C	D
1	1	2	3	a
2	4	5	6	b
3	7	8	9	c
4	10	11	12	d
5				e

Figure 56 Right-Click Menu Bar > Insert Copied Columns

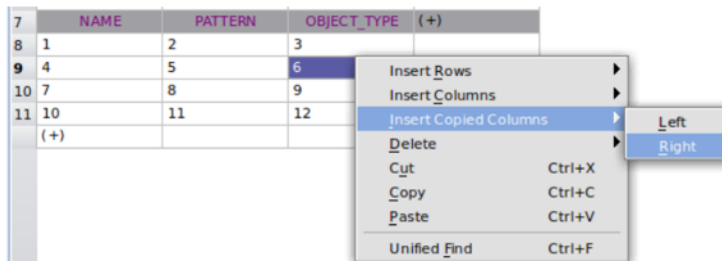


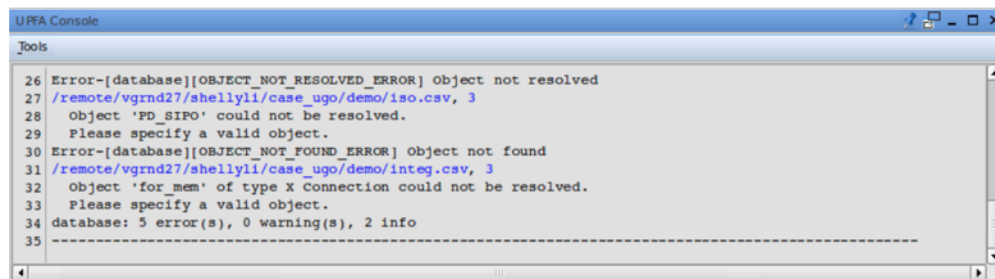
Figure 57 Result

7	NAME	PATTERN	OBJECT_TYPE	a	(+)
8	1	2	3	b	
9	4	5	6	c	
10	7	8	9	d	
11	10	11	12	e	
	(+)				

Console

Console is a text view to display the results of the executed commands. After checking power intent, all error messages are shown in the Console. Hyperlink is shown with “file, line” format. This link jumps to the corresponding position in the *Detail View* and *File List* view.

Figure 58 Console



Support for Special Rules in CSV Format

The Power Intent Editor GUI supports the following string type in CSV format:

- String with new line (\n)
- String with quotation (“)

For both these types, the format of the string saved in the CSV file is changed according to the following rules:

- String is enclosed in quotation in saved file
- Each quotation in original string is saved as double quotations

Original String	CSV String
A,B,C,D,E	"A,B,C,D,E"
A"B	"A""B"
A""B	"A""""B"

- Section name, comment, and special syntax are not supported for multiple line format in the CSV file. For example:

String	Is Correct
[Power Domain]	Yes
"[Power Domain]"	No
// comment	Yes
// "commentcomment"	No
UGO_SET_SCOPE	Yes
"UGO_SET_SCOPE"	No

The Power Intent Editor GUI does not display enclosed quotations and add it when you save it as a CSV file. For example, if a power domain `PD_TOP` has primary handler ("`ss8,ss9`") s shown in the following CSV file, GUI shows it without quotations (`ss8,ss9`) as shown in the following figure:

CSV File

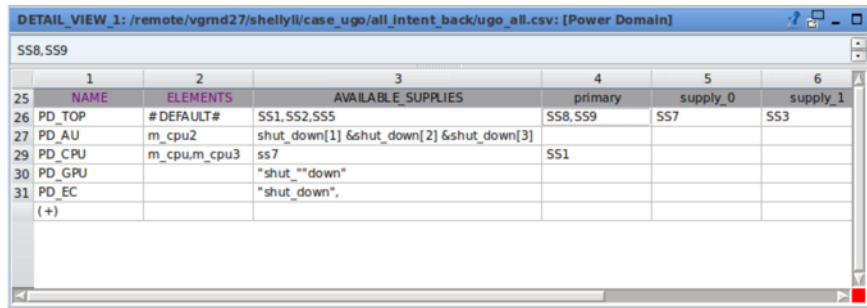
```
[Power Domain]
NAME,ELEMENTS,AVAILABLE_SUPPLIES,primary,supply_0,supply_1
PD_TOP,#DEFAULT#,"SS1,
SS2,SS5","SS8,SS9",SS7,SS3
PD_AU,m_cpu2,"shut_down[1] &
shut_down[2] &
shut_down[3] "
// " I'm comment with quotation
PD_CPU,"m_cpu,m_cpu3",ss7,SS1
```



```
PD_GPU,, ""shut_""  
down""  
PD_EC, ""shut_down"" , "
```

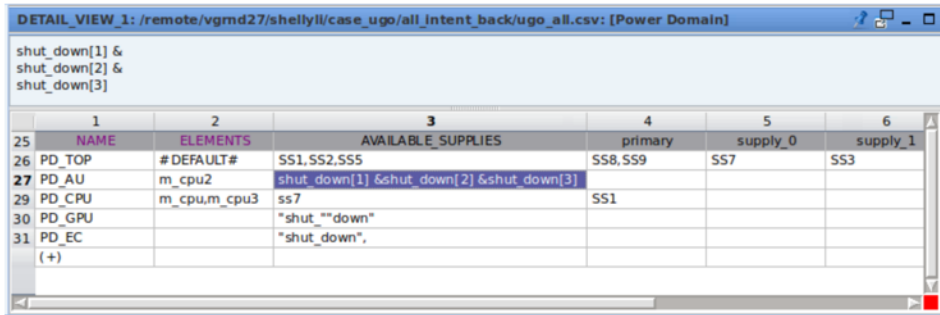
GUI Display

Figure 59 Display of Quotations in GUI



Strings are displayed in a single line in the *Detail View* table. You can get the new line information of the string from the *Text Editing Field* on the top of the *Detail View* as shown in the following figure:

Figure 60 Display of Strings in a Single Line

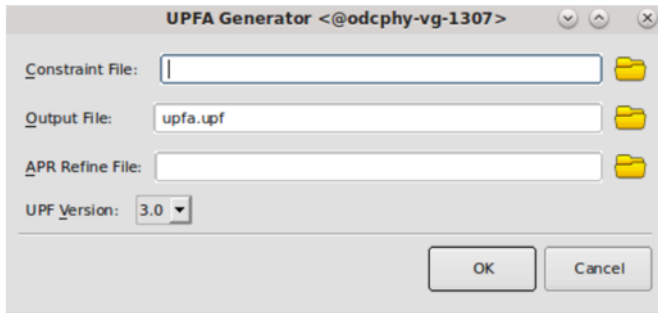


Menu Options

- The **File -> Open File** menu option opens existing CSV file.
- The **File -> Reload All Files** menu option can reload all opened files one time. Reloading CSV file does not support undo/redo.

- The **Edit -> Clear Console** menu option can clear all the messages in the *Console View*.
- The **File -> Generate** menu option allows you to generate UPF file based on the current CSV file. This option displays the *UPFA Generator* dialog box to specify Constraint File (input file), Output File, APR Refine File (output file) and UPF version.

Figure 61 UPFA Generator Dialog Box



If you specify output file in both the *Constraint File* and *Output File* fields at the same time, *Constraint File* will have higher priority than *Output File*. You must at least specify constraint file or output file.

- The **File -> Check Power Intent** menu option calls UPFA commands to check power intent of the opened files.

Tcl Options

The following Tcl options are supported:

- [ugoReloadFile](#)
- [ugoFileViewDump](#)
- [ugoDetailViewDump](#)
- [upfaClearConsole](#)
- [ugoDetailViewSectionMoved](#)
- [upfaSetPreference](#)
- [ugoDetailViewRow](#)
- [ugoDetailViewColumn](#)
- [ugoDetailViewInsertCopiedRows](#)
- [ugoDetailViewInsertCopiedColumns](#)

- [ugoDetailViewCut](#)
 - [ugoDetailViewInsertCutRows](#)
 - [ugoDetailViewInsertCutColumns](#)
 - [ugoShowUPFInVerdi](#)
-

ugoReloadFile

Command

```
ugoReloadFile
```

Description

Reloads a list of files in UGO.

Syntax

```
ugoReloadFile [-file <csv files> | -all]
```

Arguments

`-f <file name>`

Specifies the ugo script to execute when editor is open.

`-file <csv files>`

Specifies files to reload. This option cannot be used with *-all* at the same time.

`-all`

Reloads all opened files. This option cannot be used with *-file* at the same time.

Example

```
ugoReloadFile -file pd.csv ss.csv  
ugoReloadFile -all
```

ugoFileViewDump

Command

```
ugoFileViewDump
```

Description

Dumps file view.

Syntax

```
ugoFileViewDump [-errorStatus]
```

Arguments

-errorStatus

Dumps file view with file's error status.

Example

```
ugoFileViewDump -selected -errorStatus  
ugoFileViewDump -file tree.log -errorStatus
```

ugoDetailViewDump

Command

```
ugoDetailViewDump
```

Description

Dumps table in Detail View.

Syntax

```
ugoDetailViewDump [-errorStatus]
```

Arguments

-errorStatus

Dumps table with row's error status.

Example

```
ugoDetailViewDump -table -errorStatus
```

upfaClearConsole

Command

```
upfaClearConsole
```

Description

Clears console message.

Syntax

```
upfaClearConsole
```

Example

```
upfaClearConsole1
```

ugoDetailViewSectionMoved

Command

```
ugoDetailViewSectionMoved
```

Description

Reorders column/row in section.

Syntax

```
ugoDetailViewSectionMoved [-row | -column]
```

Arguments

-row

Reorders the row.

-column

Reorders the column.

Example

```
ugoDetailViewSectionMoved -row -value 2 3  
ugoDetailViewSectionMoved -column -value 2 3
```

upfaSetPreference

Command

```
upfaSetPreference
```

Description

Allows you to set up preference.

Syntax

```
upfaSetPreference [-autoNamingStrategies <on | off>]  
[-isolationPrefixName <isolationPrefixName>]  
[-retentionPrefixName <retentionPrefixName>]  
[-repeaterPrefixName <repeaterPrefixName>]  
[-levelShifterPrefixName <levelShifterPrefixName>]  
[-isolationSuffixNum <isolationSuffixNum>]  
[-retentionSuffixNum <retentionSuffixNum>]
```

```
[-repeaterSuffixNum <repeaterSuffixNum>]  
[-levelShifterSuffixNum <levelShifterSuffixNum>]
```

Arguments

`-autoNamingStrategies <on | off>`

Enable/disable auto-naming function.

`-isolationPrefixName <isolationPrefixName>`

Specifies isolation prefix name.

`-retentionPrefixName <retentionPrefixName>`

Specifies retention prefix name.

`-repeaterPrefixName <repeaterPrefixName>`

Specifies repeater prefix name.

`-levelShifterPrefixName <levelShifterPrefixName>`

Specifies level shifter prefix name.

`-isolationSuffixNum <isolationSuffixNum>`

Specifies isolation numerical suffix.

`-retentionSuffixNum < retentionSuffixNum>`

Specifies retention numerical suffix.

`-repeaterSuffixNum <repeaterSuffixNum>`

Specifies repeater numerical suffix.

`-levelShifterSuffixNum <levelShifterSuffixNum>`

Specifies level shifter numerical suffix.

Example

```
upfaSetPreference -autoNamingStrategies on \  
-isolationPrefixName iso -isolationSuffixNum 10 \  
-retentionPrefixName ret -retentionSuffixNum 10 \  
-repeaterPrefixName rep -repeaterSuffixNum 10 \  
-levelShifterPrefixName ls -levelShifterSuffixNum 10
```

ugoDetailViewRow

Command

```
ugoDetailViewRow
```

Description

Allows you to add or delete rows in a section.

Syntax

```
ugoDetailViewRow -add [row] {-count [row_count]}
```

Arguments

-add

Indicates it is an insert row operation. Should be followed by *row*.

row

Specifies the row position to insert.

-count

Optional argument. Should be followed by *row_count*. If it is not specified, the insert row count will be 1.

row_count

Specifies the number of rows to insert.

Example

```
ugoDetailViewRow -add 1  
ugoDetailViewRow -add 3 -count 3
```

ugoDetailViewColumn

Command

```
ugoDetailViewColumn
```

Description

Allows you to add or delete columns of the specified table.

Syntax

```
ugoDetailViewColumn -add [column] {-table [table_index]} {-count  
[column_count]}
```

Arguments

`-add`

Indicates an insert column operation. Should be followed by `column`.

`column`

Specifies the column position to insert.

`-table`

Optional argument. Should be followed by `table_index`. If it is not specified, `table_index` will be 1.

`table_index`

Specifies the table index to handle. Should be used with `-table`.

`-count`

Optional argument. Should be followed by `row_count`. If it is not specified, `row_count` will be 1.

`column_count`

Specifies number of columns to insert. Should be used with `-count`.

Example

```
ugoDetailViewColumn -add 1
ugoDetailViewColumn -add 1 -count 3
ugoDetailViewColumn -add 2 -table 1
ugoDetailViewColumn -add 2 -table 1 -count 3
```

ugoDetailViewInsertCopiedRows

Command

`ugoDetailViewInsertCopiedRows`

Description

Allows you to insert rows containing data at a specified row position.

Syntax

```
ugoDetailViewInsertCopiedRows -row [row_pos] -value [copied_data]
```

Arguments

`-row`

Specifies the row position to insert. Should be followed by *row_pos*.

row_pos

The number specifies the row position to insert. Should be used with *-row*.

-Value

Specify the data of the rows to insert. Should be followed by *copied_data*.

copied_data

The string specifies the data of the rows to be insert. Should be used with *-value*.

Example

```
ugoDetailViewInsertCopiedRows -row 1 -value \  
"PSW1%&SW1%&PD_SD1%&6%&  
VDD%&VDD%&POWER_SW1"
```

ugoDetailViewInsertCopiedColumns

Command

```
ugoDetailViewInsertCopiedColumns
```

Description

Allows you to insert columns containing data at a specified column position.

Syntax

```
ugoDetailViewInsertCopiedColumns -column [column_pos] -table  
[table_index] -value [copied_data]
```

Arguments

-column

Specifies the column position to insert. Should be followed by *column_pos*.

column_pos

The number specifies the column position to insert. Should be used with *-table*

-table

Specify the table index to insert. Should be followed by *table_index*.

table_index

Specifies the table index to insert. Should be used with *-table*.

`-value`

Specify the data of the columns to insert. Should be followed by *copied_data*.

`copied_data`

The string specifies the data of the rows to be insert. Should be used with *-value*.

Example

```
ugoDetailViewInsertCopiedColumns -column 1 -value  
"STATE%&SS2%&OFF%&OFF%&  
ON%&ON%&PON%&ON"
```

ugoDetailViewCut

Command

`ugoDetailViewCut`

Description

Allows you to cut the cells of the selected range.

Syntax

`ugoDetailViewCut`

Example

```
ugoDetailViewSelectCell -range {{1 2 0 1} }  
ugoDetailViewCut  
ugoDetailViewPaste -value "SS2%&OFF%&SS2%&ON" -range {5 6 4 5}
```

ugoDetailViewInsertCutRows

Command

`ugoDetailViewInsertCutRows`

Description

Allows you to insert cut rows to a specified row position of the table. Should be used together with Tcl command [ugoDetailViewCut](#).

Syntax

`ugoDetailViewInsertCutRows -row [row_pos]`

Arguments

`-row`

Specifies the row position to insert. Should be followed by `row_pos`.

`row_pos`

The number specifies the row position to insert. Should be used with `-row`.

Example

```
ugoDetailViewSelectCell -range {{1 2 0 4} }  
ugoDetailViewCut  
ugoDetailViewInsertCutRows -row 6
```

ugoDetailViewInsertCutColumns

Command

```
ugoDetailViewInsertCutColumns
```

Description

Allows you to insert cut rows to a specified column position of the table. Should be used together with Tcl command [ugoDetailViewCut](#).

Syntax

```
goDetailViewInsertCutColumns -column [column_pos]
```

Arguments

`-column`

Specifies the column position to insert. Should be followed by `column_pos`.

`column_pos`

The number specifies the column position to insert. Should be used with `-column`.

Example

```
ugoDetailViewSelectCell -range {{0 5 3 4} }  
ugoDetailViewCut  
ugoDetailViewInsertCutColumns -column 1 -table 0
```

ugoShowUPFInVerdi

Command

```
ugoShowUPFInVerdi
```

Description

Invokes Verdi to show the power design of the currently opened CSV files or specified UPF file name.

Syntax

```
ugoShowUPFInVerdi -file [upf_file_name]
```

Arguments

-file

Specifies the UPF file which must be shown in Verdi. If this argument is not specified, Verdi shows the power intent of the currently opened CSV file.

upf_file_name

Allows you to specify the absolute or relative path of the UPF file name.

Example

```
ugoShowUPFInVerdi -file [test.upf]
```

6

Implicit Naming Rule

This chapter describes the naming rule for the tool to generate implicit name for different objects in the following sections:

- [Supply net](#)
- [Supply port](#)
- [Power Switch](#)
- [Isolation Naming](#)
- [Level Shifter Naming](#)

Supply net

Syntax:

```
VDD[_<supply function>]_of[_<supply_set>][_<count>]
VSS[_<count>]
```

Description:

VDD is used as prefix for power and **VSS** is used as prefix for ground.

Verdi adds `_<count>` when there is a name conflict.

Supply port

Syntax:

```
VDD[_<direction>]_<supply function>_of[_<supply
set>][_<count>]
VSS[_<direction>]
```

Description:

VDD is used as prefix for power and **VSS** is used as prefix for ground.

Verdi ignores `_<direction>` when its value is `in`.

Verdi adds `_<count>` when there is a name conflict.

Power Switch

Syntax:

PSW_<supply set>[_<count>]

- **in_**<count>
- **out**
- **ctrl_**<count>
- **ack_**<count>
- **on_**<input supply port>
- **off_**<count>

Description:

You can use **PSW_** as prefix for power switch.

- **in_**<count>: input supply ports
- **out**: output supply ports
- **ctrl_**<count>: control ports
- **ack_**<count>: ack ports
- **on_**<input supply port>: on states
- **off_**<count>: off states

Isolation Naming

`ISO_{pd_name}_<C{clamp_value}/NoIso>_{src_name}_{sink_name}_{object_name}`

Example: `ISO_PD1_C0_SS1_SS2_OUTPUTS`

Level Shifter Naming

`LS_{pd_name}_<H2L/L2H/NoShift>_{src_name}_{sink_name}_{object_name}`

Example: `LS_PD1_H2L_SS1_SS2_IN`

7

Inherit Values for CSV Column

In UPFA, certain empty columns inherit values from previous row, so that you do not need to type duplicated values same as previous line. The following topics describe this section in detail:

- [Power Domain Intent: NAME](#)
- [Power Switch Model: MODEL_NAME](#)
- [Power Switch Strategy: SW_NAME](#)
- [Connection: SCOPE, IS_RTL](#)
- [Port Map: NAME](#)

Power Domain Intent: NAME

Example

NAME	ELEMENTS	POWER	GROUND	STATE NUM	VOLTAGE	LOGIC CONDITION
PD_TOP	#DEFAULT#	VDD	VSS	2	1	PSW[0] & PSW[50]
					0.6	

Is equivalent with:

NAME	ELEMENTS	POWER	GROUND	STATE NUM	VOLTAGE	LOGIC CONDITION
PD_TOP	#DEFAULT#	VDD	VSS	2	1	PSW[0] & PSW[50]
PD_TOP					0.6	

Counter Example

NAME	ELEMENTS	POWER	GROUND	STATE NUM	VOLTAGE	LOGIC CONDIT ION
PD_TOP	#DEFAULT#	VDD	VSS	2	1	PSW[0] & PSW[50]
					0.6	
PD_CPU						

Is NOT equivalent with:

NAME	ELEMENTS	POWER	GROUND	STATE NUM	VOLTAGE	LOGIC CONDIT ION
PD_TOP	#DEFAULT#	VDD	VSS	2	1	PSW[0] & PSW[50]
					0.6	
PD_CPU	#DEFAULT#	VDD	VSS	2	0.6	PSW[0] & PSW[50]

Power Switch Model: MODEL_NAME

Example

MODEL_ NAME	STATE	CONDITION	IN	OUT	ACK	ACK_D ELAY	ACK_CO NTRÖL
SW_M1	ON#	!EN1 & !EN2	VDD	VDDG	ACK1	10	EN1
					ACK2		EN2

Is equivalent to:

MODEL_ NAME	STATE	CONDITION	IN	OUT	ACK	ACK_D ELAY	ACK_CO NTRÖL
SW_M1	ON#	!EN1 & !EN2	VDD	VDDG	ACK1	10	EN1

SW_M1					ACK2		EN2
-------	--	--	--	--	------	--	-----

Power Switch Strategy: SW_NAME

Example

SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL
CMD_SW	SW_M1	PD_VA1	2	VDD	VDD	PSW1CELL
				VDDG	VDDG	

Is equivalent to:

SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL
CMD_SW	SW_M1	PD_VA1	2	VDD	VDD	PSW1CELL
CMD_SW				VDDG	VDDG	

Counter Example:

SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL
CMD_SW	SW_M1	PD_VA1	2	VDD	VDD	PSW1CELL
				VDDG	VDDG	
CMD_SW2						

Is NOT equivalent with:

SW_NAME	MODEL_NAME	PD	PINNUM	PORT	NET	CELL
CMD_SW	SW_M1	PD_VA1	2	VDD	VDD	PSW1CELL

CMD_SW				VDDG	VDDG	
CMD_SW2	SW_M1	PD_VA1	2	VDDG	VDDG	PSW1C ELL

Connection: SCOPE, IS_RTL

Example

SCOPE	PINNUM	HIGH_CONN	LOW_CONN	IS_RTL
m_cpu, m_cpu2	3	VDD	VDD	#NO#
		VSS	VSS	
		VDDC	VDDC	

Is equivalent to:

SCOPE	PINNUM	HIGH_CONN	LOW_CONN	IS_RTL
m_cpu, m_cpu2	3	VDD	VDD	#NO#
m_cpu, m_cpu2		VDD	VDD	#NO#
m_cpu, m_cpu2		VDD	VDD	#NO#

Port Map: NAME

Example

NAME	MAPPING
C54L08L10_Model	{CL UPF_GENERIC_CLOCK}
	{D UPF_GENERIC_DATA}

Is equivalent with:

NAME	MAPPING
C54L08L10_Model	{CL UPF_GENERIC_CLOCK}

C54L08L10_Model	{D UPF_GENERIC_DATA}
-----------------	----------------------

8

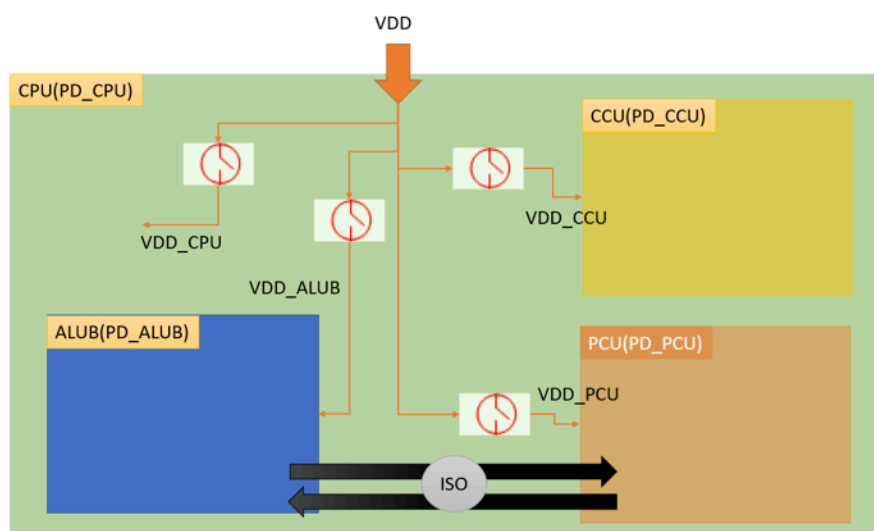
Simple CPU Design With UPFA

You can download the demo testcase from the following location:

```
<verdi_install>/demo/power/upf_demo
```

Consider the design in the following figure. There are four power domains inside modules CPU, PD_CPU, PD_ALUB, PD_CCU, and PD_PCU. The supply source is the power switches connected to VDD from outside.

Figure 62 Simple CPU Design



To describe power intent, you must define four power domains, four power switches, and one external input supply. Following is UPFA format in CSV to describe it:

```
[Power Domain]
NAME,          ELEMENTS,      primary
PD_CPU,        #DEFAULT#,    SW_CPU
PD_ALUB,        i_ALUB,        SW_ALUB
PD_CCU,         i_CCU,        SW_CCU
PD_PCU,         i_PCU,        SW_PCU
```

```
[Supply]
```

```

NAME,      DIRECTION,    power,      ground
SS,        #IN#,         VDD,        VSS
SW_CPU,    #INTERNAL#,    VDD_CPU,    VSS
SW_ALUB,    #INTERNAL#,    VDD_ALUB,    VSS
SW_CCU,     #INTERNAL#,    VDD_CCU,     VSS
SW_PCU,     #INTERNAL#,    VDD_PCU,     VSS

```

```

[Switchable Supply]
OUT,        STATE,              IN
SW_CPU,     #ON#{cpu_sw_ctrl},   SS
SW_CPU,     #OFF#{!cpu_sw_ctrl}
SW_ALUB,    #ON#{alub_sw_ctrl},   SS
SW_ALUB,    #OFF#{!alub_sw_ctrl}
SW_CCU,     #ON#{ccu_sw_ctrl},    SS
SW_CCU,     #OFF#{!ccu_sw_ctrl}
SW_PCU,     #ON#{pcu_sw_ctrl},    SS
SW_PCU,     #OFF#{!pcu_sw_ctrl}

```

Note:

Power/Ground in *[Supply]* is optional. If you do not specify it, UPFA automatically generates the name for it based on the naming rule in Implicit Naming Rule.

You can see there is an isolation between ALUB/PCU. Therefore, you can use the following format to describe it.

```

[Isolation]
NAME,      PD,      ELEMENTS, CLAMP, ISO_SIGNAL, ISO_SENSE, SUPPLY
iso_ALUB, PD_ALUB, #OUTPUTS#, 0,      alub_iso_ctrl, #HIGH#,    SS
iso_PCU,   PD_PCU,  #OUTPUTS#, 0,      pcu_iso_ctrl,  #HIGH#,    SS

```

Then, you can describe the power spec between those components as follows:

```

[Power Spec]
#SS#,      STATE,      power
SS,        HV,         1.08
SS,        LV,         0.8
SW_CPU,    HV,         1.08
SW_CPU,    LV,         0.8
SW_CPU,    CPU_OFF,    #OFF#
SW_ALUB,    HV,         1.08
SW_ALUB,    LV,         0.8
SW_ALUB,    ALUB_OFF,  #OFF#
SW_CCU,    HV,         1.08
SW_CCU,    LV,         0.8
SW_CCU,    CCU_OFF,    #OFF#
SW_PCU,    HV,         1.08
SW_PCU,    LV,         0.8
SW_PCU,    PCU_OFF,    #OFF#

#PSG#,      STATE,      SS, SW_CPU, SW_ALUB, SW_CCU, SW_PCU
PSG_CPU,    HV,         HV, HV,      HV,      HV,      HV

```

```
PSG_CPU, LV, LV, LV, LV, LV, LV
PSG_CPU, CPU_DOWN, HV, CPU_OFF, ALUB_OFF, CCU_OFF, PCU_OFF
```

Before generating the UPF, you must prepare a command file as follows to let UPFA know where your power intent is:

```
set script_dir [file dirname [file normalize [info script]]]
puts "script_dir = $script_dir"
ugo_read_power_intent -csv ${script_dir}/cpu.csv
ugo_check_power_intent
ugo_generate -upf UPF/ugo/cpu.upf -apr UPF/ugo/cpu.apr.upf
ugo_report -error
```

Finally, you can run UPFA to generate the UPF based on your power intent.

```
upfa -batch -f cpu.ugo
```

9

Known Issues and Limitations

The following sections describe limitations and known issues exist in the *UPFA* tool:

- [General Limitations](#)
 - [User-Defined Variable Limitations](#)
-

General Limitations

Following are the limitations:

- UPFA does not support to view/modify related elements in the UPF Generator for the loaded UPF files. Therefore, you can only edit these files in the file editor.
 - UPFA does not support adding power states for the loaded UPF files.
 - For loading existing UPF without HDL, you must follow the limitations of VC-UPF.
-

User-Defined Variable Limitations

Following are the limitations of the user-defined variable:

NAME VALUE

IN #IN#

test_elem test1,test2

temp_elem test3,test4

Limitation : \${test_elem}_\${temp_elem}

Usage of Tcl variable \${...} is supported for any field for which the value is keyword. For example:

```
[Supply]
NAME DIRECTION
SS1 ${IN}
```

For the `VALUE` field, each Tcl variable is decoded and ',' as separator is used to represent a list.

The `${limitation}` is represented as `test1,test2_test3,test4`. The list value is `test1, test2_test3, test4`. This usage is not straight forward, and such usage should be avoided.

Tcl (multi-dimension) array usage is not supported in your CSV.

```
% set t1 "{SS1 SS2} {SS3 SS4}"
{SS1 SS2} {SS3 SS4}
% puts [lindex $t1 0] // not supported
SS1 SS2
% puts [lindex $t1 1] // not supported
SS3 SS4
```


10

Appendix A: Tcl commands to Specify Detail Categories in Constraint File

The following sections describe this feature in detail:

- [Introduction](#)
- [Use Model](#)
- [Tcl Commands](#)
- [Example](#)
- [Specifying Category Using UPF Commands](#)
- [Priority](#)
- [Error and Warning Handling](#)
- [UPF Commands Supported by UPFA](#)
- [Constraint Variables](#)

Introduction

UPFA provides Tcl commands to specify detailed categories in the constraint file. This feature allows you to specify the strategies to generate the UPF designs in a different design flow.

Use Model

Following is the use model:

```
ugo_generate -c constraint_file -v [2.0|2.1|3.0|3.1]
```

You can specify the categories in a constraint file. It is a Tcl-based script file. The following figure shows the detailed format of the constraint file:

Figure 63 Category Format

```
1 //logic physical library are the pre-defined categories
2
3 //user can also define their own defined categories
4
5 create_category category_name -cmds { upf_command_list }
6
7 //specify how to generate upf design
8 gen_upf file.upf -c { category_list } -v version_string
9
```

Tcl Commands

UPFA supports the following new Tcl commands:

create_category

You can use the `create_category` Tcl command to create a new category with specified UPF command list.

Syntax

```
create_category category_name -cmds { upf_command_list }
```

Where, the `-cmds` option allows you to specify the UPF command list.

Example

```
create_category switch -cmds { create_power_switch }
```

Note:

All `create_power_switch` commands are generated in the `ugo_switch.upf` file.

- UPFA decides the order of UPF commands if sub UPF meets the correct power intent. See section [UPF Commands Supported by UPFA](#) for more information. For example, consider the following command:

```
create_category example -cmds { set_isolation create_power_domain }
```

In this example, power domain should be created before `set_isolation`. UPF resolves the order issue to `create_power_domain` first.

Error handling

An error message is issued if the specified command is not in the UPF LRM or not a special keyword type.

gen_upf

You can use the `gen_upf` Tcl command to specify how to generate the UPF design with specified categories.

Syntax

```
gen_upf upf_file_name -c {category_list}
```

Where,

-c: Allows you to specify the category list

Example

```
gen_upf file.upf -c { logical physical library}
```

In this example, three sub UPF files (`ugo_logical.upf`, `ugo_physical.upf`, and `ugo_library.upf`) are loaded in the `file.upf` file.

Key Points to Note

- The sub UPF file name would have prefix `ugo_`.
- The format of file name is `ugo_<category name>.upf`.
- The order of sub UPF in the top wrapper UPF file is same as the order specified in `gen_upf`.

For example: `gen_upf -c {first second third}`

In this example, in the top wrapper upf, `load_upf` order would be `ugo_first.upf`, `ugo_second.upf`, and then `ugo_third.upf`.

- For [Variable] section, the `ugo_variable.upf` file is created and loaded in the beginning.
- For [Script] section, `source` commands are created and written in the top UPF directly.

The script with `#BEGIN#` is sourced after `load_upf ugo_variable.upf`.

The script with `#END#` is sourced at the end of top wrapper UPF file.

Error handling

An error message is issued if the specified `category_name` is not defined.

Note:

UPFA issues the [REDUNDANT_SPECIFIED_CMD] warning message if the specified UPF commands in `create_category` are not used for specific UPF version.

Example

The following figure shows simple example:

Figure 64 Simple Example

```
1 create_category logical -cmds { create_power_domain }
2 create_category physical -cmds { connect_supply_net }
3 create_category library -cmds { use_interface_cell }
4 create_category switch -cmds { create_power_switch }
5 create_category set -cmds { create_supply_set }
6
7 gen_upf file.upf -c { logical physical library }
8 gen_upf name.upf -c { logical switch set }
```

- Based on the specified categories in the constraint file, UPFA generates the following five sub UPF files:

`ugo_logical.upf`, `ugo_physical.upf`, `ugo_library.upf`, `ugo_switch.upf`, and `ugo_set.upf`

UPFA also generates the following two top UPF files:

`file.upf` and `name.upf`

- If there is a [Variable] section in the power intent CSV file, UPFA generates the `ugo_variable.upf` file and loads it inside top wrapper UPF. If there is no [Variable] section in the CSV, then the `ugo_variable.upf` file is not generated.
- If there is a [Script] section in the power intent CSV file, and if the script contains #BEGIN#, it is sourced at the beginning of top wrapper UPF file. If the script contains #END#, it is sourced at the end of the top wrapper UPF file. Otherwise, if there is no corresponding CSV, then sourced commands are not generated inside the top wrapper UPF file.
- The following three sub UPF files are loaded in the top UPF file `file.upf`:

`ugo_logical.upf`, `ugo_physical.upf`, `ugo_library.upf`

With `ugo_variable.upf` for [Variable] and source command for [Script], the paths in [Script] are directly used in top wrapper UPF as shown in the following `file.upf` example:

```
/// file.upf ///
/// gen_upf file.upf -c { logical physical library }
set dir [file dirname [info script]]
load_upf ${dir}/ugo_variable.upf
source pre_script.tcl
load_upf ${dir}/ugo_logical.upf
load_upf ${dir}/ugo_physical.upf
load_upf ${dir}/ugo_library.upf
source post_script.tcl
```

- The following three sub UPF files are loaded in the top UPF file `name.upf`:

`ugo_logical.upf`, `ugo_switch.upf`, `ugo_set.upf`

With `ugo_variable.upf` for [Variable] and source command for [Script], the paths in [Script] are directly used in top wrapper UPF as shown in the following `name.upf` example:

```
/// name.upf ///
/// gen_upf name.upf -c { logical switch set }
set dir [file dirname [info script]]
load_upf ${dir}/ugo_variable.upf
source pre_script.tcl
load_upf ${dir}/ugo_logical.upf
load_upf ${dir}/ugo_switch.upf
load_upf ${dir}/ugo_set.upf
source post_script.tcl
```

Specifying Category Using UPF Commands

You can use the following UPF commands to specify categories:

create_supply_set

For the `create_supply_set` UPF command, define a new format to specify category with different supply function as follows:

Syntax

```
create_supply_set:<keyword>
```

Where, <keyword> can be one of the following:

`no_func`, `func_supply_set`, `func_supply_net`, `others`

The following table describes these keywords in detail:

Keyword	Condition (How UGO detects the command)	Description
no_func	The <code>-function</code> option is not specified. For example: <code>create_supply_set SS0</code>	New supply set definition.
func_supply_set	For all <code>-function</code> options, the <code>net_name</code> argument is a function of a supply set. For example: <code>create_supply_set SS1</code> <code>-function {groundSS0.ground}</code> <code>-function {nwell SS0.power}</code> <code>-function {pwellSS0.ground}</code> <code>-update</code>	Specifies the ground, nwell, pwell connection.
func_supply_net	For all <code>-function</code> options, the <code>net_name</code> argument is a supply net. For example: <code>create_supply_set SS0</code> <code>-function {power VDD}</code> <code>-function {ground VSS}</code> <code>-update</code>	Connection to supply net.
others	For scenario that are not covered by above three. For example: <code>create_supply_set SS0</code> <code>-function {power SS0.power}</code> <code>-function {ground VSS}</code> <code>-update</code>	

Example

```
create_category set -cmds { create_supply_set: no_func }
gen_upf set.upf -c { set }
```

All `no_func` related supply functions specified for the `create_supply_set` command are generated in the `ugo_set.upf` file.

Key Points to Note

- If keyword is not specified, all `create_supply_set` commands are covered.
- An error message is issued if usage is mixed (with no keyword and with keyword) in single category. For example:

```
create_category cat1 -cmds { create_supply_set:no_func
create_supply_set }
```

- UPFA reports a warning if a command does not meet the conditions of any of the specified. For example, if power intent contains following:

```
create_supply_set SS1 -function {ground VDD} -function {nwell
SS0.power} -function {pwell SS0.ground} -update
```

- A warning is issued if there is no category specified: `create_supply_set:others`

connect_supply_net

For the `connect_supply_net` UPF command, define a new format to specify category with different purpose as follows:

Syntax

```
connect_supply_net:<keyword>
```

Where, <keyword> can be one of the following:

`hier_supply_set`, `hier_supply_net`, `top_port`, `others`

The following table describes these keywords in detail:

Keyword	Condition (How UGO detects the command)	Description
<code>hier_supply_set</code>	All port_list are instance pins, and net_name is a function of a supply set. For example: <code>connect_supply_net -ports xxx/yyy/vdd ss.power</code>	Connection to IP that is instantiated in RTL.
<code>hier_supply_net</code>	All port_list are instance pins, and net_name is a supply net. For example: <code>connect_supply_net -ports xxx/yyy/vdd vdd</code>	Connection to IP that is added during layout.
<code>top_port</code>	port_list is a supply port, and net_name is a supply net. For example: <code>connect_supply_net -ports vdd vdd</code>	Connection from the top supply port (of current scope) to a supply net.
<code>others</code>	For scenario that are not covered by above three. For example: <code>connect_supply_net -ports vdd ss.power</code>	

Note:

If the purpose is not specified, all `connect_supply_net` commands are generated together in the `ugo_csn.upf` file.

Example

```
create_category csn -cmds { connect_supply_net:top_port}  
gen_upf csn.upf -c { csn}
```

All `connect_supply_net` commands with `top_port` purpose are generated together in the `ugo_csn.upf` file.

Key Points to Note

- If keyword is not specified, all `create_supply_net` commands are covered.
- An error is issued if usage is mixed (with no keyword and with keyword) in single category. For example:

```
create_category cat1 -cmds { connect_supply_net:top_port  
connect_supply_net }
```

- UPFA reports a warning message if a command specified does not meet conditions of any of specified. For example, if power intent contains following:

```
connect_supply_net -ports p1/VDD SS1.power
```

A warning is issued if no category is specified:

```
connect_supply_net:hier_supply_set
```

Priority

If you specify both `-upf` option and `gen_upf` command in a constraint file, the `gen_upf` command has the higher priority.

Error and Warning Handling

- In the `gen_upf` command, if certain UPF command is specified multiple times in the `category_list`, UPFA issues a warning message and generates UPF design without any error recovery.
- UPFA issues the following warning message if power intent CSV defines some spec (for example, PSW), but corresponding UPF command (for example, `create_power_switch`) is not mapped to any category, including the command with keyword.

Example:

```
Warning-[generator][NON_COVERED_CMD] UPF command
is not mapping to any category
UPF command 'connect_supply_net:top_port' is not
specified in any category.
Consider specifying a category that includes the
command.
```

- UPFA issues the following warning message when `create_category` specifies some UPF commands, but when UPF is generated corresponding power intent CSV does not exist.

```
Warning-[generator][REDUNDANT_SPECIFIED_CMD]
Specified UPF command in category is not in power
intent
Redundant UPF command 'create_logic_net' specified
which is not in the power intent.
```

To disable this warning, specify the following command in the constraint file:

```
set disable_redundant_specified_cmd_warning 1
```

UPF Commands Supported by UPFA

The following commands are supported by UPFA to generate UPF command. You can specify these commands inside `create_category`. All sub UPF files follow this order to generate UPF commands.

- `[upf_version]`
- `[set tclvar]`
- `[source pre-script]`
- `[load_upf]`
- `find_objects`
- `set_design_attributes`
- `create_supply_port`
- `create_supply_net`
- `connect_supply_net:top_port`
- `create_supply_set:no_func`
- `create_power_domain`

- `create_supply_set:func_supply_set`
- `create_supply_set:func_supply_net`
- `create_supply_set:others`
- `set_port_attributes`
- `set_related_supply_net`
- `create_logic_port`
- `create_logic_net`
- `connect_logic_net`
- `create_power_switch`
- `map_power_switch`
- `create_power_state_group`
- `add_power_state`
- `create_pst`
- `add_pst_state`
- `add_state_transition`
- `describe_state_transition`
- `set_isolation`
- `map_isolation_cell`
- `set_level_shifter`
- `map_level_shifter_cell`
- `use_interface_cell`
- `set_retention`
- `map_retention_cell`
- `set_repeater`
- `associate_supply_set`
- `connect_supply_net:hier_supply_set`
- `connect_supply_net:hier_supply_net`

- `connect_supply_net:others`
- `[source post-script]`

Key Points to Note

- `[set tclvar]`, `[source pre-script]`, `[source post-script]` are fixed and cannot be specified in any category. These are generated in the top-wrap UPF file.
- The `create_supply_set` command dominates `create_supply_set:keyword` and `connect_supply_net` dominates `connect_supply_net:keyword`.

```
create_category cat { create_supply_set } is equal to
create_category cat { create_supply_set:no_func
  create_supply_set:func_supply_set create_supply_set:func_supply_net
  create_supply_set:others }
```

```
create_category cat { connect_supply_net } is equal to
create_category cat { connect_supply_net:top_port
  connect_supply_net:hier_supply_set connect_supply_net:hier_supply_net
  connect_supply_net:others }
```

- The `use_interface_cell` command dominates the `map_isolation_cell` and `map_level_shifter_cell` commands.
- The `create_power_state_group` and `describe_state_transition` commands are supported only for UPF 3.0. The `create_pst` and `add_pst_state` commands are supported only for UPF 2.0.

Constraint Variables

You can set variables in the constraint file to generate different UPF syntax.

set all_refer_supply_set_function 1

1. `[Supply] power,ground,newll,pwell` can reuse each other

Sample:

[Power Domain Intent]					
NAME	ELEMENTS	POWER	GROUND	NWELL	PWELL
PD_VA0	DEFAULT	VDDC	VSS	VDDC	VSS
PD_VA1	u_VA1_TOP	VDDVA1	VSS	VDDC	VSS

```
create_category create_supply_set -cmds { create_supply_set:no_func
  create_supply_set:func_supply_set }
create_category create_supply_set update -cmds
{ create_supply_set:func_supply_set }
```

Expected Output:

```
create_supply_set.upf

create_supply_set SS_PD_VA0
create_supply_set SS_PD_VA0 -function {nwell SS_PD_VA0.power}
  -function {pwell SS_PD_VA0.ground} -update
create_supply_set SS_PD_VA0 {ground SS_PD_VA0.ground} -function {nwell
  SS_PD_VA0.power} -function {pwell SS_PD_VA0.ground}

create_supply_set_physical.upf

create_supply_set SS_PD_VA0 -function {power VDDC} -function {ground
  VSS} -update
create_supply_set SS_PD_VA1 -function {power VDDVA1} -update
```

Rules:

- If the same supply net is specified multiple times in the [Power Domain Intent] table, generated `create_supply_set` commands should refer to that object by supply net name only once. The rest of the occurrences are referred to as supply set function.
- Single `create_supply_set` command should not mix supply net name and supply set function. Instead, generate separate `create_supply_set` commands for each kind to be directed to separate sub UPF.
- The `create_supply_set` command without `-update` should not refer to its own function. Instead, generate two `create_supply_set` commands, one for definition and the other for updating with function.
- When choosing the occurrence to use the supply net name for rule 1, scan power and ground in the table and choose the first occurrence, and then scan nwell and pwell and choose the first occurrence.

2. [Connection]: IS_RTL uses supply set function

Sample:

[Connection]				
SCOPE	PINNUM	HIGH_CONN	LOW_CONN	IS_RTL
m_cpu, m_cpu2	3	VDD	VDD	#YES#
		VSS	VSS	
		VDDC	VDDC	
m_inst_not_in_RTL	1	VDDC	VDDC	#NO#

Generated UPF:

```
foreach i [concat m_cpu1 m_cpu2] {
    lappend __ugo_connection__VDDC_VDDC_0 ${i}/VDDC
}
connect_supply_net SS_PD_VA1.power -ports
    ${__ugo_connection__VDDC_VDDC_0}
foreach i [concat m_cpu1 m_cpu2] {
    lappend __ugo_connection__VDD_VDD_1 ${i}/VDD
}
connect_supply_net SS_PD_VA0.power -ports
    ${__ugo_connection__VDD_VDD_1}
foreach i [concat m_cpu1 m_cpu2] {
    lappend __ugo_connection__VSS_VSS_2 ${i}/VSS
}
connect_supply_net SS_PD_VA0.ground -ports
    ${__ugo_connection__VSS_VSS_2}

foreach i [concat m_inst_not_in_RTL] {
    lappend __ugo_connection__VDDC_VDDC_3 ${i}/VDDC
}
connect_supply_net VDDC -ports ${__ugo_connection__VDDC_VDDC_3}
```

set apr_spec <0 | 1 | 2 | 3 | 4>

value = 0, nwell/pwell/ground will be in RTL, power will be in APR

value = 1, nwell/pwell will be in RTL, power/ground will be in APR

value = 2, all (nwell/pwell/ground/power) will be in APR

value = 3, all (nwell/pwell/ground/power) will be in RTL

value = 4, power/ground will be in RTL, nwell/pwell will be in APR

set no_implicit_supply_function 1

Tool does not generate unspecified implicit supply function.

set flatten_connection 1

For [Connection], tool flattens foreach style to generate
connect_supply_net/connect_logic_net.

set using_supply_set_for_domain_primary 1

If CSV specified power domain or power domain primary for supply set attribute, tool finds exact supply set and generate in UPF syntax.

set disable_opt_pst 1

Disables power state table optimization feature.

set strict_lrm 1

Generates UPF which strictly follows UPF LRM.

11

Appendix B: ISOGEN Native Support in UPFA

The following sections describe the ISOGEN native support in UPFA in detail:

- [Introduction](#)
- [Prerequisites](#)
- [Syntax](#)
- [Inputs](#)
- [Usage Model](#)
- [Output](#)
- [Isolation Configuration File Format](#)
- [Hierarchical Isolation Configuration File Format](#)

Introduction

The ISOGEN tool is a set of scripts that operate on post-process of a VCLP run on an RTL+UPF design that has no isolation strategies. Based on the `ISO_STRATEGY_MISSING` violations and other options, ISOGEN creates isolation strategies in the UPF that are used in the source UPF. The ISOGEN tool feature and functionality is natively supported in UPFA.

Prerequisites

RTL design should be ready in UPFA (using `ugo_vlog_analyze`, `ugo_elaborate`, or `ugo_include_vclp_script`)

The template UPF file should be passed by VCLP.

Syntax

```
ugo_optimize_power_intent  
-isogen
```

```
[-upf <template.upf>]
[-iso_config <iso config file>]
[-dump_cross_domain <iso info file>]
[-dump_iso_domain <iso info file>]
[-v 2.0|3.0]
[-map_iso]
[-allow_unconnected]
[-src_sink_iso]
[-strict_lrm]
[-iso_place <source|sink>]
[-hetero_iso]
[-wrapper_upf <generated wrapper UPF file name>]
-hier_isogen
[-upf <template.upf>]
[-hier_iso_config <hierarchical iso config file>]
[-v 2.0|3.0]
[-map_iso]
[-strict_lrm]
[-iso_place <source|sink>]
[-hetero_iso]
[-wrapper_upf <generated wrapper UPF file name>]
```

Description

- **-isogen:** Enables the ISOGEN flow.
- **-upf:** Specifies the UPF file input to the ISOGEN flow.
- **-iso_config:** Specifies the isolation configuration file. The configuration file is in JSON format.
- **-dump_cross_domain:** Generates all cross-domain settings required for the UPF. The output is saved to *<iso info file>*.
- **-dump_iso_domain:** Generates all required isolation domain settings required for the UPF. The output is saved to *<iso info file>*.
- **-v:** Specifies the generated UPF version for the isolation command. The supported versions are 2.0 and 3.0.
- **-map_iso:** Generates the isolation command along with *map_isolation_cell*.
- **-src_sink_iso:** Generates *-source/-sink* type isolation policy.
- **-strict_lrm:** Generates UPF by strictly following the UPF LRM.
- **-wrapper_upf:** Specifies the output wrapper UPF file name.
- **-hier_isogen:** Enables the hierarchical ISOGEN flow. You cannot use this option with *-isogen*.

- `-iso_place`: Specifies source or sink. Allows you to set up the preference of inserting isolation at the source domain or sink domain.
- `-hetero_iso`: Handles hetero isolation fanout scenario and inserts isolation at the sink domain.

Inputs

- `iso.config`: Configuration file (JSON format) to specify isolation strategy information for ISOGEN.
- `hier_iso.config`: Configuration file (JSON format) to specify isolation strategy information for hierarchical ISOGEN.
- `template.upf`: The UPF file processed to generate isolation strategy. It should be passed by VCLP.

Usage Model

Use Model-1

```
ugo_vlog_analyze ... ### import design by UPFA
ugo_elaborate ... ### elaborate design by UPFA
ugo_optimize_power_intent -isogen -iso_config iso.config -upf
template.upf
```

Use Model-2

```
ugo_include_vclp_script -f to_vclp.tcl      ### the script should contains
analyze, elaborate, read_upf
ugo_optimize_power_intent -isogen -iso_config iso.config
```

Use Model-3

```
ugo_include_vclp_script -f to_vclp.tcl      ### the script should contains
analyze, elaborate
ugo_optimize_power_intent -isogen -upf template.upf -iso_config
iso.config
```

Output

- `<template>_isolation.upf`
With missing isolation strategy commands in the UPF.
- `<template>_wrapper.upf`

With sourced `template.upf` and `template_isolation.upf`.

```
source template.upf
source template_isolation.upf
```

- For isolation supply, `-isolation_supply` is used in UPF 3.0 and `-isolation_supply_set` is used in UPF 2.0.
- The `map_isolation_cell` command is generated along with each isolation strategy with the following rule:

```
map_isolation_cell <iso name> -domain <domain> -lib_cells <cell name>
```

Where, **<cell name>** is the following under specific criteria:

- `$FW_AND_CELL`: Default value
- `$FW_OR_CELL`: When the isolation's clamp value is 1
- `$FW_CLOCK_CELL`: When the isolation has `clock` type element (should be defined in `clock.list`)

You must set the above three TCL variables before using the `isolation.upf` file.

```
set FW_AND_CELL <iso cell for clamp value 0>
set FW_OR_CELL <iso cell for clamp value 1>
set FW_CLOCK_CELL <iso cell for clock>
```

Isolation Configuration File Format

You must specify isolation signal, isolation clamp, and isolation sense under specific condition to generate complete isolation strategy commands. The isolation configuration file helps UPFA to complete the missing isolation strategy commands. The following code shows the format of the isolation configuration file:

```
{
  "spa_clamp_files": {
    "0": [ "spa_clamp_0.list" ],
    "1": [ "spa_clamp_1.list" ],
    "Z": [],
    "latch": []
  },
  "clamp_value_info_files": {
    "0": [ "isolation_clamp_0.list" ],
    "1": [ "isolation_clamp_1.list" ],
    "Z": [],
    "latch": []
  },
  "diff_ctrl_info_files": [ "diff_ctrl.list" ],
  "clock_info_files": [ "clock.list" ],
}
```

```
"include_info_files": [ "include.list" ],
"exclude_info_files": [ "exclude.list" ],
"isolation_info": [
  {
    "include_files": [ "isolation_template.info" ]
  }
]
```

The config template is at the following location:

`$VERDI_HOME/share/ugo/etc/iso_gen`

The isolation configuration file (`iso.config`) is a JSON file. It contains the following ISOGEN information:

- `clamp_value_info_files`

This field specifies a list of files. The files contain the signals to specify the clamp value.

Format of the file: only the signal name is required, and regex is supported:

```
abc_tf_en
.*testing.*\[0\]
```

The file is a list of signals (wildcard supported) that are intended to be clamped at values. For example, IP can include signals that must be isolated to 1 inside `pcf20_aon_clamp1.list` and `isolation_clamp1.list`.

- `diff_ctrl_info_files`

Specifies a list of files.

You can list the signals in a file to let the flow know the signals to isolate with a different control signal. Only signal name is required, and regex is supported.

```
abc_tf_en
.*testing.*\[0\]
```

- `isolation_info`

You can specify two types of JSON objects in this block. One is to specify the included file and the other is to specify the information explicitly. The key word is `include_files`. Once the keyword is present, it is the first kind of JSON object. Other keywords (like `source_domain`, `sink_domain`, and so on) are ignored.

- `spa_clamp_files`

This field specifies a list of files. The files contain the signals to specify the `set_port_attributes` clamp value.

Format of the file: only the signal name is required, and regex is supported:

```
abc_tf_en
.*testing.*\[0\]
```

- `include_files`

Specifies a list of files that represent the isolation information with the following format:

```
<source power domain>,<destination power domain>, <isolation
control signal>,<isolation supply>,<isolation location>,<duplicate
or not>,<isolation sense>,<source or sink>, <isolation clamp>,
<isolation cells>
```

The `duplicate` keyword should be specified. If keyword is present, the signals specified in `diff_ctrl_info_files` apply isolation control specified with `duplicate`. The source or sink keyword should be specified if it must be forced to insert at the source/sink domain.

- `clock_info_files` (optional)

Specifies a list of files.

Clock signals (wildcard supported) and isolation cells are used for clock signals.

The clock signals apply `Cell: $FW_CLOCK_CELL`

The clamp value 1 applies `Cell: $FW_OR_CELL`

The clamp value latch applies `Cell: $FW_LATCH_CELL`

By default, `Cell: $FW_AND_CELL` is applied

Only signal name is required, regex is supported:

```
abc_tf_en
.*testing.*\[0\]
```

- `include_info_files` (optional)

Specifies a list of files. To force isolation on certain signals as desired by IPs, ISOGEN includes the signals in the file under the generated isolation strategy. The signals should be defined with the complete hierarchy path with respect to the top module.

Following is the format:

```
<source power domain>,<sink power domain>,<clock or normal>,<clamp
value>,<signal with full hierarch path>
```

Example: `PD1,PD2,normal,0,unit1/sigA`

The `unit1/sigA` element is added to the elements list of the generated isolation command with matching cross domain as follows:

```
<source power domain>,<sink power domain>, and the clamp_value <clamp value>
```

If no isolation command is generated with the specified constraint, the signal is ignored:

```
(1) crossing-domain <src domain>, <sink domain> and (2) clamp value  
<clamp value>
```

- `exclude_info_files` (optional)

Specifies a list of files.

The format of the files should be the list of signals (wildcard supported).

```
SigA  
.*SigB_.*
```

When this list is provided, ISOGEN excludes the signals included for isolation.

Note:

- The flavor of regular expression is Python style.
- Matching mechanism is performed. For example, signal name is

```
top/a1/a2/pdd_exx_pert
```

The pattern `pdd_exx_pert`, `pdd_exx_p.*` matches.

The pattern `pdd_exx` does not match.

Hierarchical Isolation Configuration File Format

You must specify partition scope, partition UPF, and partition isolation configuration file to generate partition isolation strategy. The following code shows the format of the isolation configuration file:

```
{  
  "PARTITION": [  
    {"SCOPE": "paracemem", "UPF": "${UPF_INSTALL_DIR}/paracemem.upf",  
    "ISO_CONFIG": "config_iso/isogen_paracemem.config",  
    "ISO_CONSTRAINT": "config_iso/isogen_paracemem.constraint"},  
    {"SCOPE": "parcfmia", "UPF": "${UPF_INSTALL_DIR}/parcfmia.upf",  
    "ISO_CONFIG": "config_iso/isogen_parcfmia.config"},  
    {"SCOPE": "parcse", "UPF": "${UPF_INSTALL_DIR}/parcse.upf",  
    "ISO_CONFIG": "config_iso/isogen_parcse.config"}  
  ]  
}
```

The hierarchical isolation configuration file (`hier_iso.config`) is a JSON file. It contains the following ISOGEN information:

- `SCOPE`: Specifies the partition name required to generate isolation.
- `UPF`: Specifies the partition UPF file for generating the partition wrapper UPF file.
- `ISO_CONFIG`: Specifies the partition isolation configuration file for generating isolation.
- `ISO_CONSTRAINT`: Optional. Specifies the partition isolation constraint file for generating isolation.